École Supérieure Privée d'Ingénierie et de Technologies
ESPRIT

Rapport de Projet de Fin d'Études

Présenté en vue de l'obtention du
Diplôme National d'Ingénieur en Informatique
Option : **TWIN** (Technologies du Web et de l'Internet)

Conception et Développement d'une Plateforme SaaS Multi-Boutiques Intelligente pour le E-commerce

Sellio : plateforme de création, personnalisation et gestion de boutiques en ligne intégrant des services d'intelligence artificielle

---

Réalisé par : **Rania KHEDHRI**

Encadrant académique : *M. Talel Zouari*
Encadrant professionnel : M. Malek NAOUAR

Réalisé au sein de **Antigone**

Année universitaire 2025 – 2026

Remerciements

La réalisation de ce projet marque l'aboutissement de plusieurs mois de travail, d'apprentissage et de réflexion. Elle n'aurait pas été la même sans les personnes qui m'ont accompagné et soutenu tout au long de cette expérience.

Je tiens tout particulièrement à remercier mon encadrant académique, ***M. Talel Zouari***, pour son suivi, sa disponibilité et la pertinence de ses conseils.

Mes remerciements vont également à mon encadrant professionnel et directeur de l'entreprise Antigone, **M. Malek Naouar**, pour la confiance qu'il m'a accordée dès le début de cette aventure. La liberté qui m'a été donnée, ainsi que ses orientations au fil du projet, m'ont permis d'aller au-delà d'une simple boutique en ligne et de construire progressivement une plateforme multi-boutiques plus ambitieuse.

Je remercie également toute l'équipe de **Antigone** pour son accueil, les échanges, les conseils et les moments partagés. Évoluer dans cet environnement professionnel m'a permis d'enrichir mes compétences, mais aussi d'aborder ce projet avec une vision plus concrète des enjeux et des exigences du monde professionnel.

Enfin, j'adresse mes remerciements aux membres du jury pour le temps consacré à la lecture et à l'évaluation de ce travail. J'espère que ce rapport saura retranscrire aussi fidèlement que possible le chemin parcouru, les difficultés rencontrées, les choix effectués et tout ce que ce projet m'a permis d'apprendre.

À toutes celles et ceux qui ont contribué, de près ou de loin, à cette expérience : merci.

Dédicace

À ma mère, ***Neyra****,*
pour son amour, sa patience et son soutien constant,
et pour avoir toujours cru en moi, même dans les moments de doute.

À mon père, ***Moncef****,*
pour ses sacrifices, ses encouragements et les valeurs qu'il m'a transmises,
qui m'ont permis d'avancer avec détermination tout au long de mon parcours.

À mon grand frère, ***Bairem****,*
pour sa présence, son soutien et tous les conseils partagés au fil des années.

À ma petite sur, ***Yosr****,*
pour son affection, sa présence et cette énergie qui apporte toujours
une touche de légèreté dans les journées les plus chargées.

À ***toute ma famille****,*
pour sa présence, son affection et ses encouragements,
et pour avoir toujours occupé une place essentielle dans mon parcours.

À mes amis,
pour leurs encouragements, leur présence et tous les beaux moments partagés,
qui ont rendu ce parcours plus léger et plus agréable.

À toutes les personnes qui ont partagé une partie de ce chemin avec moi,
je dédie ce travail avec toute mon affection.

Rania Khedhri

Table des matières

Introduction Générale **1**

1 Cadre général du projet **3**
1.1 Cadre général . . . 3
1.2 Présentation de l'organisme d'accueil . . . 4
1.2.1 Fiche d'identité d'Antigone . . . 4
1.2.2 Domaines d'activité d'Antigone . . . 4
1.3 Contexte du projet . . . 5
1.3.1 Problématique . . . 5
1.3.2 Étude de l'existant . . . 6
1.4 Solution proposée . . . 8
1.4.1 Espace plateforme et intégration des commerçants . . . 8
1.4.2 Catalogue et vitrine personnalisable . . . 8
1.4.3 Parcours transactionnel . . . 8
1.4.4 Marketing et fidélisation . . . 9
1.4.5 Essayage virtuel et intelligence comportementale . . . 9
1.4.6 Qualité et déploiement . . . 9
1.4.7 Apports de la solution . . . 9
1.5 Étude des méthodologies de développement . . . 10
1.5.1 Choix de la méthodologie . . . 10
1.5.2 Étude comparative des cadres de développement . . . 11
1.5.3 Les rôles dans la méthode Scrum . . . 12

2 Analyse et spécification des besoins **13**
2.1 Analyse et spécification des besoins . . . 13
2.1.1 Identification des acteurs . . . 13
2.1.2 Spécification des besoins . . . 14
2.1.3 Diagramme de cas d'utilisation général . . . 19
2.2 Analyse globale . . . 23
2.2.1 Diagramme de classes d'analyse . . . 23
2.2.2 Architecture globale . . . 23
2.3 Pilotage du projet avec Scrum . . . 29
2.3.1 Backlog du produit . . . 29
2.4 Environnement de travail . . . 30
2.4.1 Environnement matériel de développement . . . 30
2.4.2 Environnement logiciel de développement . . . 30

3 Release 1 : Fondations de la plateforme Sellio **33**
3.1 Backlog du Sprint 1 . . . 33
3.2 Diagramme de cas d'utilisation du Sprint 1 . . . 35
3.3 Services web du Sprint 1 . . . 36
3.4 Les cas d'utilisation du Sprint 1 . . . 37
3.4.1 Cas d'utilisation ń S'authentifier ż . . . 37
3.5 Réalisation du Sprint 1 . . . 40
3.6 Backlog du Sprint 2 . . . 41
3.7 Diagramme de cas d'utilisation du Sprint 2 . . . 44
3.8 Services web du Sprint 2 . . . 45
3.9 Les cas d'utilisation du Sprint 2 . . . 46
3.9.1 Cas d'utilisation ń Créer une boutique et faire vérifier son identité ż 46
3.9.2 Cas d'utilisation ń S'inscrire comme commerçant ż . . . 49
3.9.3 Cas d'utilisation ń Supprimer un commerçant et sa boutique ż . . . 50
3.10 Réalisation du Sprint 2 . . . 52

4 Release 2 : Catalogue produit et vitrine personnalisable **56**
4.1 Backlog du Sprint 3 . . . 56
4.2 Diagramme de cas d'utilisation du Sprint 3 . . . 57
4.3 Services web du Sprint 3 . . . 58
4.4 Les cas d'utilisation du Sprint 3 . . . 59
4.4.1 Cas d'utilisation ń Ajouter un produit ż . . . 59
4.5 Réalisation du Sprint 3 . . . 61
4.6 Backlog du Sprint 4 . . . 63
4.7 Diagramme de cas d'utilisation du Sprint 4 . . . 65
4.8 Services web du Sprint 4 . . . 65
4.9 Les cas d'utilisation du Sprint 4 . . . 66
4.9.1 Cas d'utilisation ń Personnaliser la page d'accueil ż . . . 66
4.10 Réalisation du Sprint 4 . . . 68

5 Release 3 : Parcours transactionnel **74**
5.1 Backlog du Sprint 5 . . . 74
5.2 Diagramme de cas d'utilisation du Sprint 5 . . . 75
5.3 Services web du Sprint 5 . . . 76
5.4 Les cas d'utilisation du Sprint 5 . . . 77
5.4.1 Cas d'utilisation ń Passer commande et payer par carte ż . . . 77
5.5 Réalisation du Sprint 5 . . . 79
5.6 Backlog du Sprint 6 . . . 81
5.7 Diagramme de cas d'utilisation du Sprint 6 . . . 82
5.8 Services web du Sprint 6 . . . 83
5.9 Les cas d'utilisation du Sprint 6 . . . 85
5.9.1 Cas d'utilisation ń Traiter une demande de retour ż . . . 85
5.10 Réalisation du Sprint 6 . . . 87

6 Release 4 : Marketing et fidélisation **90**
6.1 Backlog du Sprint 7 . . . 90
6.2 Diagramme de cas d'utilisation du Sprint 7 . . . 91
6.3 Services web du Sprint 7 . . . 92
6.4 Les cas d'utilisation du Sprint 7 . . . 93
6.4.1 Cas d'utilisation ń Créer un code promo ciblé ż . . . 93

6.5 Réalisation du Sprint 7 . . . 95
6.6 Backlog du Sprint 8 . . . 96
6.7 Diagramme de cas d'utilisation du Sprint 8 . . . 97
6.8 Services web du Sprint 8 . . . 98
6.9 Les cas d'utilisation du Sprint 8 . . . 99
6.9.1 Cas d'utilisation ń Gagner des points et changer de niveau ż . . . 99
6.10 Réalisation du Sprint 8 . . . 100

7 Release 5 : Essayage virtuel en temps réel par IA générative 103
7.1 Architecture de l'essayage virtuel . . . 103
7.1.1 Flux d'architecture . . . 103
7.1.2 Couche de présentation . . . 104
7.1.3 Couche d'intermédiation . . . 104
7.1.4 Couche de génération . . . 104
7.2 Mise en uvre de l'essayage virtuel . . . 104
7.2.1 Critères de choix . . . 104
7.2.2 Comparaison des solutions candidates . . . 104
7.2.3 Solution retenue . . . 105
7.2.4 Configuration du modèle . . . 105
7.2.5 Gestion des erreurs . . . 106
7.3 Backlog du Sprint 9 . . . 106
7.4 Diagramme de cas d'utilisation du Sprint 9 . . . 107
7.5 Services web du Sprint 9 . . . 108
7.6 Les cas d'utilisation du Sprint 9 . . . 108
7.6.1 Cas d'utilisation ń Essayer un vêtement virtuellement ż . . . 108
7.7 Réalisation du Sprint 9 . . . 111

8 Release 6 : Intelligence comportementale et apprentissage automatique 112
8.1 Architecture de l'intelligence comportementale . . . 112
8.1.1 Flux d'architecture . . . 112
8.1.2 Couche de collecte . . . 113
8.1.3 Couche de stockage . . . 113
8.1.4 Couche d'entraînement . . . 113
8.1.5 Couche de service et de restitution . . . 114
8.2 Données d'entraînement et d'évaluation . . . 114
8.2.1 Pourquoi des jeux de données publics réels . . . 114
8.2.2 Nettoyage des données . . . 115
8.2.3 Protocole d'évaluation . . . 115
8.3 Moteur de recommandation . . . 116
8.3.1 Choix du modèle . . . 116
8.3.2 Modèle retenu et configuration . . . 117
8.3.3 Résultats . . . 118
8.4 Segmentation des visiteurs . . . 118
8.4.1 Choix du modèle . . . 118
8.4.2 Configuration . . . 119
8.4.3 Résultats . . . 119
8.5 Prédiction de l'attrition . . . 120
8.5.1 Définition du problème . . . 120
8.5.2 Choix du modèle . . . 120
8.5.3 Résultats . . . 121

8.5.4 Limites . . . 122
8.5.5 Fonctionnement d'une boutique à l'autre : les huit secteurs . . . 122
8.6 Backlog du Sprint 10 . . . 123
8.7 Diagramme de cas d'utilisation du Sprint 10 . . . 125
8.8 Diagramme de classes du Sprint 10 . . . 125
8.9 Services web du Sprint 10 . . . 126
8.10 Les cas d'utilisation du Sprint 10 . . . 127
8.10.1 Cas d'utilisation ń Recevoir des recommandations personnalisées ż . 127
8.11 Réalisation du Sprint 10 . . . 128
8.12 Backlog du Sprint 11 . . . 129
8.13 Diagramme de cas d'utilisation du Sprint 11 . . . 131
8.14 Services web du Sprint 11 . . . 131
8.15 Les cas d'utilisation du Sprint 11 . . . 132
8.15.1 Cas d'utilisation ń Consulter l'analyse comportementale et les risques d'attrition ż . . . 132
8.16 Réalisation du Sprint 11 . . . 134

9 Release 7 : Qualité, conteneurisation et intégration continue 136
9.1 Backlog du Sprint 12 . . . 136
9.2 Diagramme de cas d'utilisation du Sprint 12 . . . 137
9.3 Choix des outils . . . 138
9.4 Stratégie de tests . . . 138
9.5 Conteneurisation . . . 140
9.5.1 Images . . . 140
9.5.2 Composition de la plateforme . . . 140
9.6 Pipeline d'intégration et de déploiement continus . . . 140
9.7 Réalisation du Sprint 12 . . . 142

Conclusion et perspectives 144

Table des figures

1.1 Logo d'Antigone 4
1.2 Fonctionnement du cadre Scrum [6] 12

2.1 Diagramme de cas d'utilisation de l'équipe Sellio 20
2.2 Diagramme de cas d'utilisation du Commerçant 21
2.3 Diagramme de cas d'utilisation du Visiteur et du Client 22
2.4 Diagramme de classes d'analyse 23
2.5 Architecture logique de la plateforme Sellio 27
2.6 Architecture physique de la plateforme Sellio 29

3.1 Diagramme de cas d'utilisation du Sprint 1 36
3.2 Diagramme de séquence système du cas ń S'authentifier ż 39
3.3 Diagramme de séquence objet du cas ń S'authentifier ż 40
3.4 Interface de connexion à Sellio 40
3.5 Gestion des rôles et permissions 41
3.6 Liste des clients de la boutique 41
3.7 Diagramme de cas d'utilisation du Sprint 2 45
3.8 Diagramme de séquence système du cas ń Créer une boutique et faire vérifier son identité ż 48
3.9 Diagramme de séquence objet du cas ń Créer une boutique et faire vérifier son identité ż 49
3.10 Diagramme de séquence système du cas ń S'inscrire comme commerçant ż . 50
3.11 Diagramme de séquence système du cas ń Supprimer un commerçant et sa boutique ż 52
3.12 Diagramme de séquence objet du cas ń Supprimer un commerçant et sa boutique ż 52
3.13 Page d'accueil de Sellio 53
3.14 Assistant de création de boutique : informations et secteur 53
3.15 Console d'administration Sellio 54
3.16 Revue d'un dossier de vérification par l'administrateur 54
3.17 Comptes de la plateforme dans la console Sellio 55

4.1 Diagramme de cas d'utilisation du Sprint 3 58
4.2 Diagramme de séquence système du cas ń Ajouter un produit ż 60
4.3 Diagramme de séquence objet du cas ń Ajouter un produit ż 61
4.4 Liste des produits de la boutique 61
4.5 Fiche d'un produit de type ń Bague ż dans une boutique de bijoux 62
4.6 Fiche d'un produit cosmétique 63
4.7 Gestion des catégories 63

4.8 Diagramme de cas d'utilisation du Sprint 4 65
4.9 Diagramme de séquence système du cas ń Personnaliser la page d'accueil ż 67
4.10 Diagramme de séquence objet du cas ń Personnaliser la page d'accueil ż . . 68
4.11 Une même boutique dans les huit modèles de vitrine 69
4.12 Choix du modèle et du secteur 70
4.13 Page ń Apparence ż avec l'aperçu en direct de la vitrine 70
4.14 Onglet ń Couleurs ż : palettes et réglages par élément 71
4.15 Éditeur de la page d'accueil 71
4.16 Fiche produit sur la vitrine 72
4.17 Fiche d'une bague : choix du tour de doigt et caractéristiques 72

5.1 Diagramme de cas d'utilisation du Sprint 5 76
5.2 Diagramme de séquence système du cas ń Passer commande et payer par carte ż 78
5.3 Diagramme de séquence objet du cas ń Passer commande et payer par carte ż 79
5.4 Tiroir de panier 79
5.5 Tunnel de commande : adresse, livraison et code promo 80
5.6 Paiement par carte avec Stripe 80
5.7 Diagramme de cas d'utilisation du Sprint 6 83
5.8 Diagramme de séquence système du cas ń Traiter une demande de retour ż 86
5.9 Diagramme de séquence objet du cas ń Traiter une demande de retour ż . . 87
5.10 Gestion des commandes 87
5.11 Tableau de bord des ventes 88
5.12 Traitement des retours 88
5.13 Paramétrage de la TVA et des zones de livraison 89
5.14 Modération des avis clients 89

6.1 Diagramme de cas d'utilisation du Sprint 7 92
6.2 Diagramme de séquence système du cas ń Créer un code promo ciblé ż . . 94
6.3 Diagramme de séquence objet du cas ń Créer un code promo ciblé ż 95
6.4 Gestion des promotions 95
6.5 Création d'une bannière ciblée 96
6.6 E-mail marketing 96
6.7 Diagramme de cas d'utilisation du Sprint 8 98
6.8 Diagramme de séquence système du cas ń Gagner des points et changer de niveau ż 100
6.9 Diagramme de séquence objet du cas ń Gagner des points et changer de niveau ż 100
6.10 Configuration du programme de fidélité 101
6.11 Niveaux et avantages du programme de fidélité 101
6.12 Carte de fidélité dans l'espace client 102

7.1 Architecture de l'essayage virtuel 103
7.2 Diagramme de cas d'utilisation du Sprint 9 108
7.3 Diagramme de séquence système du cas ń Essayer un vêtement virtuellement ż 110
7.4 Diagramme de séquence objet du cas ń Essayer un vêtement virtuellement ż 110
7.5 Accès à l'essayage virtuel depuis la fiche produit 111
7.6 Essayage virtuel en direct 111

8.1 Architecture de l'intelligence comportementale 113

8.2 Diagramme de cas d'utilisation du Sprint 10 . . . 125
8.3 Diagramme de classes du microservice d'analyse . . . 126
8.4 Diagramme de séquence système du cas ń Recevoir des recommandations personnalisées ż . . . 128
8.5 Diagramme de séquence objet du cas ń Recevoir des recommandations personnalisées ż . . . 128
8.6 Recommandations sur la fiche produit . . . 129
8.7 Produits souvent achetés ensemble dans le panier . . . 129
8.8 Diagramme de cas d'utilisation du Sprint 11 . . . 131
8.9 Diagramme de séquence système du cas ń Consulter l'analyse comportementale ż . . . 133
8.10 Diagramme de séquence objet du cas ń Consulter l'analyse comportementale ż . . . 134
8.11 Tableau de bord ń Comportement ż . . . 134
8.12 Niveau de risque d'attrition des clients . . . 135

9.1 Diagramme de cas d'utilisation du Sprint 12 . . . 138
9.2 Pipeline d'intégration et de déploiement continus . . . 141
9.3 Chaîne DevOps de Sellio . . . 142
9.4 Exécution du pipeline dans GitHub Actions . . . 142
9.5 Historique des exécutions du workflow CI . . . 143

Liste des tableaux

1.1 Fiche d'identité d'Antigone [1] 4
1.2 Comparaison de solutions existantes de création et de gestion de boutiques en ligne [2, 3, 4, 5] 7
1.3 Étude comparative des méthodologies de développement 10
1.4 Étude comparative des cadres de développement Agile 11
1.5 Répartition des rôles Scrum dans notre projet 12

2.1 Besoins fonctionnels du module Plateforme, authentification et vérification 15
2.2 Besoins fonctionnels du module Catalogue et vitrine 15
2.3 Besoins fonctionnels du module Transactions 16
2.4 Besoins fonctionnels du module Marketing et fidélisation 16
2.5 Besoins fonctionnels du module Intelligence 17
2.6 Besoins fonctionnels du module Qualité et déploiement 17
2.7 Exigences non fonctionnelles du système 18
2.8 Comparaison des architectures logicielles étudiées 24
2.9 Répartition des données entre les bases des microservices 25
2.10 Comparaison MongoDB / PostgreSQL 26
2.11 Synthèse du backlog produit de Sellio 30
2.12 Outils utilisés pour la modélisation, le développement et le suivi 31
2.13 Langages de programmation utilisés 31
2.14 Frameworks et bibliothèques utilisés 32

3.1 Backlog du Sprint 1 35
3.2 Services web du Sprint 1 37
3.3 Description textuelle du cas d'utilisation ń S'authentifier ż 38
3.4 Backlog du Sprint 2 44
3.5 Services web du Sprint 2 46
3.6 Description textuelle du cas ń Créer une boutique et faire vérifier son identité ż 47
3.7 Description textuelle du cas ń S'inscrire comme commerçant ż 49
3.8 Description textuelle du cas ń Supprimer un commerçant et sa boutique ż . 51

4.1 Backlog du Sprint 3 57
4.2 Services web du Sprint 3 59
4.3 Description textuelle du cas ń Ajouter un produit ż 59
4.4 Types de produits par secteur 62
4.5 Backlog du Sprint 4 65
4.6 Services web du Sprint 4 66
4.7 Description textuelle du cas ń Personnaliser la page d'accueil ż 67

5.1 Backlog du Sprint 5 . . . 75
5.2 Services web du Sprint 5 . . . 76
5.3 Description textuelle du cas ń Passer commande et payer par carte ż . . . 77
5.4 Backlog du Sprint 6 . . . 82
5.5 Services web du Sprint 6 . . . 84
5.6 Description textuelle du cas ń Traiter une demande de retour ż . . . 85

6.1 Backlog du Sprint 7 . . . 91
6.2 Services web du Sprint 7 . . . 93
6.3 Description textuelle du cas ń Créer un code promo ciblé ż . . . 94
6.4 Backlog du Sprint 8 . . . 97
6.5 Services web du Sprint 8 . . . 99
6.6 Description textuelle du cas ń Gagner des points et changer de niveau ż . . . 99

7.1 Comparaison des solutions d'essayage virtuel . . . 105
7.2 Configuration de l'essayage virtuel . . . 106
7.3 Backlog du Sprint 9 . . . 107
7.4 Services web du Sprint 9 . . . 108
7.5 Description textuelle du cas ń Essayer un vêtement virtuellement ż . . . 109

8.1 Jeux de données publics utilisés . . . 114
8.2 Principales étapes de nettoyage . . . 115
8.3 Métriques d'évaluation . . . 116
8.4 Comparaison des approches de recommandation . . . 117
8.5 Résultats du moteur de recommandation sur la période de test . . . 118
8.6 Comparaison des algorithmes de segmentation . . . 119
8.7 Segments de visiteurs obtenus sur le jeu Cosmetics . . . 120
8.8 Comparaison des modèles de prédiction de l'attrition . . . 121
8.9 Résultats de la prédiction de l'attrition sur la période de test . . . 121
8.10 Ce qui est appris par boutique et ce qui est pré-entraîné . . . 122
8.11 Modèle d'attrition utilisé selon le secteur . . . 123
8.12 Backlog du Sprint 10 . . . 124
8.13 Services web du Sprint 10 . . . 126
8.14 Description textuelle du cas ń Recevoir des recommandations personnalisées ż . . . 127
8.15 Backlog du Sprint 11 . . . 131
8.16 Services web du Sprint 11 . . . 132
8.17 Description textuelle du cas ń Consulter l'analyse comportementale et les risques d'attrition ż . . . 133

9.1 Backlog du Sprint 12 . . . 137
9.2 Comparaison des solutions d'intégration continue . . . 138
9.3 Tests automatisés de Sellio . . . 139
9.4 Images Docker de Sellio . . . 140
9.5 Durées d'une exécution du pipeline . . . 141

Introduction Générale

Le commerce en ligne occupe aujourd'hui une place importante dans le développement des petites marques. En Tunisie, beaucoup d'entre elles s'appuient encore principalement sur Instagram, WhatsApp et une gestion manuelle des commandes. Cette organisation devient rapidement difficile à maintenir lorsque l'activité augmente, notamment pour le suivi des stocks, des paiements, des livraisons et du comportement des clients. Des solutions comme Shopify ou WooCommerce existent, mais elles restent peu adaptées à certaines spécificités locales telles que le dinar tunisien, la TVA ou les zones de livraison, et nécessitent souvent des extensions payantes pour les fonctions avancées.

Antigone, agence tunisienne de conseil en stratégie et en marketing digital, accompagne plusieurs marques confrontées à ces besoins. Après avoir développé une boutique en ligne pour NaturEssence, spécialisée dans les cosmétiques naturels, l'agence a constaté que les mêmes attentes revenaient chez plusieurs clients : gestion des variantes produits, personnalisation de la vitrine, commandes, marketing et analyse du comportement client. Cette expérience a conduit à l'idée de créer une solution commune et multi-boutiques.

C'est dans ce cadre que s'inscrit notre projet de fin d'études, réalisé au sein d'Antigone dans le cadre du cycle ingénieur à l'École ESPRIT. Il consiste à concevoir et développer **ń Sellio ż**, une plateforme SaaS intelligente permettant à chaque commerçant de créer et gérer sa propre boutique dans l'un des huit secteurs proposés : mode, beauté, sport, high-tech, maison, épicerie, bijoux et enfants.

Chaque boutique passe par une étape de vérification avant activation. Le commerçant peut ensuite choisir parmi huit modèles de vitrine, personnaliser l'apparence de sa boutique et gérer son catalogue, ses commandes, ses promotions et son programme de fidélité. La plateforme repose sur une architecture microservices avec Spring Boot, une passerelle API, PostgreSQL et des interfaces React. Elle intègre également Stripe pour le paiement en ligne et l'API Decart pour l'essayage virtuel en temps réel.

Sellio comporte aussi un volet d'apprentissage automatique comprenant le suivi comportemental des visiteurs, un moteur de recommandation hybride, la segmentation des visiteurs et la prédiction de l'attrition, à partir de jeux de données publics réels. La plateforme est enfin testée, conteneurisée avec Docker et intégrée à un pipeline GitHub Actions.

Ce rapport est organisé en neuf chapitres :

— Le **chapitre 1** présente le contexte du projet, l'organisme d'accueil, la problématique, l'étude de l'existant et la méthodologie.

— Le **chapitre 2** traite de l'analyse des besoins, de l'architecture, du backlog produit et de la planification des sprints.

— Le **chapitre 3** présente la Release 1 : authentification, rôles, création de boutique, KYC et administration Sellio.

— Le **chapitre 4** couvre la Release 2 : catalogue, huit secteurs, huit modèles de vitrine et personnalisation.

— Le **chapitre 5** présente la Release 3 : panier, commande, paiement Stripe, retours, TVA, livraison et avis.

— Le **chapitre 6** est consacré à la Release 4 : promotions, bannières, e-mailing, fidélité et segments.

— Le **chapitre 7** présente la Release 5 : essayage virtuel en temps réel avec l'API Decart.

— Le **chapitre 8** traite de la Release 6 : recommandation, segmentation et prédiction de l'attrition.

— Le **chapitre 9** présente la Release 7 : tests, Docker, intégration et livraison continues.

Le rapport se termine par une conclusion générale présentant les résultats obtenus et les perspectives d'évolution de Sellio.

Chapitre 1

Cadre général du projet

Introduction

Ce premier chapitre pose les bases du projet. Nous présentons d'abord l'organisme d'accueil, puis le contexte qui a conduit à la naissance de Sellio et la problématique à laquelle il répond. Nous étudions ensuite les principales solutions de création de boutiques en ligne afin d'en dégager les limites, avant de décrire la solution proposée. Nous terminons par l'étude des méthodologies de développement ayant conduit au choix de Scrum.

1.1 Cadre général

Ce projet s'inscrit dans le cadre du projet de fin d'études en vue de l'obtention du Diplôme National d'Ingénieur en Informatique, option **TWIN** (Technologies du Web et de l'Internet), à l'École ESPRIT. Il a été réalisé au sein de l'agence Antigone dans le cadre d'un stage initialement prévu pour une durée de six mois, débuté le **2 février 2026**, puis prolongé par avenant jusqu'au **29 octobre 2026**.

Le sujet officiel, ń *Conception et développement d'une plateforme transactionnelle intelligente* ż, consiste à concevoir une plateforme de commerce électronique distribuée, capable d'analyser en temps réel les interactions des utilisateurs et d'exploiter ces données à l'aide de techniques d'apprentissage automatique afin de personnaliser l'expérience d'achat.

Ce sujet s'est concrétisé sous la forme de **Sellio**, une plateforme SaaS permettant à des commerçants issus de huit secteurs d'activité : mode, beauté, sport, high-tech, maison, épicerie, bijoux et articles pour enfants , de créer et d'exploiter leur propre boutique en ligne.

1.2 Présentation de l'organisme d'accueil

Antigone est une agence de stratégie et de marketing digital basée à Radès, fondée en 2021. Elle accompagne ses clients dans la construction de leur stratégie digitale : diagnostic de la situation existante, définition des axes de communication, puis mise en uvre à travers la création de sites web, d'applications mobiles et de campagnes. Une partie croissante de ses clients sont de petites marques qui souhaitent vendre en ligne, ce qui a naturellement conduit l'agence à s'intéresser au commerce électronique.

FIGURE 1.1 – Logo d'Antigone

1.2.1 Fiche d'identité d'Antigone

Le tableau 1.1 présente la fiche d'identité d'Antigone.

Nom de la société	**Antigone**
Fondateur et représentant	M. Malek Naouar
Adresse	12, Rue Habib Thameur, Radès 2040, Ben Arous
Téléphone	(+216) 55 557 193
Adresse e-mail	antigoneconsulting.info@gmail.com
Site web	https ://antigoneagency.com/
Logo	Antigone

TABLE 1.1 – Fiche d'identité d'Antigone [1]

1.2.2 Domaines d'activité d'Antigone

Les activités d'Antigone s'organisent autour de trois axes :

1. **Stratégie de marketing digital :**
   - réalisation d'un **diagnostic** de la présence digitale du client ;
   - définition des **axes de communication** les plus pertinents ;
   - accompagnement dans la **construction et l'optimisation** de la stratégie marketing.
2. **Conseil, études et accompagnement :**
   - **conseils et études** sur mesure ;
   - démarche pragmatique fondée sur le **marketing des services** et le **marketing de l'innovation**.
3. **Création et développement digital :**
   - conception et développement de **sites web** et de **boutiques en ligne** ;
   - conception et développement d'**applications mobiles** ;
   - mise à disposition de solutions adaptées à la réussite des projets clients.

1.3 Contexte du projet

Dans cette section, nous décrivons le problème, examinons les solutions existantes, puis présentons la solution proposée.

1.3.1 Problématique

Le projet est né d'un développement précédent : **NaturEssence**, une boutique en ligne de cosmétiques naturels réalisée par Antigone pour l'un de ses clients. Au fil des échanges avec d'autres marques accompagnées par l'agence, un constat s'est imposé : une boutique de vêtements, une bijouterie, une épicerie fine ou une autre marque de cosmétiques avaient au fond les mêmes besoins, un catalogue avec variantes, une vitrine à l'image de la marque, un tunnel de commande, des promotions, un suivi des clients. Refaire ce travail boutique par boutique coûte cher et produit des applications isolées, chacune à maintenir séparément.

Du côté des commerçants, la situation n'est pas meilleure. Beaucoup vendent aujourd'hui via Instagram ou WhatsApp : les commandes sont prises en message privé, le stock est tenu dans un carnet ou un tableur, les tailles et les couleurs disponibles sont communiquées au cas par cas, et aucune donnée n'est conservée sur ce que les visiteurs regardent, ajoutent au panier puis abandonnent. Ceux qui passent à une plateforme comme Shopify se heurtent à d'autres obstacles : abonnement en devises, paramétrage de la TVA et des zones de livraison tunisiennes, et fonctions avancées (recommandations, essayage virtuel, analyse prédictive) proposées sous forme d'extensions payantes indépendantes, qui ne partagent pas leurs données.

Enfin, une plateforme qui héberge plusieurs commerçants pose des questions propres au modèle SaaS : comment s'assurer qu'un commerçant est bien celui qu'il prétend être avant de le laisser encaisser des paiements, comment isoler strictement les données d'une boutique de celles d'une autre, y compris ses clients, et comment offrir à chacun une vitrine et une fiche produit réellement adaptées à son métier sans développer un site par client. Enfin, une plateforme de cette taille doit pouvoir être modifiée sans régression : chaque évolution doit être construite, testée et livrée de façon automatique.

La problématique de ce projet peut être résumée ainsi :

Comment concevoir une plateforme transactionnelle distribuée, fondée sur une architecture microservices, qui permette à des commerçants de secteurs variés de créer, personnaliser et exploiter leur propre boutique en ligne en toute sécurité, tout en analysant en temps réel les interactions de leurs visiteurs grâce à l'apprentissage automatique afin de personnaliser l'expérience d'achat et d'anticiper la perte de clients ?

1.3.2 Étude de l'existant

Avant de concevoir Sellio, nous avons étudié plusieurs solutions de création et de gestion de boutiques en ligne afin d'identifier leurs principaux apports et leurs limites par rapport aux besoins du projet. Trois plateformes internationales largement utilisées ont été retenues, auxquelles nous avons ajouté Converty, une solution tunisienne permettant de mieux situer Sellio par rapport aux réalités du marché local.

— **Shopify** [2] est l'une des principales plateformes SaaS de création de boutiques en ligne. Elle propose des thèmes personnalisables, une gestion du catalogue avec variantes, des outils de paiement ainsi qu'un vaste écosystème d'applications. Cependant, son fonctionnement reste principalement pensé pour un marché international. L'abonnement est facturé en devises et certaines adaptations au contexte tunisien nécessitent des configurations ou des applications supplémentaires. Plusieurs fonctionnalités avancées, telles que la recommandation, la fidélisation ou l'essayage virtuel, reposent également sur des applications tierces.

— **WooCommerce** [3] est une extension open source de WordPress permettant de transformer un site en boutique en ligne. Elle se distingue par sa grande flexibilité et par l'absence d'abonnement obligatoire à la plateforme elle-même. En contrepartie, le commerçant doit gérer son hébergement, les mises à jour, la sécurité et la compatibilité entre les différentes extensions. WooCommerce n'est pas une plateforme multi-boutiques SaaS par nature : chaque boutique repose généralement sur sa propre installation.

— **Wix eCommerce** [4] privilégie la simplicité de prise en main. Son éditeur visuel permet de construire et de personnaliser une boutique sans connaissances techniques particulières. La plateforme fournit les fonctions courantes de gestion des produits, des commandes et du contenu. Ses possibilités deviennent cependant plus limitées lorsqu'il s'agit de représenter des caractéristiques métier complexes ou de mettre en place des mécanismes avancés d'analyse comportementale et de prédiction.

— **Converty** [5] constitue une alternative particulièrement intéressante dans le contexte tunisien. Il s'agit d'une plateforme SaaS permettant aux commerçants de créer et de gérer une boutique en ligne sans avoir à développer leur propre solution. Elle propose notamment la gestion des produits, des commandes, des clients et des stocks, ainsi que des modèles de boutiques personnalisables et une application mobile de gestion.

  Contrairement aux solutions internationales étudiées, Converty est directement orientée vers le marché tunisien et prend en compte plusieurs pratiques locales, telles que l'utilisation du dinar tunisien, le paiement à la livraison et l'intégration avec des acteurs locaux de la livraison. La plateforme met également en avant certaines fonctionnalités basées sur l'intelligence artificielle, notamment des outils d'assistance à la création et à la vente.

Le tableau 1.2 compare ces quatre solutions selon les critères les plus pertinents pour notre projet.

Critères	**Shopify**	**WooCommerce**	**Wix eCommerce**	**Converty**
Plateforme SaaS hébergeant plusieurs boutiques	Oui	Non	Oui	Oui
Fiches produit adaptées au secteur (tour de doigt, pointure, allergènes...)	Partiel (méta-champs)	Extensions	Partiel	Non
Gabarits de vitrine personnalisables sans code	Oui	Partiel	Oui	Oui
Adaptation au marché tunisien (dinar, livraison, pratiques locales)	Configuration nécessaire	Configuration nécessaire	Configuration nécessaire	Oui
Paiement à la livraison	Selon configuration	Via extensions	Selon configuration	Oui
Intégration avec des transporteurs tunisiens	Via applications ou configuration	Via extensions	Limitée	Oui
Vérification d'identité du commerçant (KYC)	Via le prestataire de paiement	Non native	Via le prestataire de paiement	Non
Recommandations personnalisées	Applications tierces	Extensions	Limitées	Non
Prédiction de l'attrition	Applications tierces possibles	Non native	Non native	Non
Essayage virtuel	Applications tierces	Extensions	Non natif	Non
Fonctionnalités basées sur l'intelligence artificielle	Oui, natives ou via applications	Via extensions	Certaines fonctionnalités	Oui, certaines fonctions d'assistance
Programme de fidélité	Applications tierces	Extensions	Applications ou extensions	Non
Application mobile de gestion	Oui	Selon extensions	Oui	Oui
Modèle économique	Abonnement mensuel en devises	Logiciel gratuit + hébergement et extensions	Abonnement mensuel en devises	Offre adaptée au marché tunisien

TABLE 1.2 – Comparaison de solutions existantes de création et de gestion de boutiques en ligne [2, 3, 4, 5]

Cette comparaison montre que Converty répond déjà à plusieurs besoins propres au marché tunisien, notamment le dinar, le paiement à la livraison et l'intégration avec des services locaux.

Sellio se différencie toutefois par une approche plus intégrée : une plateforme multi-boutiques adaptée à huit secteurs, avec personnalisation avancée des vitrines et exploita-

tion des données transactionnelles et comportementales.

Elle intègre directement un moteur de recommandation, la segmentation des visiteurs, la prédiction de l'attrition et l'essayage virtuel en temps réel.

Sellio se positionne ainsi comme une solution SaaS locale enrichie par une couche d'intelligence comportementale intégrée.

1.4 Solution proposée

La solution retenue est une plateforme web baptisée **Sellio**. Elle se compose de trois espaces : le **site de la plateforme** (présentation, inscription des commerçants et console d'administration), le **back-office du commerçant** pour gérer sa boutique, et la **vitrine** publique de chaque boutique, sur laquelle ses clients achètent.

1.4.1 Espace plateforme et intégration des commerçants

- Page d'accueil Sellio, inscription et connexion des commerçants ;
- Inscription des commerçants confirmée par un code envoyé par e-mail, réinitialisation du mot de passe ;
- Assistant de création de boutique : informations, secteur parmi huit, modèle de vitrine parmi huit, identité visuelle, carte bancaire, pièce d'identité et selfie ;
- Vérification KYC (*Know Your Customer*) par l'équipe Sellio, qui approuve ou refuse la boutique avec un motif ;
- Console d'administration : statistiques globales, boutiques et propriétaires, blocage, suspension ou suppression d'un commerçant et de sa boutique ; le super administrateur ajoute des administrateurs et gère les rôles. L'équipe Sellio n'a jamais accès aux clients des boutiques.

1.4.2 Catalogue et vitrine personnalisable

- Gestion des produits et des catégories ; fiche produit adaptée au secteur et au type du produit (bague, montre, chaussures, machine de fitness, smartphone…) ;
- Valeurs choisies dans des listes normalisées (tailles, couleurs, matières, pierres, allergènes…), extensibles par chaque boutique ;
- Huit modèles de vitrine complets (**Minimal**, **Bold**, **Luxury**, **Sport**, **Tech**, **Artisan**, **Pop**, **Éditorial**) ;
- Personnalisation sans code avec aperçu en direct : logo et taille, couleurs, polices, barre d'annonce, coordonnées et réseaux sociaux, et page d'accueil (sections affichées et ordonnées, textes, produits mis en avant).

1.4.3 Parcours transactionnel

- Panier dynamique avec choix obligatoire de la taille, de la pointure ou du format pour les articles concernés ;
- Tunnel de commande avec zones de livraison, TVA et codes promotionnels ;
- Paiement en ligne sécurisé par **Stripe** ;
- Suivi des statuts de commande, historique client, demandes de retour et politique de retour ;
- Avis clients modérés par le commerçant.

1.4.4 Marketing et fidélisation

— Promotions : codes promo et remises automatiques ;

— Bannières de page d'accueil ciblées par segment de clientèle ;

— Campagnes e-mail (newsletter, contact individuel) via l'API Brevo ;

— Programme de fidélité à points avec niveaux (Nouveau, Fidèle, VIP, Inactif) et montée de niveau automatique.

1.4.5 Essayage virtuel et intelligence comportementale

— **Cabine d'essayage virtuel en temps réel** : le client active sa caméra et voit le vêtement porté sur lui, généré en direct par le modèle *Lucy VTON* de l'API **Decart** ;

— **Suivi comportemental** : collecte des événements de navigation (vues, recherches, ajouts au panier, achats) dans une table partitionnée ;

— **Moteur de recommandation hybride** : ń produits similaires ż, ń pour vous ż, ń souvent achetés ensemble ż et ń populaires ż ;

— **Segmentation** des visiteurs par K-Means et **prédiction de l'attrition** des clients par forêt aléatoire ou gradient boosting selon le secteur ;

— Tableau de bord ń Comportement ż destiné au commerçant.

1.4.6 Qualité et déploiement

— Tests automatisés des règles critiques (isolation des boutiques, droits, modèles d'apprentissage) ;

— Conteneurisation de toute la plateforme avec Docker et lancement en une commande ;

— Intégration et livraison continues avec GitHub Actions : chaque modification est construite et testée, et les images sont publiées automatiquement.

1.4.7 Apports de la solution

— **Une plateforme pour plusieurs boutiques** : un seul code, une seule infrastructure, et une boutique opérationnelle en quelques minutes au lieu d'un développement sur mesure .

— **Une confiance vérifiée** : aucun commerçant ne peut vendre avant la validation de son identité et de son moyen de paiement ; seules les informations non sensibles de la carte sont conservées, conformément à la norme PCI-DSS .

— **Une isolation stricte des données** : chaque requête est rattachée à la boutique de l'utilisateur connecté ; chaque client appartient à une seule boutique, et même l'équipe Sellio n'accède pas aux clients des commerçants .

— **Une vitrine à l'image de la marque et adaptée à son métier** sans écrire de code, grâce aux huit modèles, aux fiches produit par secteur et aux éditeurs avec aperçu en direct .

— **Une adaptation au contexte local et international** : gestion multidevise, prise en charge du dinar tunisien, TVA configurable et zones de livraison paramétrables.

— **Une intelligence intégrée** : recommandation, segmentation et prédiction de l'attrition partagent les mêmes données que le reste de la plateforme, et chaque modèle a été évalué sur des données publiques réelles, avec comparaison à une référence simple .

— **Une expérience d'achat différenciante** grâce à l'essayage virtuel, qui réduit l'incertitude sur le rendu d'un vêtement, principale cause d'hésitation et de retours dans la mode en ligne .

— **Une architecture évolutive**, où chaque domaine (identité, catalogue, commandes, marketing, analyse) est un microservice qui peut évoluer et être déployé séparément ;

— **Une livraison fiable**, chaque modification étant testée et conteneurisée automatiquement avant d'être publiée.

1.5 Étude des méthodologies de développement

Avant de présenter les différentes approches, il convient de distinguer la **méthodologie**, qui définit les principes généraux du processus de développement, du **cadre de développement**, qui fournit une structure concrète (rôles, cérémonies, artefacts) pour appliquer ces principes.

1.5.1 Choix de la méthodologie

Le tableau 1.3 compare les principales méthodologies de développement logiciel.

Méthodologie	**Avantages**	**Inconvénients**
Waterfall	— Phases claires et structurées. — Planification simple du projet.	— Difficulté à s'adapter aux changements des besoins. — Retour d'information tardif.
RUP	— Forte traçabilité. — Processus orienté vers la gestion des risques.	— Gestion complexe. — Charge documentaire importante.
2TUP	— Approche itérative séparant branche fonctionnelle et branche technique. — Prise en compte des risques techniques.	— Moins flexible lorsque les besoins évoluent. — Approche davantage orientée processus.
Agile	— Approche itérative et adaptative. — Retour d'information continu. — Bonne adaptation aux changements.	— Nécessite une communication régulière. — Dépend de l'implication des parties prenantes.

Table 1.3 – Étude comparative des méthodologies de développement

L'**approche Agile** a été retenue. Le périmètre de Sellio a en effet beaucoup évolué pendant le stage : le projet a commencé comme une boutique unique (NaturEssence), a été découpé en microservices, puis transformé en plateforme multi-boutiques, avant d'intégrer l'essayage virtuel, l'apprentissage automatique, de nouveaux secteurs et une

chaîne DevOps. Seule une démarche itérative permettait d'absorber ces évolutions sans remettre en cause le travail déjà livré.

1.5.2 Étude comparative des cadres de développement

Le tableau 1.4 compare les principaux cadres Agile : **Scrum**, **Kanban** et **Extreme Programming (XP)**.

Cadre	**Avantages**	**Inconvénients**
Scrum	— Organisation du travail en sprints. — Backlog permettant de prioriser les fonctionnalités. — Suivi régulier de l'avancement. — Rôles clairement définis.	— Nécessite une organisation et une communication régulières. — Moins adapté aux tâches très imprévisibles.
Kanban	— Gestion visuelle des tâches. — Grande flexibilité des priorités. — Limitation du travail en cours.	— Moins structuré que Scrum. — Pas de rôles ni de sprints fixes.
XP	— Forte orientation vers la qualité du code. — Tests fréquents et intégration continue.	— Nécessite une forte implication de l'équipe. — Pratiques (programmation en binôme) difficiles à appliquer par une seule développeuse.

TABLE 1.4 – Étude comparative des cadres de développement Agile

Nous avons retenu **Scrum** [6]. Son découpage en sprints de deux semaines correspond au rythme des points de suivi avec l'encadrant professionnel, et le backlog permet de prioriser les user stories d'un périmètre qui s'est élargi au fil du projet. Le projet a ainsi été organisé en sept releases et douze sprints, présentés au chapitre suivant.

La figure 1.2 présente le fonctionnement du cadre Scrum, depuis la planification du sprint jusqu'à la revue et à la rétrospective.

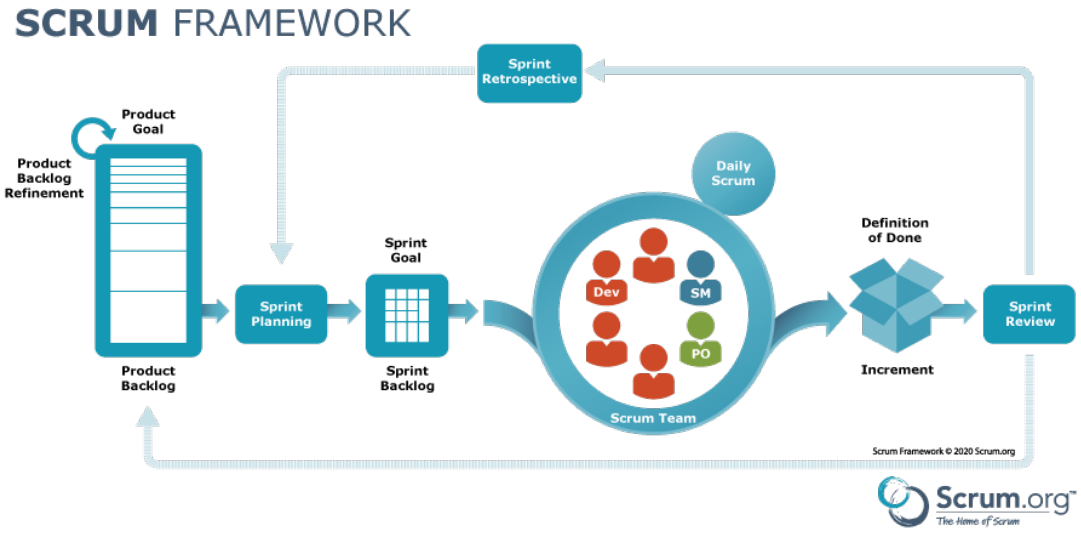


FIGURE 1.2 – Fonctionnement du cadre Scrum [6]

1.5.3 Les rôles dans la méthode Scrum

Scrum repose sur trois rôles :

- Product Owner : il définit les besoins, priorise les fonctionnalités et maximise la valeur du produit.
- Scrum Master : facilitateur, il veille à la bonne application de Scrum, anime les cérémonies et lève les obstacles.
- Équipe de développement : elle réalise les tâches planifiées dans chaque sprint, de l'analyse jusqu'à la livraison.

Le tableau 1.5 présente les personnes qui ont occupé ces rôles dans notre projet.

Rôle Scrum	**Personne responsable**
Product Owner	M. Malek Naouar
Scrum Master	*Karim Arouchi*
Équipe de développement	Rania Khedhri

TABLE 1.5 – Répartition des rôles Scrum dans notre projet

Conclusion

Dans ce chapitre, nous avons présenté l'organisme d'accueil et le contexte du projet, formulé la problématique, étudié les solutions existantes et décrit la solution proposée. Nous avons enfin justifié le choix de Scrum. Le chapitre suivant détaille l'analyse et la spécification des besoins.

Chapitre 2

Analyse et spécification des besoins

Introduction

Dans ce chapitre, nous analysons en détail les besoins de la plateforme Sellio. Nous identifions d'abord les acteurs, puis les besoins fonctionnels et non fonctionnels. Nous présentons ensuite les diagrammes de cas d'utilisation, l'architecture retenue, le backlog produit et la planification des sprints, et enfin l'environnement de travail.

2.1 Analyse et spécification des besoins

La spécification des besoins fonctionnels et non fonctionnels est la première étape du cycle de développement : elle fixe ce que la plateforme doit offrir et les qualités qu'elle doit garantir.

2.1.1 Identification des acteurs

Sellio met en relation une plateforme, des commerçants et les clients de ces commerçants. Chaque acteur y joue un rôle distinct.

Acteurs principaux

- **Administrateur Sellio :** membre de l'équipe de la plateforme. Il consulte les statistiques globales, la liste des boutiques et de leurs propriétaires, valide ou refuse les dossiers de vérification (KYC), suspend une boutique, bloque ou supprime un commerçant. Il n'a jamais accès aux clients, aux commandes ni aux analyses des boutiques.
- **Super administrateur Sellio :** administrateur Sellio qui, en plus, ajoute des administrateurs, change le rôle des comptes de la plateforme et supprime un compte Sellio.
- **Commerçant :** propriétaire d'une boutique. Il s'inscrit, crée sa boutique, soumet son dossier de vérification, puis gère depuis le back-office son catalogue, sa vitrine, ses commandes, ses retours, ses promotions, son programme de fidélité, ses campagnes e-mail et consulte l'analyse du comportement de ses visiteurs.
- **Collaborateur de boutique :** membre de l'équipe d'un commerçant, doté d'un rôle aux permissions limitées (par exemple gestion des commandes uniquement).

- **Visiteur :** internaute non connecté qui parcourt la vitrine d'une boutique, recherche des produits, reçoit des recommandations, remplit son panier et peut utiliser la cabine d'essayage virtuel.
- **Client :** visiteur inscrit sur une boutique ; son compte n'existe que dans cette boutique, et la même personne peut avoir un autre compte, avec un autre mot de passe, dans une autre boutique. En plus des actions du visiteur, il passe commande, paie en ligne, suit ses commandes, demande un retour, publie des avis et cumule des points de fidélité.

Acteurs secondaires

- **Stripe :** prestataire de paiement. Il traite les paiements des commandes (*PaymentIntent*) et l'enregistrement de la carte du commerçant lors de la vérification (*SetupIntent*).
- **Decart :** service d'intelligence artificielle générative qui produit en temps réel la vidéo d'essayage virtuel à partir du flux caméra du client et de l'image du vêtement.
- **Brevo [7] :** service d'envoi d'e-mails transactionnels et marketing (codes de vérification, réinitialisation du mot de passe, invitations, confirmations de commande, newsletters).
- **GitHub Actions et Dependabot :** services de GitHub qui construisent, testent et publient la plateforme à chaque modification, et proposent les mises à jour de dépendances.
- **Moteur d'apprentissage automatique :** ensemble de scripts Python (scikit-learn) appelés par le microservice d'analyse pour entraîner les modèles et produire recommandations, segments et scores d'attrition.

2.1.2 Spécification des besoins

Nous classons les besoins en deux familles : les besoins fonctionnels, qui décrivent **ce que** le système doit faire, et les besoins non fonctionnels, qui décrivent **comment** il doit le faire.

Besoins fonctionnels

Les besoins sont regroupés par module, ce qui reflète le découpage de la plateforme en microservices. Pour chaque fonctionnalité, nous précisons le ou les acteurs concernés.

Module Plateforme, authentification et vérification Le tableau 2.1 présente les besoins de ce module.

Fonctionnalité	**Acteur(s) concerné(s)**
S'inscrire et s'authentifier de façon sécurisée (JWT et jeton de rafraîchissement)	Commerçant, Client, Administrateurs Sellio
Confirmer son adresse par un code envoyé par e-mail ; réinitialiser son mot de passe	Commerçant, Client, Brevo
Avoir un compte client distinct dans chaque boutique	Client
Créer une boutique (nom, secteur, identité visuelle)	Commerçant
Soumettre un dossier de vérification : carte bancaire, pièce d'identité (CIN ou passeport) et selfie	Commerçant
Valider ou refuser un dossier de vérification avec motif	Administrateur Sellio
Consulter les statistiques, les boutiques et leurs propriétaires	Administrateur Sellio
Suspendre une boutique, bloquer un commerçant	Administrateur Sellio
Supprimer un commerçant avec sa boutique et toutes ses données	Administrateur Sellio
Ajouter un administrateur, changer le rôle d'un compte de la plateforme	Super administrateur Sellio
Gérer les rôles et permissions de l'équipe de la boutique	Commerçant

TABLE 2.1 – Besoins fonctionnels du module Plateforme, authentification et vérification

Module Catalogue et vitrine Le tableau 2.2 présente les besoins de ce module.

Fonctionnalité	**Acteur(s) concerné(s)**
Gérer les produits, leurs images et leurs variantes	Commerçant, Collaborateur
Renseigner une fiche adaptée au secteur et au type du produit (bague, montre, chaussures, machine…)	Commerçant, Collaborateur
Choisir les valeurs dans des listes normalisées et les étendre	Commerçant, Collaborateur
Gérer les catégories	Commerçant, Collaborateur
Choisir le secteur et l'un des huit modèles de vitrine	Commerçant
Personnaliser l'apparence : logo et taille, couleurs, polices, barre d'annonce, coordonnées, réseaux sociaux	Commerçant
Personnaliser la page d'accueil (sections, ordre, textes, produits mis en avant) avec aperçu en direct	Commerçant
Parcourir la vitrine, filtrer et rechercher les produits	Visiteur, Client

TABLE 2.2 – Besoins fonctionnels du module Catalogue et vitrine

Module Transactions Le tableau 2.3 présente les besoins de ce module.

Fonctionnalité	**Acteur(s) concerné(s)**
Gérer un panier (taille obligatoire pour les articles concernés)	Visiteur, Client
Passer commande avec livraison, TVA et code promo	Client
Payer en ligne par carte	Client, Stripe
Suivre ses commandes et demander un retour	Client
Gérer les commandes, leurs statuts et les retours	Commerçant, Collaborateur
Paramétrer la TVA, les zones de livraison et la politique de retour	Commerçant
Publier un avis ; modérer et répondre aux avis	Client ,Commerçant

TABLE 2.3 – Besoins fonctionnels du module Transactions

Module Marketing et fidélisation Le tableau 2.4 présente les besoins de ce module.

Fonctionnalité	**Acteur(s) concerné(s)**
Créer des codes promo et des remises automatiques	Commerçant
Publier des bannières ciblées par segment	Commerçant
Envoyer une newsletter ou contacter un client par e-mail	Commerçant, Brevo
Configurer le programme de fidélité et ses niveaux	Commerçant
Consulter son solde de points et son niveau	Client

TABLE 2.4 – Besoins fonctionnels du module Marketing et fidélisation

Module Intelligence (essayage virtuel et apprentissage automatique) Le tableau 2.5 présente les besoins de ce module.

Fonctionnalité	**Acteur(s) concerné(s)**
Essayer virtuellement un vêtement en temps réel avec sa caméra	Visiteur, Client, Decart
Collecter les événements de navigation	Visiteur, Client (implicite)
Recevoir des recommandations : produits similaires, pour vous, souvent achetés ensemble, populaires	Visiteur, Client, Moteur ML
Consulter l'analyse comportementale : entonnoir, segments de visiteurs, produits consultés	Commerçant
Identifier les clients à risque d'attrition	Commerçant, Moteur ML
Ré-entraîner les modèles de la boutique	Commerçant, Moteur ML

TABLE 2.5 – Besoins fonctionnels du module Intelligence

Module Qualité et déploiement Le tableau 2.6 présente les besoins de ce module.

Fonctionnalité	**Acteur(s) concerné(s)**
Construire et tester automatiquement chaque modification	Équipe de développement , GitHub Actions
Construire et publier les images Docker de la plateforme	GitHub Actions
Lancer toute la plateforme en une commande	Équipe de développement
Proposer les mises à jour de dépendances	Dependabot

TABLE 2.6 – Besoins fonctionnels du module Qualité et déploiement

Besoins non fonctionnels

Les besoins non fonctionnels définissent les qualités attendues du système.

Le tableau 2.7 les présente selon le modèle de qualité ISO/IEC 25010 [8].

TABLE 2.7 – Exigences non fonctionnelles du système

Caractéristique	**Sous-caractéristique**	**Description**
Sécurité	Authentification	Jeton d'accès JWT (24 h) et jeton de rafraîchissement (7 jours) ; mots de passe hachés avec BCrypt.
	Contrôle d'accès	Rôles et permissions vérifiés côté serveur ; un commerçant n'accède qu'aux données de sa propre boutique ; l'équipe Sellio n'accède à aucune donnée de client.
	Protection des paiements	Aucune donnée de carte ne transite par nos serveurs ; seuls la marque, les 4 derniers chiffres, l'expiration et la référence Stripe sont stockés (PCI-DSS).
	Confiance	Une boutique reste ń en attente ż tant que son dossier KYC n'est pas validé ; un compte bloqué ne peut plus se connecter.
	Secrets	Aucune clé dans le code ni dans les images : elles sont lues au démarrage depuis un fichier `.env` exclu du dépôt.
Performance	Temps de réponse	Recommandations servies depuis des modèles pré-entraînés ; les événements sont envoyés par lots pour ne pas ralentir la navigation.
	Volume de données	Table d'événements partitionnée par mois, index BRIN sur la date, agrégats quotidiens et rétention de 13 mois.
Fiabilité	Tolérance aux pannes	La panne d'un microservice (par exemple l'analyse) n'empêche pas les achats ; la vitrine masque simplement les sections de recommandations.
	Intégrité	Transactions ACID pour les commandes et les paiements.
Utilisabilité	Ergonomie	Interfaces responsives ; thème sombre par défaut sur l'espace Sellio avec bascule vers le thème clair.
	Personnalisation	Toute modification du thème est visible immédiatement sur la vitrine, sans écrire de code.
Maintenabilité	Modularité	Un microservice par domaine métier, une bibliothèque partagée pour les entités et DTO.
	Évolutivité	Ajout d'un modèle de vitrine, d'un secteur, d'un type de produit ou d'un modèle ML sans toucher aux autres services.
	Testabilité	Tests automatisés (JUnit, Vitest, pytest) exécutés à chaque modification.

Suite à la page suivante

Caractéristique	Sous-caractéristique	Description
Portabilité	Compatibilité	Navigateurs modernes (Chrome, Firefox, Edge) ; l'essayage virtuel exige une caméra et WebRTC.
	Déploiement	Chaque service est livré sous forme d'image Docker ; la plateforme se lance en une commande (Docker Compose).
Confidentialité	Données personnelles	Le suivi comportemental repose sur un identifiant de visiteur anonyme ; les pièces d'identité ne sont visibles que par l'administrateur Sellio.

2.1.3 Diagramme de cas d'utilisation général

Le système est utilisé par six acteurs principaux : **le Super administrateur** et **l'Administrateur Sellio**, **le Commerçant**, **le Collaborateur de boutique**, **le Visiteur** et **le Client**. Les sous-sections suivantes présentent les cas d'utilisation de chacun.

2.1.3.1 Cas d'utilisation de l'équipe Sellio

La figure 2.1 présente les cas d'utilisation des administrateurs de la plateforme ; le super administrateur hérite de tous les cas de l'administrateur.

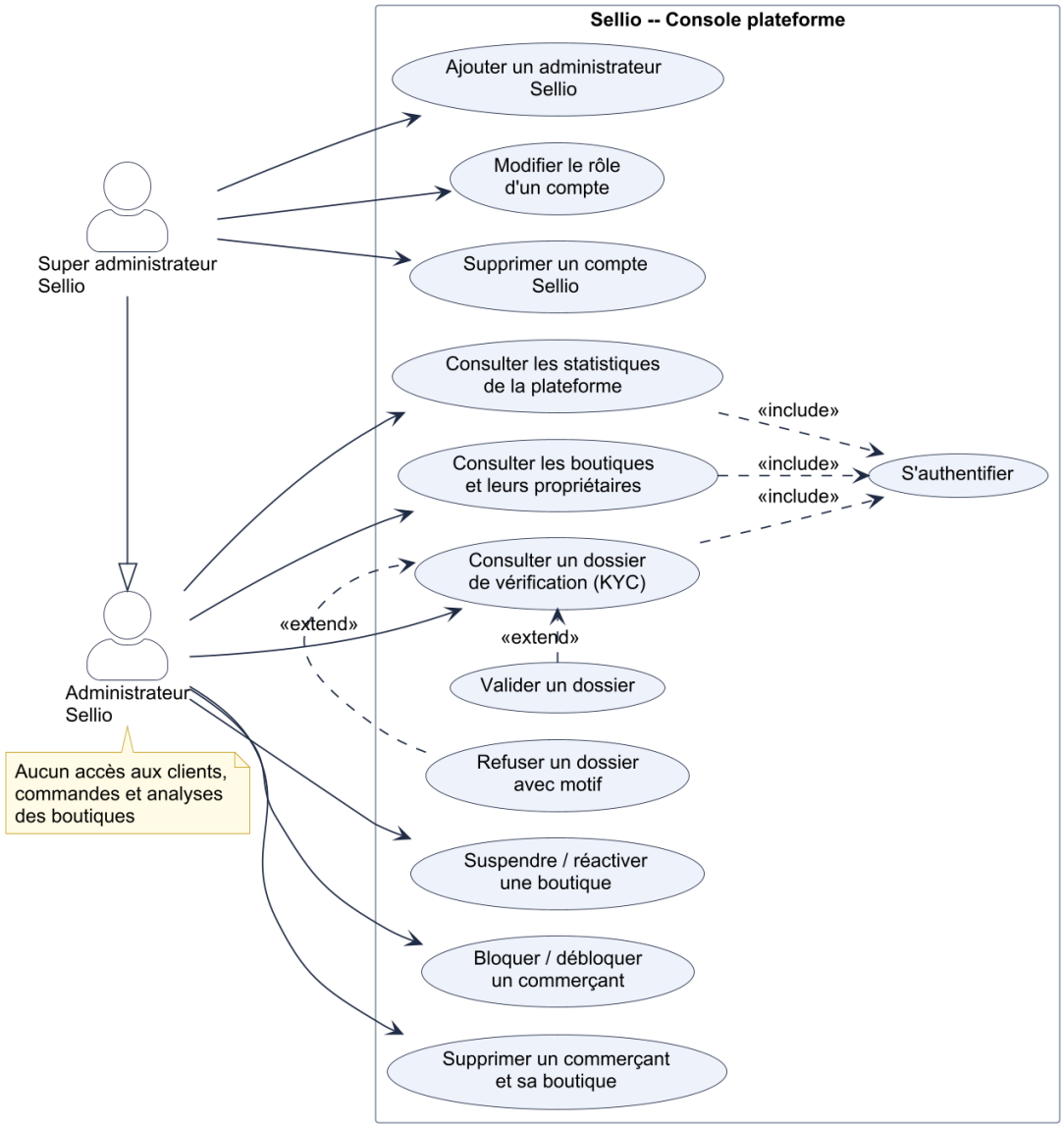


FIGURE 2.1 – Diagramme de cas d'utilisation de l'équipe Sellio

2.1.3.2 Cas d'utilisation du Commerçant

La figure 2.2 présente les cas d'utilisation du commerçant. Le collaborateur de boutique hérite d'une partie de ces cas, selon les permissions de son rôle.

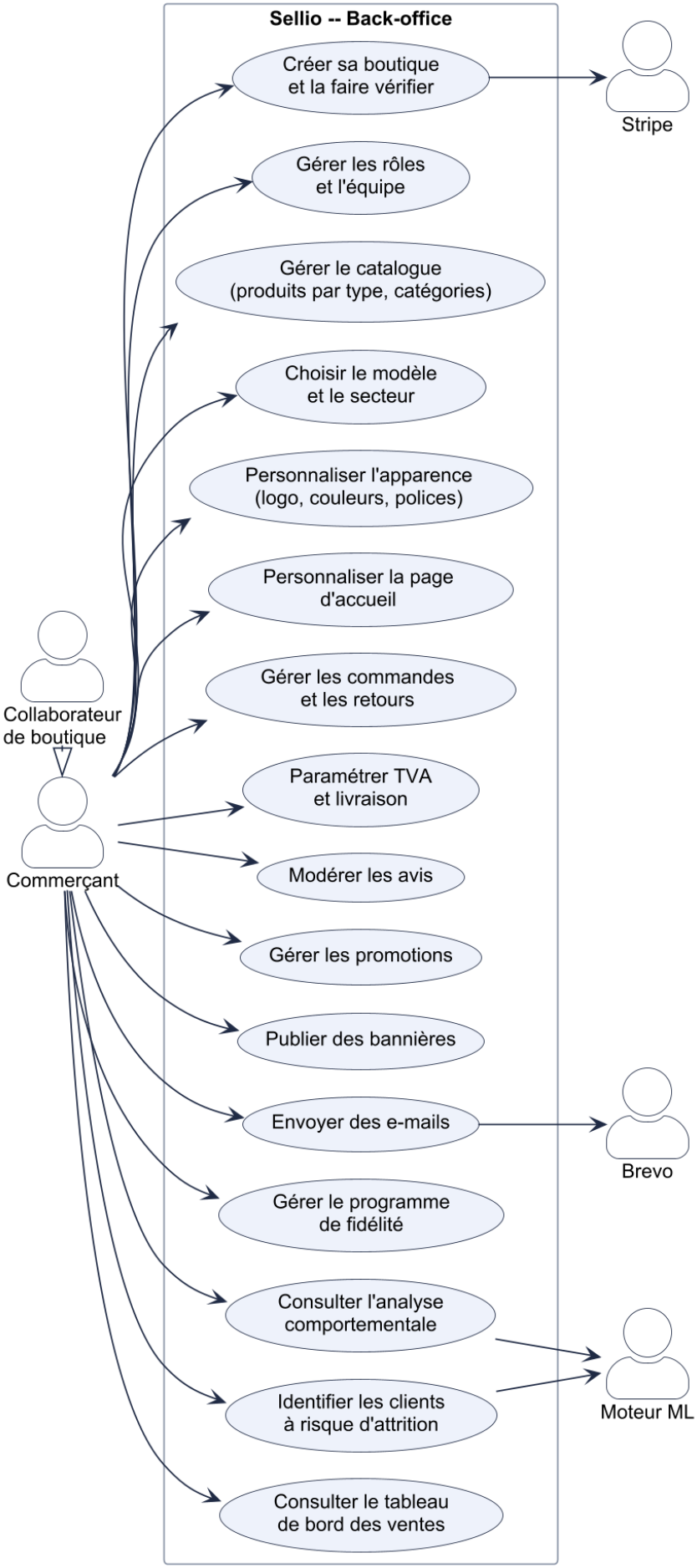


FIGURE 2.2 – Diagramme de cas d'utilisation du Commerçant

2.1.3.3 Cas d'utilisation du Visiteur et du Client

La figure 2.3 présente les cas d'utilisation du visiteur et du client. Le client hérite de tous les cas du visiteur.

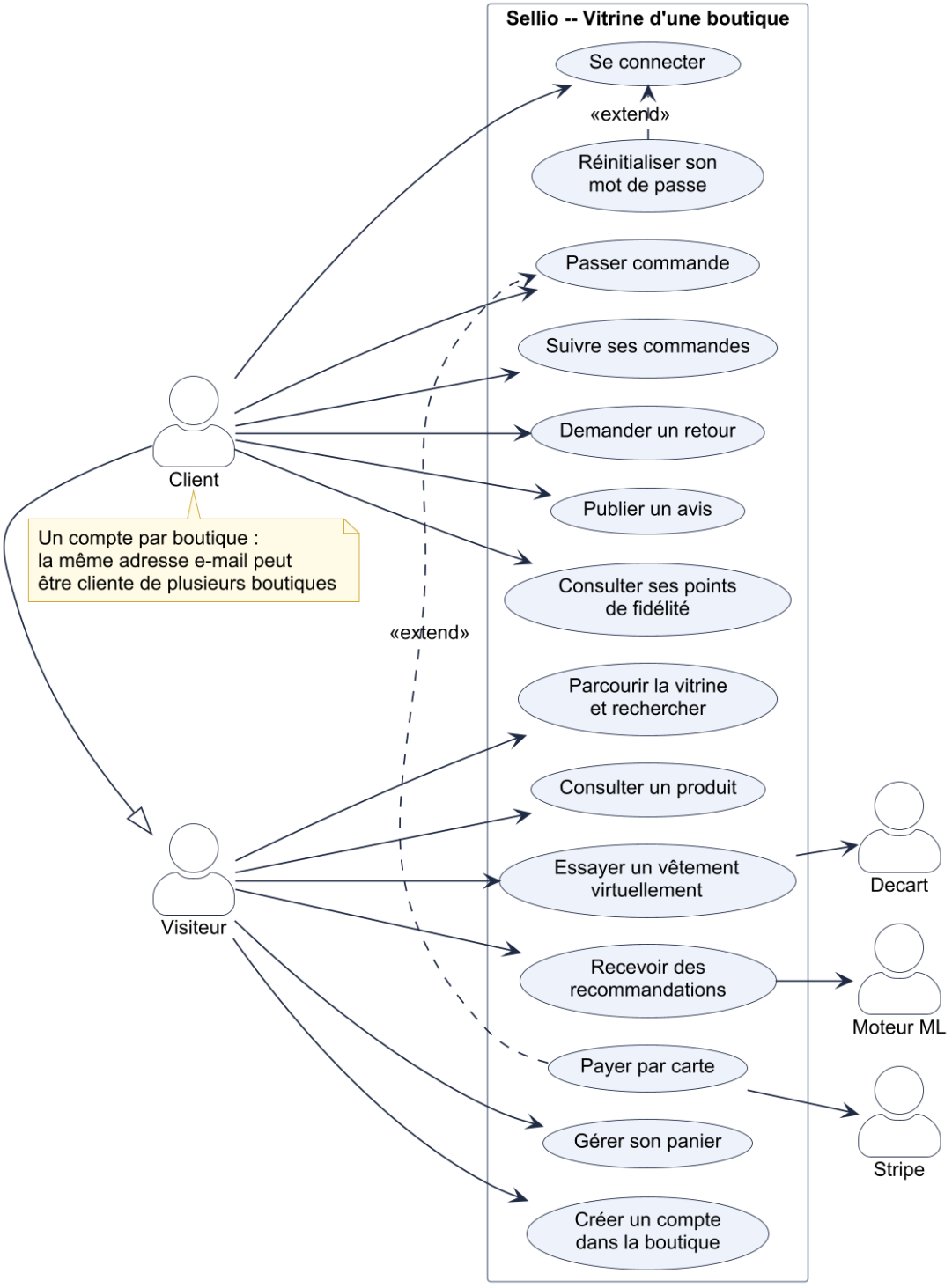


FIGURE 2.3 – Diagramme de cas d'utilisation du Visiteur et du Client

2.2 Analyse globale

2.2.1 Diagramme de classes d'analyse

La figure 2.4 présente le diagramme de classes d'analyse. La classe centrale est `Shop` : chaque utilisateur, produit, catégorie, commande, promotion ou événement comportemental est rattaché à une boutique, ce qui matérialise l'isolation multi-boutiques. Une boutique possède un dossier de vérification (`MerchantVerification`), un statut (`PENDING`, `ACTIVE`, `REJECTED`, `SUSPENDED`), un secteur, un modèle de vitrine, des couleurs (`theme`) et sa personnalisation (`settings`, objet JSON : logo, coordonnées, page d'accueil…). Le diagramme présente le modèle du domaine dans son ensemble ; physiquement, ces classes sont réparties entre les bases des microservices (tableau 2.9), et les associations qui franchissent une frontière de service (une commande et son client, un avis et son auteur) sont matérialisées par un identifiant. Les caractéristiques propres au secteur d'un produit sont rangées dans `Product.attributes`, ce qui évite une table par secteur. Les codes envoyés par e-mail (`AuthCode`) ne sont stockés que sous forme d'empreinte.

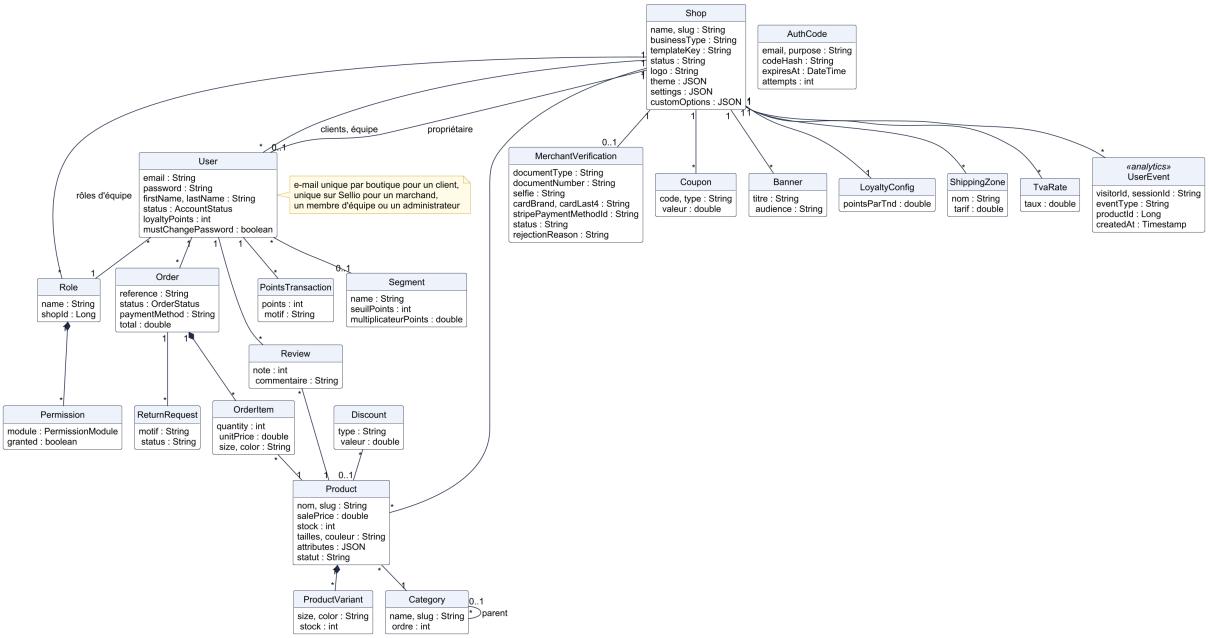


FIGURE 2.4 – Diagramme de classes d'analyse

2.2.2 Architecture globale

L'architecture influence directement les performances, la maintenabilité et l'évolutivité de la plateforme. Trois approches ont été comparées : **l'architecture monolithique**, **l'architecture N-tiers** et **l'architecture microservices**.

Le tableau 2.8 présente leurs avantages et leurs limites.

Architecture	**Caractéristiques**
Monolithique	— Présentation, logique métier et accès aux données dans une seule unité de déploiement. — **Avantages** : simplicité de développement et de déploiement, débogage facile, cohérence transactionnelle native. — **Limites** : mise à l'échelle uniquement globale, un défaut dans un module peut affecter tout le système, maintenance difficile quand le code grossit.
Client–serveur N-tiers	— Séparation en couches : présentation, logique métier, données. — **Avantages** : séparation claire des responsabilités, plusieurs clients peuvent consommer la même couche métier. — **Limites** : le serveur d'application reste une unité de déploiement unique, avec un couplage interne entre modules.
Microservices	— Décomposition en services indépendants, chacun responsable d'un domaine métier, déployés séparément et communiquant par API REST derrière une passerelle. — **Avantages** : mise à l'échelle service par service, isolation des pannes, déploiements indépendants, liberté technologique par service. — **Limites** : complexité opérationnelle, latence réseau entre services, cohérence des données distribuées plus difficile.

TABLE 2.8 – Comparaison des architectures logicielles étudiées

L'**architecture microservices** a été retenue, pour trois raisons. D'abord, le sujet lui-même demande une plateforme distribuée. Ensuite, les domaines de Sellio ont des charges très différentes : le service d'analyse reçoit un flux continu d'événements et lance des entraînements de modèles coûteux, alors que le service d'identité traite peu de requêtes ; les séparer permet de dimensionner chacun selon sa charge et d'éviter qu'un entraînement ne ralentisse les achats. Enfin, le projet est né d'un monolithe (NaturEssence) dont la croissance rendait les évolutions de plus en plus risquées ; la migration vers des services séparés a rendu chaque domaine plus facile à faire évoluer.

2.2.2.1 Une base de données par microservice

Chaque microservice possède **sa propre base de données** et il est le seul à y accéder (*database per service*). Un service ne lit jamais les tables d'un autre : il lui demande l'information par une API interne. Ce principe garantit qu'un service peut faire évoluer son schéma, être sauvegardé ou dimensionné sans effet sur les autres, et qu'une panne de base n'atteint qu'un seul domaine. Le tableau 2.9 présente la répartition des données.

Base	Propriétaire	Données
`sellio_auth`	auth-service	Comptes, rôles et permissions, segments, boutiques et leur personnalisation, dossiers KYC, codes e-mail, jetons de rafraîchissement, fidélité (configuration et points)
`sellio_catalog`	catalog-service	Produits, variantes, catégories, collections, avis
`sellio_orders`	order-service	Commandes et lignes, retours et politique de retour, codes promo et utilisations, remises, zones de livraison, TVA
`sellio_marketing`	marketing-service	Bannières, apparence par défaut de Sellio
`sellio_analytics`	analytics-service	Événements (partitionnés par mois), agrégats quotidiens, recommandations, segments, historique des entraînements

Table 2.9 – Répartition des données entre les bases des microservices

Ce découpage a trois conséquences sur la conception :

— **Références par identifiant.** Une commande ne pointe pas vers l'entité client d'un autre service : elle conserve l'identifiant du client et une copie des informations utiles à son affichage (nom, e-mail, adresse ; nom et prix de chaque article au moment de l'achat).

— **API internes entre services.** Les échanges passent par des routes `/internal/**` protégées par un jeton de service et non exposées par la passerelle : le service de commandes demande au catalogue de réserver le stock et au service d'identité de créditer les points de fidélité ; le service d'analyse demande chaque nuit au catalogue les produits et au service de commandes les commandes payées.

— **Boutique connue de tous sans lecture de la base d'identité.** Le jeton JWT porte l'identifiant et le *slug* de la boutique de l'utilisateur ; pour les requêtes publiques de la vitrine, la correspondance *slug* / boutique est obtenue auprès du service d'identité et mise en cache.

Les cinq bases sont hébergées par le même serveur PostgreSQL, ce qui suffit à une plateforme de cette taille ; chacune pourra être déplacée sur son propre serveur sans changer le code, seule son adresse de connexion étant à modifier. Une bibliothèque commune (`shared-lib`) regroupe les DTO et la sécurité ; chaque service ne déclare que les entités de sa propre base.

2.2.2.2 Choix du système de gestion de base de données

Le tableau 2.10 résume la comparaison qui nous a conduits à retenir PostgreSQL.

Critère	**MongoDB**	**PostgreSQL**
Transactions (commande + paiement + stock)	Multi-documents possibles mais plus coûteuses	ACID natives
Intégrité référentielle (boutique, client, commande)	À gérer dans l'application	Clés étrangères et contraintes
Données flexibles (attributs, thème)	Natif	Type JSONB
Volume d'événements	Bon	Partitionnement déclaratif et index BRIN
Requêtes analytiques (agrégats, jointures)	Pipeline d'agrégation	SQL standard, fonctions de fenêtre
Une base par service sur un même serveur	Oui	Oui (bases isolées, rôles distincts)

Table 2.10 – Comparaison MongoDB / PostgreSQL

Les données de Sellio sont avant tout relationnelles et transactionnelles (une commande relie un client, des produits, une boutique et un paiement) ; PostgreSQL couvre en outre le besoin de flexibilité (JSONB) et de volume (partitionnement), d'où ce choix [9].

2.2.2.3 Architecture logique de la plateforme

La figure 2.5 présente l'architecture logique de Sellio et ses couches.

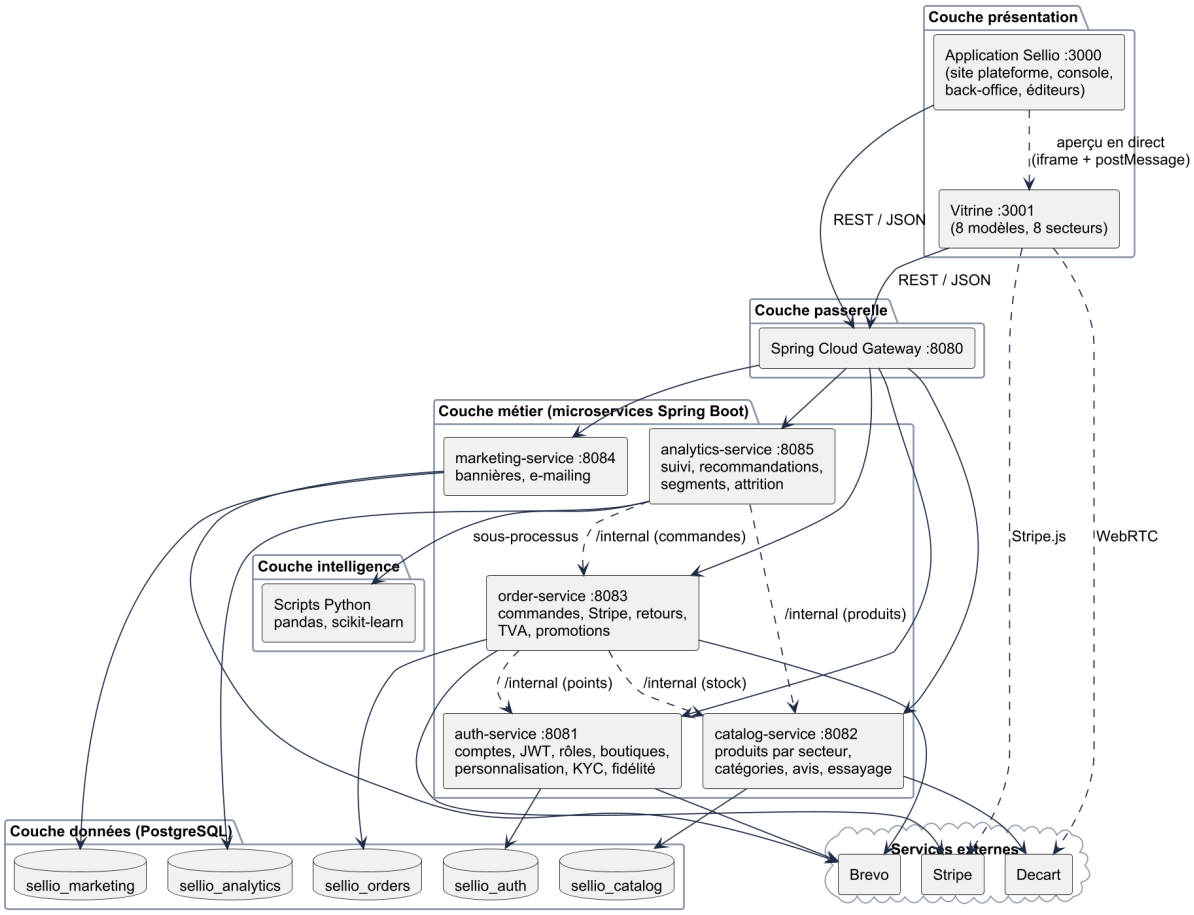


FIGURE 2.5 – Architecture logique de la plateforme Sellio

La couche présentation (front-end) :

Elle se compose de deux applications React :

— **l'application Sellio** (port 3000), qui regroupe le site de la plateforme (accueil, inscription, connexion), la console de l'administrateur Sellio et le back-office de chaque commerçant ;

— **la vitrine** (port 3001), commune à toutes les boutiques : la boutique affichée est déterminée par son identifiant (*slug*) ; son modèle, ses couleurs, ses polices et sa page d'accueil sont chargés dynamiquement. Les éditeurs du back-office l'affichent dans un cadre pour en montrer l'aperçu en direct.

La couche passerelle :

Une passerelle **Spring Cloud Gateway** (port 8080) est le point d'entrée unique des API. Elle route chaque requête vers le bon microservice selon son chemin (`/api/v1/auth/**` vers le service d'identité, `/api/v1/public/products/**` vers le catalogue, etc.).

La couche métier (back-end) :

Elle est composée de cinq microservices Spring Boot :

— **auth-service** (8081) : comptes, JWT, rôles et permissions, boutiques et leur personnalisation, vérification KYC, console plateforme, fidélité et segments ;

— **catalog-service** (8082) : produits et leurs caractéristiques par secteur, variantes, catégories, avis, téléversement d'images et jeton d'essayage virtuel ;

— **order-service** (8083) : tunnel de commande, paiement Stripe, commandes, retours, TVA, livraison, promotions et tableau de bord ;

— **marketing-service** (8084) : bannières et e-mailing ;

— **analytics-service** (8085) : collecte des événements, recommandations, analyse comportementale et prédiction de l'attrition.

La couche intelligence :

Des scripts **Python** (pandas, scikit-learn) entraînent et appliquent les modèles. Le service d'analyse les exécute comme sous-processus et stocke les modèles entraînés par boutique.

La couche données :

Chaque microservice dispose de sa propre base **PostgreSQL** (`sellio_auth`, `sellio_catalog`, `sellio_orders`, `sellio_marketing`, `sellio_analytics`) ; les images de produits sont stockées sur disque et servies par le catalogue.

Les services externes :

Stripe (paiements), **Decart** (essayage virtuel) et **Brevo** (e-mails).

2.2.2.4 Architecture physique de la plateforme

La figure 2.6 présente le déploiement physique : chaque composant s'exécute dans un conteneur Docker, orchestré par Docker Compose sur un même hôte. Le navigateur communique en HTTPS avec les conteneurs nginx qui servent les applications React et avec la passerelle ; celle-ci relaie les appels vers les microservices. Le conteneur PostgreSQL héberge les cinq bases, une par microservice, dont les données sont conservées dans un volume ; les services communiquent entre eux par leurs API internes. Le navigateur communique directement avec Stripe (saisie de carte) et avec Decart (flux vidéo WebRTC), les jetons nécessaires étant délivrés par nos services.

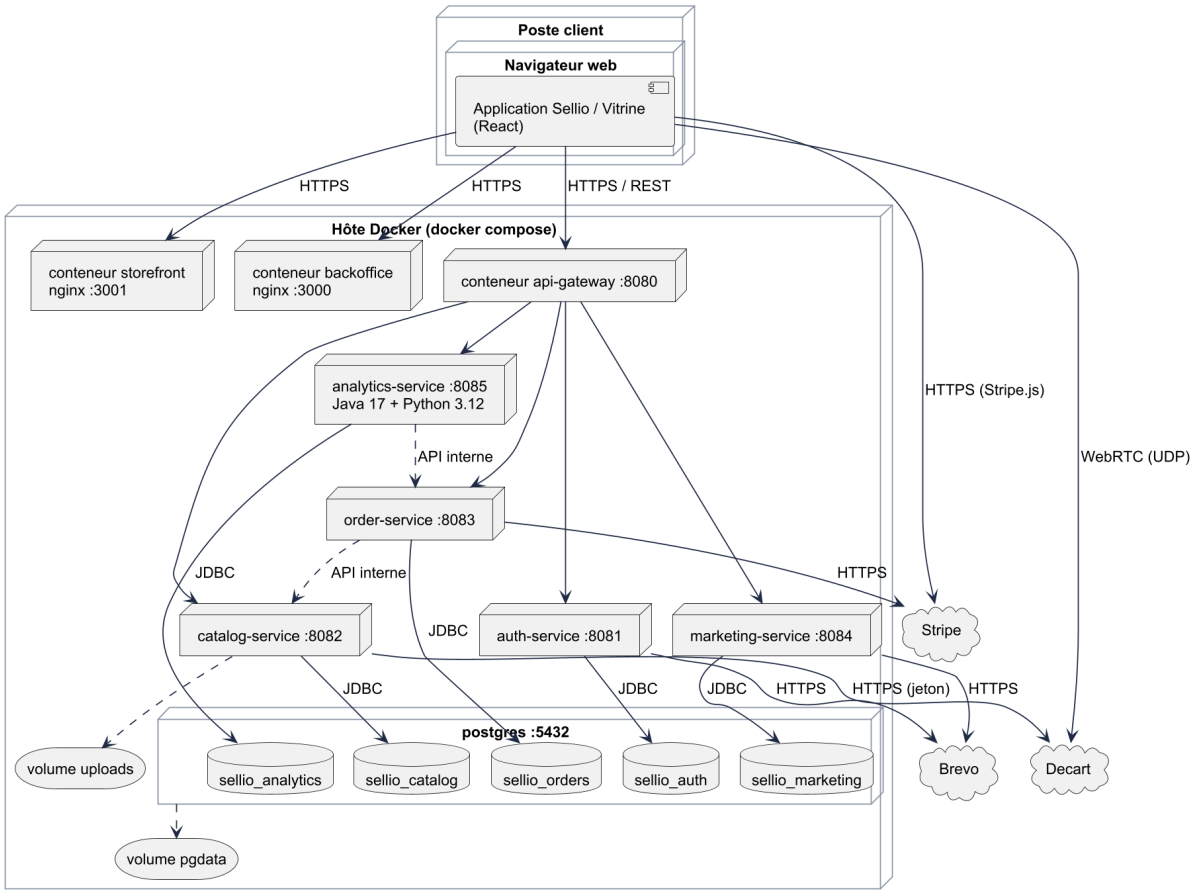


FIGURE 2.6 – Architecture physique de la plateforme Sellio

2.3 Pilotage du projet avec Scrum

2.3.1 Backlog du produit

Le backlog produit regroupe les principales fonctionnalités de Sellio. Afin de conserver une vue globale du projet, les besoins sont ici synthétisés par grands modules. Les user stories détaillées sont présentées dans les sprints correspondants.

Module	Fonctionnalités principales	Complexité
Authentification et utilisateurs	Inscription, connexion, gestion des profils, rôles, permissions et réinitialisation du mot de passe.	Élevée
Plateforme et boutiques	Création de boutiques, gestion multi-boutiques, vérification KYC et console d'administration Sellio.	Élevée
Catalogue et vitrine	Gestion des produits et catégories, fiches adaptées aux huit secteurs, modèles de vitrine et personnalisation.	Élevée
Parcours transactionnel	Panier, commande, paiement Stripe, gestion des commandes, retours, TVA, livraison et avis.	Élevée
Marketing et fidélisation	Promotions, bannières, e-mailing, programme de fidélité et gestion des niveaux clients.	Moyenne
Essayage virtuel	Essayage de vêtements en temps réel à partir de la caméra grâce à l'API Decart.	Élevée
Intelligence comportementale	Collecte des événements, recommandations personnalisées, segmentation des visiteurs et prédiction de l'attrition.	Élevée
Qualité et DevOps	Tests automatisés, conteneurisation avec Docker et pipeline CI/CD avec GitHub Actions.	Élevée

TABLE 2.11 – Synthèse du backlog produit de Sellio

2.4 Environnement de travail

2.4.1 Environnement matériel de développement

Le développement a été réalisé sur un ordinateur portable présentant les caractéristiques suivantes :

— **Modèle :** MSI GF63 Thin 10SC

— **Système d'exploitation :** Microsoft Windows 11 Professionnel (64 bits)

— **Processeur :** Intel Core i5-10500H @ 2.50 GHz, 6 curs et 12 processeurs logiques

— **Mémoire RAM :** 16 Go

2.4.2 Environnement logiciel de développement

Le tableau 2.12 présente les principaux outils utilisés.

Outil	Description	Logo
Visual Studio Code [10]	Éditeur principal pour le front-end React, le back-end Spring Boot et les scripts Python.	
Git et GitHub [11]	Gestion de versions et hébergement du dépôt du projet.	
PlantUML [12]	Génération des diagrammes UML à partir de descriptions textuelles.	
PostgreSQL [9]	Système de gestion de base de données relationnelle.	
Maven [13]	Construction et gestion des dépendances des microservices Java.	
Stripe Dashboard [14]	Suivi des paiements et des cartes en mode test.	
Docker [15]	Images des services et des front-ends ; Docker Compose pour lancer la plateforme.	
GitHub Actions [16]	Intégration et livraison continues, publication des images.	

TABLE 2.12 – Outils utilisés pour la modélisation, le développement et le suivi

2.4.2.1 Langages de programmation

Le tableau 2.13 présente les langages utilisés.

Langage	Usage	Logo
Java 17 [17]	Microservices (API REST, logique métier, sécurité).	
JavaScript (JSX) [18]	Applications React : Sellio, back-office et vitrine.	
Python 3 [19]	Nettoyage des données, entraînement et évaluation des modèles.	
SQL	Requêtes, partitionnement et agrégats PostgreSQL.	

TABLE 2.13 – Langages de programmation utilisés

2.4.2.2 Frameworks et bibliothèques

Le tableau 2.14 présente les frameworks et bibliothèques utilisés.

Framework	**Rôle**	**Logo**
Spring Boot 3.2 [20]	Socle des microservices (Spring Web, Spring Data JPA, Spring Security).	
Spring Cloud Gateway [21]	Passerelle API : point d'entrée unique et routage vers les services.	
React 18 [22] et Vite [23]	Interfaces web à base de composants ; serveur de développement et compilation.	
Tailwind CSS [24]	Mise en forme utilitaire ; variables CSS pour les thèmes des vitrines.	
scikit-learn [25], pandas, NumPy	Modèles d'apprentissage automatique et préparation des données.	
Stripe.js [14]	Saisie sécurisée de la carte et confirmation des paiements.	
SDK Decart [26]	Connexion WebRTC au modèle d'essayage virtuel en temps réel.	
JUnit 5 [27], Vitest [28], pytest [29]	Tests automatisés du back-end, du front-end et des modèles.	

TABLE 2.14 – Frameworks et bibliothèques utilisés

Conclusion

Dans ce chapitre, nous avons identifié les acteurs, spécifié les besoins fonctionnels et non fonctionnels, justifié l'architecture microservices et le choix de PostgreSQL, puis présenté le backlog produit et la planification en sept releases et douze sprints. Le chapitre suivant présente la première release.

Chapitre 3

Release 1 : Fondations de la plateforme Sellio

Introduction

Ce chapitre présente la première release, qui pose les fondations de Sellio. Le premier sprint découpe l'application existante en microservices derrière une passerelle et sécurise l'accès (JWT, rôles et permissions, comptes clients propres à chaque boutique, réinitialisation du mot de passe). Le second transforme la boutique unique en plateforme multi-boutiques : inscription des commerçants confirmée par un code envoyé par e-mail, assistant de création de boutique, vérification d'identité (KYC) et console d'administration de la plateforme, avec ses deux niveaux d'administrateurs.

3.1 Backlog du Sprint 1

Le tableau 3.1 présente le **backlog du Sprint 1**, consacré à l'architecture microservices, à l'authentification et au contrôle d'accès.

User Story	Tâches	Estimation
En tant qu'utilisateur, je souhaite créer un compte	Découper l'application en microservices (auth, catalog, order, marketing, analytics) et une bibliothèque partagée.	1
	Mettre en place la passerelle Spring Cloud Gateway et ses règles de routage.	1
	Développer l'API d'inscription avec hachage BCrypt.	1
	Rattacher chaque compte client à une boutique : l'e-mail d'un client est unique dans sa boutique, celui d'un commerçant, d'un membre d'équipe ou d'un administrateur l'est sur toute la plateforme (deux index uniques partiels).	1
	Développer l'interface d'inscription.	1
	Tester la fonctionnalité.	1

Suite à la page suivante

User Story	Tâches	Estimation
En tant qu'utilisateur, je souhaite me connecter	Développer l'API de connexion et la génération du jeton JWT signé.	1
	Développer le filtre JWT partagé par tous les microservices ; identifier l'utilisateur par son identifiant (et non par son e-mail, qui peut exister dans plusieurs boutiques).	1
	Chercher le compte parmi les clients de la boutique (connexion depuis une vitrine) ou parmi les comptes Sellio ; conserver la session de chaque vitrine séparément dans le navigateur.	1
	Développer l'interface de connexion et la gestion des erreurs.	1
	Tester la fonctionnalité.	1
En tant qu'utilisateur, je souhaite rester connecté	Développer le jeton de rafraîchissement persistant et l'API `/refresh`.	1
	Renouveler automatiquement le jeton côté client (intercepteur HTTP).	1
En tant qu'utilisateur, je souhaite me déconnecter	Développer l'API de déconnexion (révocation du jeton de rafraîchissement) et le bouton associé.	1
En tant qu'utilisateur, je souhaite réinitialiser mon mot de passe	Générer un code à six chiffres (empreinte SHA-256, validité 10 minutes, 3 essais, un envoi par minute), propre à la boutique pour un client.	1
	Envoyer le code par e-mail (Brevo) aux couleurs de la boutique ou de Sellio.	1
	Développer les pages ń Mot de passe oublié ż de la vitrine et de Sellio.	1
En tant que client, je souhaite consulter et modifier mon profil	Développer l'API et l'interface ń Mon profil ż.	1
	Tester la fonctionnalité.	1

Suite à la page suivante

User Story	Tâches	Estimation
En tant que commerçant, je souhaite créer des rôles et leur associer des permissions	Développer l'API CRUD des rôles et l'association des permissions.	1
	Développer l'interface ń Rôles et permissions ż (matrice des permissions).	1
	Tester la fonctionnalité.	1
En tant que commerçant, je souhaite gérer les comptes de mon équipe et de mes clients	Développer l'API de gestion des utilisateurs (création, recherche, filtres, statut, statistiques).	1
	Développer les interfaces ń Clients ż, ń Détail client ż et ń Ajouter un compte ż.	1
	Inviter un membre de l'équipe : mot de passe temporaire envoyé par e-mail, à remplacer obligatoirement à la première connexion.	1
	Tester la fonctionnalité.	1
En tant que système, je veux restreindre chaque endpoint selon les permissions	Configurer Spring Security dans chaque microservice (endpoints publics, profil, administration).	1
	Protéger les routes du back-office côté interface (garde d'authentification).	1
	Total	**28**

TABLE 3.1 – Backlog du Sprint 1

3.2 Diagramme de cas d'utilisation du Sprint 1

La figure 3.1 présente le diagramme de cas d'utilisation du premier sprint.

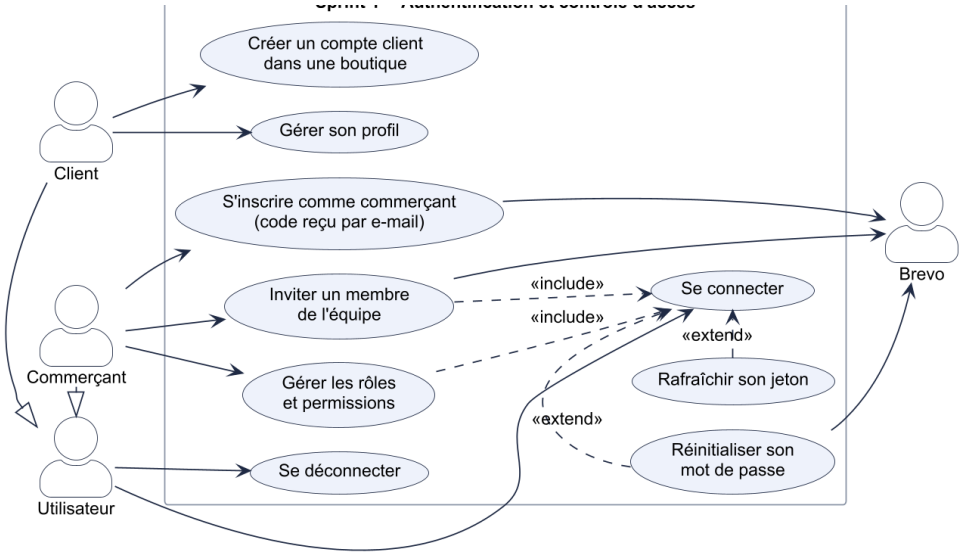


FIGURE 3.1 – Diagramme de cas d'utilisation du Sprint 1

3.3 Services web du Sprint 1

Les services web permettent aux applications de communiquer via des API REST. Toutes les requêtes passent par la passerelle (`http://localhost:8080`), qui les route vers le service compétent. Le tableau 3.2 présente les services web du Sprint 1.

Méthode	**URL**	**Description**
Authentification — /api/v1/auth (auth-service)		
POST	/api/v1/auth/register	Créer un compte client dans une boutique et retourner les jetons.
POST	/api/v1/auth/login	Authentifier un utilisateur et retourner le jeton d'accès et le jeton de rafraîchissement.
POST	/api/v1/auth/refresh	Obtenir un nouveau jeton d'accès.
POST	/api/v1/auth/logout	Révoquer le jeton de rafraîchissement.
POST	/api/v1/auth/forgot-password	Envoyer un code de réinitialisation (boutique précisée depuis une vitrine).
POST	/api/v1/auth/reset-password	Choisir un nouveau mot de passe avec le code reçu.
POST	/api/v1/auth/change-password	Changer son mot de passe (obligatoire après un mot de passe temporaire).
GET	/api/v1/auth/me	Retourner l'utilisateur connecté, son rôle et sa boutique.
Profil — /api/v1/profile		

Méthode	URL	Description
GET	/api/v1/profile	Consulter son profil.
PUT	/api/v1/profile	Modifier son profil.
Rôles et permissions — /api/v1/admin/roles		
GET	/api/v1/admin/roles	Lister les rôles.
POST	/api/v1/admin/roles	Créer un rôle.
PUT	/api/v1/admin/roles/{id}	Modifier un rôle.
DELETE	/api/v1/admin/roles/{id}	Supprimer un rôle.
PUT	/api/v1/admin/roles/{id}/permissions	Associer des permissions à un rôle.
GET	/api/v1/admin/roles/permissions-matrix	Obtenir la matrice rôles / permissions.
Utilisateurs — /api/v1/admin/users		
GET	/api/v1/admin/users	Lister les utilisateurs de la boutique.
POST	/api/v1/admin/users	Créer un compte.
PUT	/api/v1/admin/users/{id}	Modifier un compte.
PATCH	/api/v1/admin/users/{id}/status	Activer ou désactiver un compte.
GET	/api/v1/admin/users/search	Rechercher un utilisateur.
GET	/api/v1/admin/users/stats	Statistiques des comptes.

Table 3.2 – Services web du Sprint 1

3.4 Les cas d'utilisation du Sprint 1

3.4.1 Cas d'utilisation ń S'authentifier ż

3.4.1.1 Description textuelle

Le tableau 3.3 présente la description textuelle du cas d'utilisation ń S'authentifier ż.

Acteurs	Commerçant, Collaborateur, Client, Administrateur Sellio
Objectif	Accéder à la plateforme de façon sécurisée et obtenir un jeton reflétant son rôle, ses permissions et sa boutique.
Pré-condition	L'utilisateur possède un compte.

Scénario principal	1. L'utilisateur saisit son e-mail et son mot de passe. 2. L'interface envoie la requête à la passerelle, qui la route vers le service d'identité. 3. Le système recherche le compte : parmi les clients de la boutique si la connexion vient d'une vitrine, parmi les commerçants, membres d'équipe et administrateurs sinon. Il compare le mot de passe à son empreinte BCrypt. 4. Le système vérifie que le compte n'est pas bloqué. 5. Le système génère un jeton d'accès JWT (24 heures) contenant l'identité, le rôle et la boutique, ainsi qu'un jeton de rafraîchissement (7 jours). 6. Le système retourne les jetons et le profil de l'utilisateur. 7. L'interface stocke les jetons et redirige l'utilisateur selon son rôle : console Sellio pour l'administrateur, back-office pour le commerçant, vitrine pour le client.
Scénario alternatif	**Identifiants incorrects :** à l'étape 3, le système renvoie le message générique ń Email ou mot de passe incorrect ż, sans préciser le champ erroné, pour ne pas faciliter l'énumération des comptes. **Compte bloqué :** à l'étape 4, le système répond 401 avec ń Votre compte a été bloqué. Contactez l'équipe Sellio. ż **Jeton expiré :** lors d'un appel ultérieur, l'intercepteur utilise le jeton de rafraîchissement pour obtenir un nouveau jeton sans redemander le mot de passe. **Même e-mail dans deux boutiques :** un client inscrit dans deux boutiques possède deux comptes distincts, avec deux mots de passe ; la boutique depuis laquelle il se connecte détermine le compte utilisé.

TABLE 3.3 – Description textuelle du cas d'utilisation ń S'authentifier ż

3.4.1.2 Diagramme de séquence système

La figure 3.2 présente le diagramme de séquence système de ce cas d'utilisation.

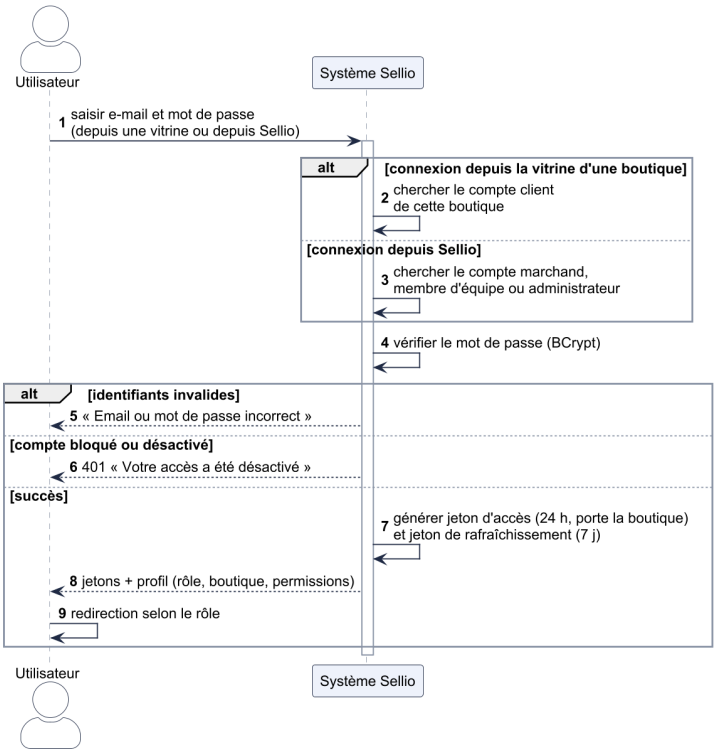


FIGURE 3.2 – Diagramme de séquence système du cas ń S'authentifier ż

3.4.1.3 Diagramme de séquence objet

La figure 3.3 présente le diagramme de séquence objet : la requête traverse la passerelle, le contrôleur `AuthController` et le service `AuthService`, qui interroge le dépôt `UserRepository` selon l'origine de la connexion (`findClientByEmail` pour une vitrine, `findAccountByEmail` pour Sellio) avant de générer les jetons.

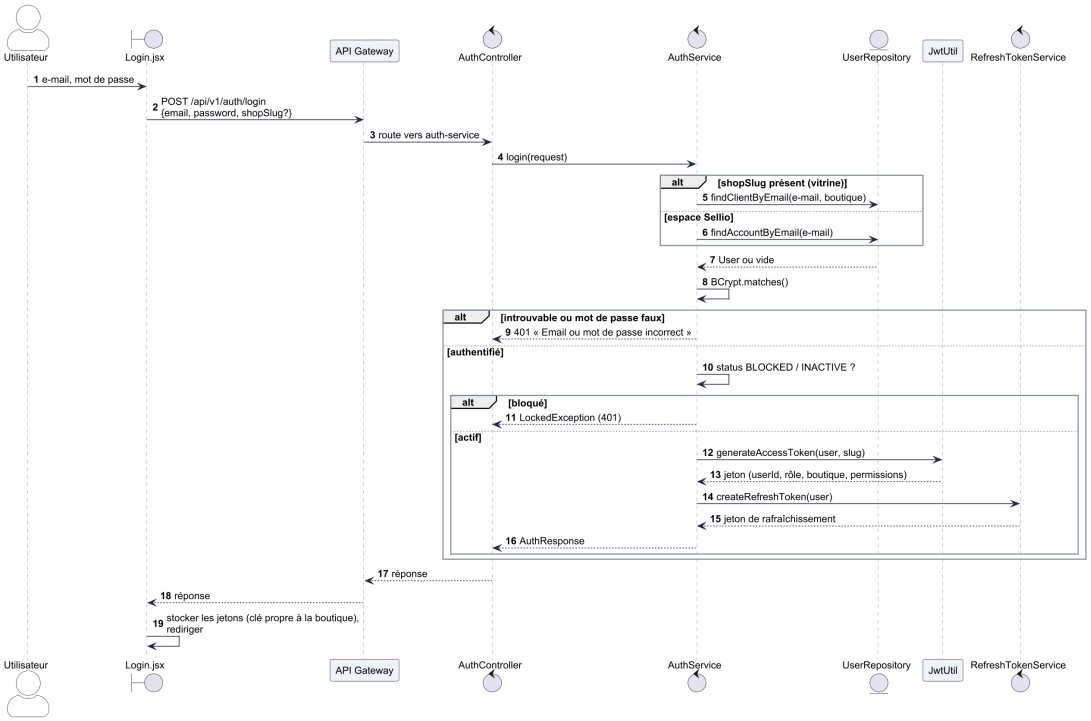


FIGURE 3.3 – Diagramme de séquence objet du cas ń S'authentifier ż

3.5 Réalisation du Sprint 1

Interface de connexion : la figure 3.4 présente la page de connexion de Sellio, affichée par défaut en thème sombre avec une bascule vers le thème clair.

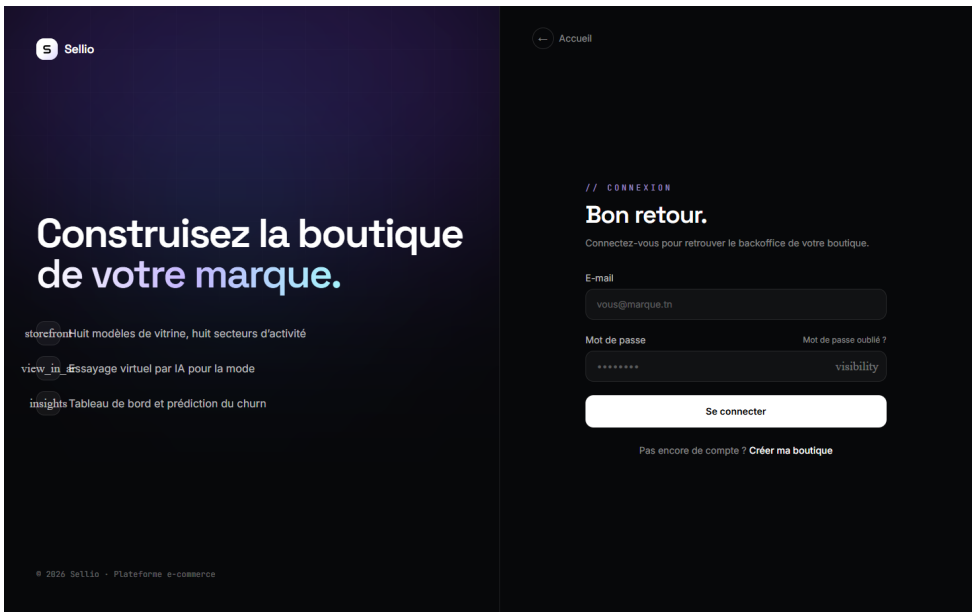


FIGURE 3.4 – Interface de connexion à Sellio

Interface de gestion des rôles et permissions : la figure 3.5 présente la matrice des permissions : le commerçant crée un rôle (par exemple ń Préparateur de commandes ż) et coche les modules auxquels il donne accès.

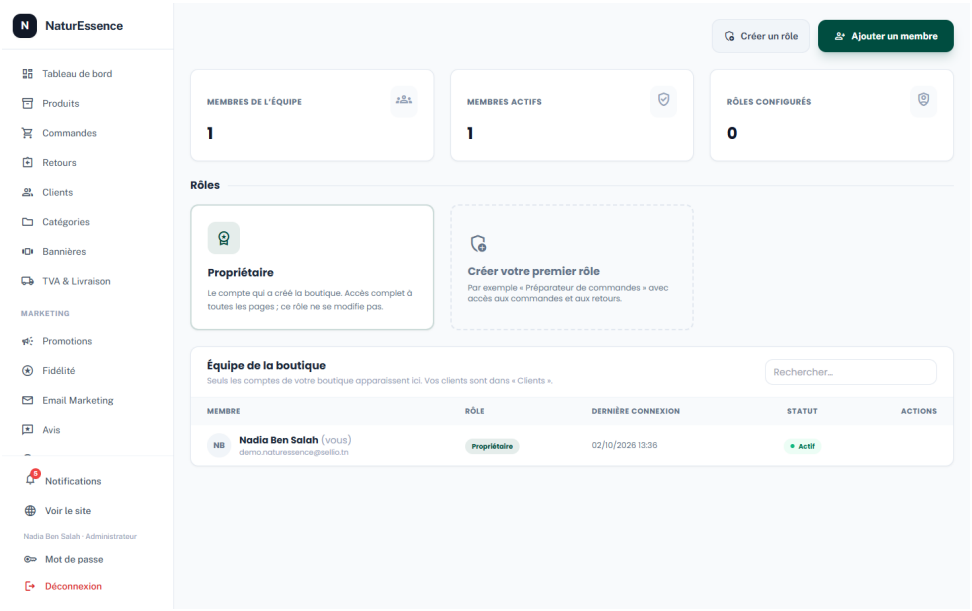


FIGURE 3.5 – Gestion des rôles et permissions

Interface de gestion des clients : la figure 3.6 présente la liste des clients de la boutique, avec recherche, filtres par statut et par segment, et accès à la fiche détaillée.

FIGURE 3.6 – Liste des clients de la boutique

3.6 Backlog du Sprint 2

Le tableau 3.4 présente le **backlog du Sprint 2**, consacré à la transformation en plateforme multi-boutiques.

User Story	Tâches	Estimation
En tant que visiteur, je souhaite découvrir Sellio sur une page d'accueil	Concevoir et développer la page d'accueil Sellio (thème sombre premium).	1
	Développer la bascule thème sombre / thème clair.	1
En tant que commerçant, je souhaite m'inscrire sur Sellio	Développer l'API d'inscription commerçant (`/register-merchant`) : l'inscription reste en attente, le mot de passe déjà haché.	1
	Confirmer l'adresse par un code à six chiffres envoyé par e-mail (Brevo), avec renvoi limité.	1
	Développer les interfaces d'inscription et de connexion Sellio.	1
En tant que commerçant, je souhaite créer ma boutique avec un assistant	Ajouter l'entité `Shop` et rattacher utilisateurs, produits, commandes et événements à une boutique.	1
	Filtrer toutes les requêtes par la boutique de l'utilisateur connecté (isolation multi-boutiques).	1
	Développer l'API de création de boutique (nom, *slug*, secteur parmi huit, modèle parmi huit, logo, couleurs).	1
	Développer l'assistant en cinq étapes avec retour en arrière.	1
	Tester la fonctionnalité.	1
En tant que commerçant, je souhaite ouvrir ma vitrine depuis le back-office	Développer le lien ń Voir le site ż vers la vitrine de la boutique.	1
En tant que commerçant, je souhaite enregistrer ma carte bancaire	Développer l'API Stripe de création d'un *SetupIntent*.	1
	Intégrer Stripe Elements et confirmer la carte côté navigateur.	1

Suite à la page suivante

User Story	Tâches	Estimation
	Stocker uniquement la marque, les 4 derniers chiffres, l'expiration et la référence `pm_`.	1
En tant que commerçant, je souhaite soumettre ma pièce d'identité et un selfie	Développer l'entité `MerchantVerification` et l'API de soumission avec contrôles (CIN à 8 chiffres, passeport, taille des images).	1
	Développer l'étape ń Identité ż (CIN recto-verso ou passeport, selfie).	1
	Développer la page de suivi du dossier et la nouvelle soumission après refus.	1
En tant qu'administrateur Sellio, je souhaite valider ou refuser un dossier	Développer les API d'approbation et de refus (motif obligatoire).	1
	Développer l'écran de revue du dossier dans la console.	1
	Tester la fonctionnalité.	1
En tant qu'administrateur Sellio, je souhaite consulter les boutiques et leurs propriétaires	Créer le compte administrateur Sellio au démarrage (*seed*).	1
	Développer les API de statistiques, de liste et de détail des boutiques.	1
	Développer la console avec pagination.	1
En tant qu'administrateur Sellio, je souhaite suspendre une boutique ou bloquer un commerçant	Développer les API de changement de statut d'une boutique et d'un utilisateur.	1
	Refuser la connexion d'un compte bloqué et afficher un message de boutique fermée sur la vitrine.	1
	Tester la fonctionnalité.	1

Suite à la page suivante

User Story	Tâches	Estimation
En tant qu'administrateur Sellio, je souhaite supprimer un commerçant et sa boutique	Développer la suppression en cascade : chaque microservice expose une route interne qui efface les données de la boutique dans sa propre base, appelée par le service d'identité.	1
	Développer la confirmation par saisie du nom de la boutique dans la console.	1
	Tester : aucune ligne restante, le compte du même client dans une autre boutique intact.	1
En tant que super administrateur, je souhaite gérer les administrateurs Sellio et les rôles des comptes	Créer le rôle `SELLIO_ADMIN` et l'API d'ajout d'un administrateur (mot de passe temporaire par e-mail).	1
	Développer le changement de rôle avec garde-fous : au moins un super administrateur, pas de modification de son propre rôle, pas de compte Sellio propriétaire d'une boutique.	1
	Développer l'onglet ń Comptes ż de la console.	1
En tant que système, je veux interdire à l'équipe Sellio l'accès aux données des boutiques	Refuser dans le filtre JWT, pour les rôles Sellio, les API des boutiques (clients, commandes, analyses) et retirer la liste des clients de la console.	1
	Total	**33**

Table 3.4 – Backlog du Sprint 2

3.7 Diagramme de cas d'utilisation du Sprint 2

La figure 3.7 présente le diagramme de cas d'utilisation du deuxième sprint.

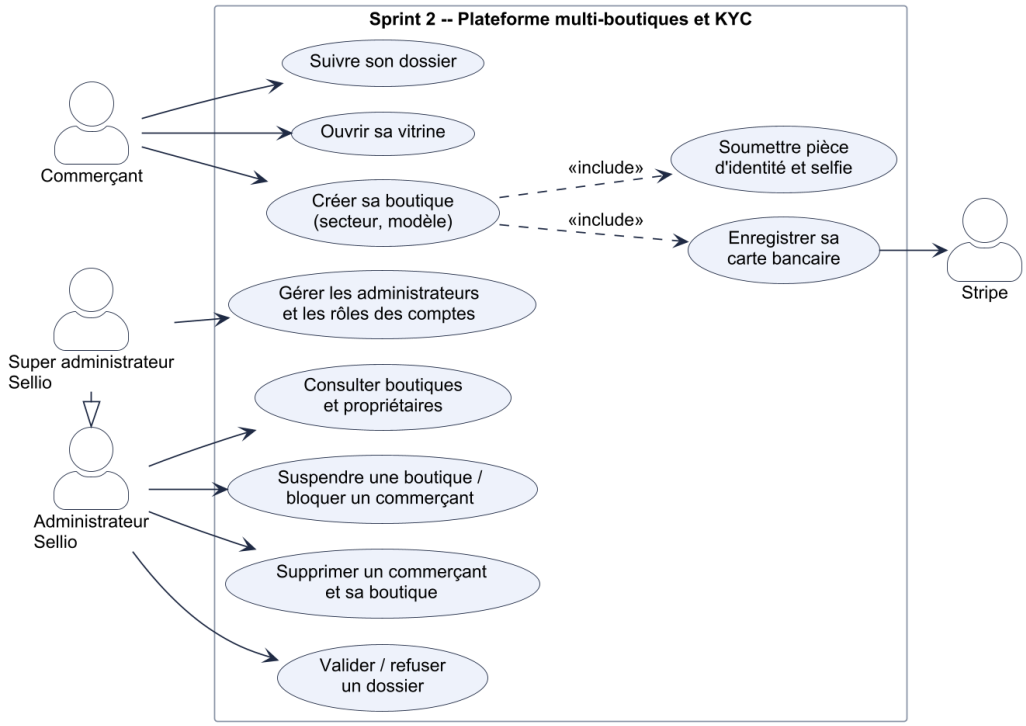


FIGURE 3.7 – Diagramme de cas d'utilisation du Sprint 2

3.8 Services web du Sprint 2

Le tableau 3.5 présente les services web du Sprint 2.

Méthode	**URL**	**Description**
Commerçant et boutique — /api/v1/auth		
POST	/api/v1/auth/register-merchant	Démarrer l'inscription d'un commerçant et envoyer le code.
POST	/api/v1/auth/register-merchant/verify	Vérifier le code et créer le compte commerçant.
POST	/api/v1/auth/register-merchant/resend	Renvoyer un code.
POST	/api/v1/auth/my-shop	Créer sa boutique avec son dossier de vérification.
GET	/api/v1/auth/my-shop	Consulter sa boutique.
PATCH	/api/v1/auth/my-shop	Modifier les informations de sa boutique.
GET	/api/v1/auth/my-shop/verification	Consulter l'état de son dossier.
PUT	/api/v1/auth/my-shop/verification	Soumettre à nouveau un dossier refusé.
GET	/api/v1/public/shops/{slug}	Informations publiques d'une boutique (statut, gabarit, thème).

Méthode	URL	Description
POST	/api/v1/public/stripe/setup-intent	Créer un *SetupIntent* Stripe pour enregistrer la carte.
Console plateforme — /api/v1/admin/platform		
GET	/api/v1/admin/platform/stats	Statistiques globales (boutiques, commerçants, dossiers en attente).
GET	/api/v1/admin/platform/shops	Lister les boutiques et leurs propriétaires (paginé).
GET	/api/v1/admin/platform/shops/{id}	Détail d'une boutique.
PATCH	/api/v1/admin/platform/shops/{id}/status	Suspendre ou réactiver une boutique.
PATCH	/api/v1/admin/platform/users/{id}/status	Bloquer ou débloquer un commerçant.
GET	/api/v1/admin/platform/users	Lister les commerçants et les administrateurs Sellio (jamais les clients).
POST	/api/v1/admin/platform/users	Ajouter un administrateur Sellio (super administrateur).
PATCH	/api/v1/admin/platform/users/{id}/role	Changer le rôle d'un compte (super administrateur).
DELETE	/api/v1/admin/platform/users/{id}	Supprimer un commerçant avec sa boutique, ou un compte Sellio.
GET	/api/v1/admin/platform/verifications	Lister les dossiers de vérification.
GET	/api/v1/admin/platform/verifications/{id}	Consulter un dossier (pièces et selfie).
POST	/api/v1/admin/platform/verifications/{id}/approve	Valider un dossier : la boutique passe à `ACTIVE`.
POST	/api/v1/admin/platform/verifications/{id}/reject	Refuser un dossier avec un motif : la boutique passe à `REJECTED`.

Table 3.5 – Services web du Sprint 2

3.9 Les cas d'utilisation du Sprint 2

3.9.1 Cas d'utilisation ń Créer une boutique et faire vérifier son identité ż

3.9.1.1 Description textuelle

Le tableau 3.6 présente la description textuelle de ce cas d'utilisation.

Acteurs	Commerçant, Administrateur Sellio, Stripe
Objectif	Ouvrir une boutique qui ne pourra vendre qu'après vérification de l'identité et du moyen de paiement de son propriétaire.

Pré-condition	Le commerçant est inscrit et connecté ; il ne possède pas encore de boutique.
Scénario principal	1. Le commerçant saisit le nom de sa boutique ; le système propose un identifiant (*slug*). 2. Il choisit le secteur parmi huit (mode, beauté, sport, high-tech, maison, épicerie, bijoux, enfants) et l'un des huit modèles de vitrine, recommandés selon le secteur, puis son logo et sa palette. 3. À l'étape ń Carte ż, le système crée un *SetupIntent* Stripe ; le commerçant saisit sa carte dans le champ sécurisé Stripe, qui la confirme sans que le numéro ne transite par Sellio. 4. À l'étape ń Identité ż, il choisit CIN ou passeport, photographie sa pièce et prend un selfie. 5. Le système contrôle le dossier (format du numéro, validité de la carte, taille des images), crée la boutique au statut `PENDING` et le dossier au statut en attente. 6. Le back-office affiche ń Dossier en cours de vérification ż ; la vitrine n'est pas encore ouverte au public. 7. L'administrateur Sellio consulte le dossier, compare le selfie à la pièce d'identité et le valide. 8. La boutique passe au statut `ACTIVE` ; le commerçant accède à tout le back-office.
Scénario alternatif	**Carte refusée par Stripe :** à l'étape 3, le message de Stripe est affiché et le commerçant peut saisir une autre carte. **Dossier incomplet :** à l'étape 5, le système indique la pièce manquante (par exemple ń Le numéro de CIN doit contenir 8 chiffres ż). **Dossier refusé :** à l'étape 7, l'administrateur saisit obligatoirement un motif ; la boutique passe à `REJECTED`, le motif est affiché au commerçant qui peut soumettre un nouveau dossier.

TABLE 3.6 – Description textuelle du cas ń Créer une boutique et faire vérifier son identité ż

3.9.1.2 Diagramme de séquence système

La figure 3.8 présente le diagramme de séquence système de ce cas.

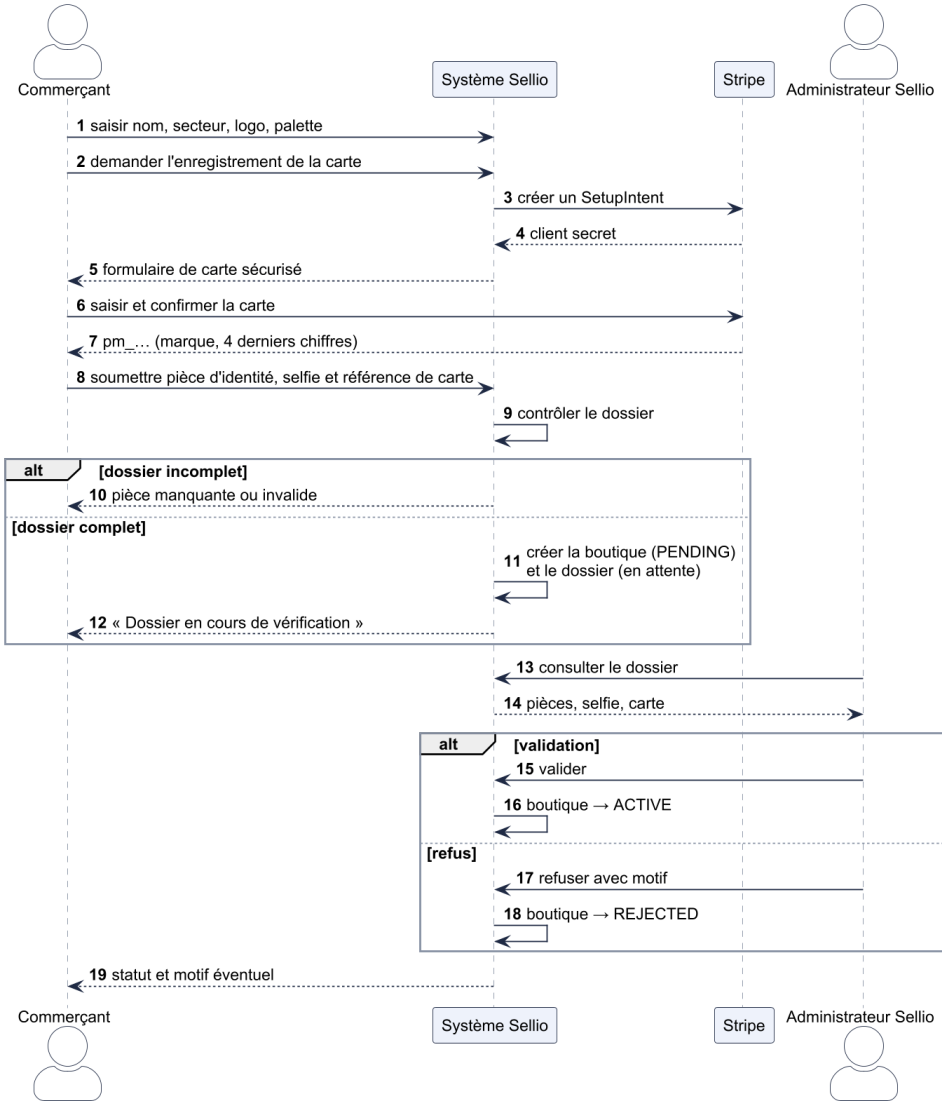


FIGURE 3.8 – Diagramme de séquence système du cas ń Créer une boutique et faire vérifier son identité ż

3.9.1.3 Diagramme de séquence objet

La figure 3.9 présente le diagramme de séquence objet de ce cas.

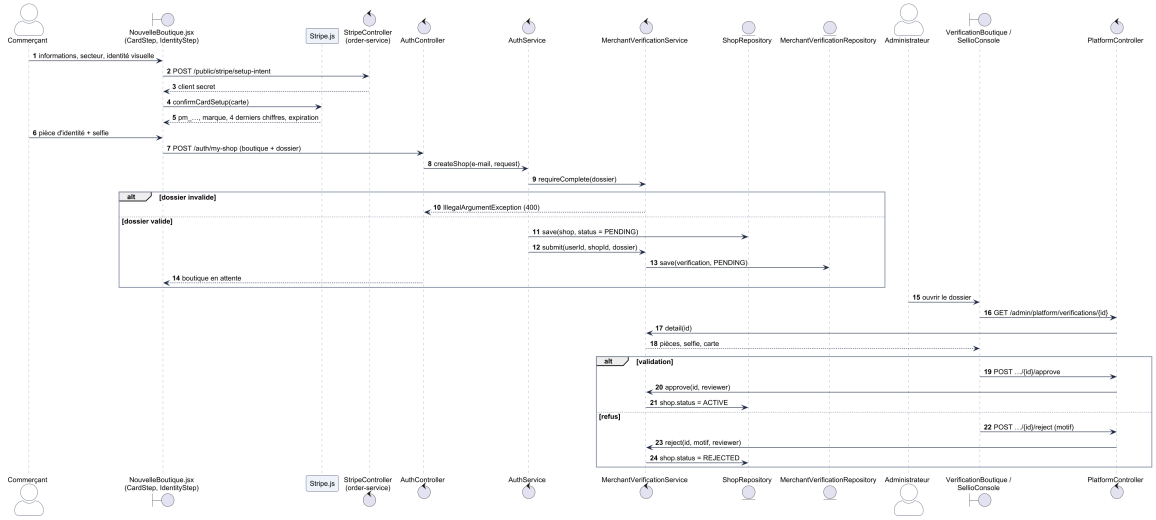


FIGURE 3.9 – Diagramme de séquence objet du cas ń Créer une boutique et faire vérifier son identité ż

3.9.2 Cas d'utilisation ń S'inscrire comme commerçant ż

3.9.2.1 Description textuelle

Le tableau 3.7 présente la description textuelle de ce cas d'utilisation.

Acteurs	Commerçant, Brevo
Objectif	Créer un compte commerçant dont l'adresse e-mail a été vérifiée.
Pré-condition	Aucun compte Sellio n'utilise cette adresse.
Scénario principal	1. Le commerçant saisit son nom, son e-mail, son téléphone et un mot de passe d'au moins huit caractères. 2. Le système conserve l'inscription en attente, avec le mot de passe déjà haché, et génère un code à six chiffres dont seule l'empreinte est stockée. 3. Le code est envoyé par e-mail via Brevo. 4. Le commerçant saisit le code ; le système crée le compte et l'authentifie. 5. Le commerçant est dirigé vers l'assistant de création de boutique.
Scénario alternatif	**Code incorrect :** le système indique le nombre d'essais restants ; après trois erreurs, un nouveau code doit être demandé. **Code expiré :** au-delà de dix minutes, le commerçant demande un nouveau code (un envoi par minute au plus).

TABLE 3.7 – Description textuelle du cas ń S'inscrire comme commerçant ż

3.9.2.2 Diagramme de séquence système

La figure 3.10 présente le diagramme de séquence système de ce cas.

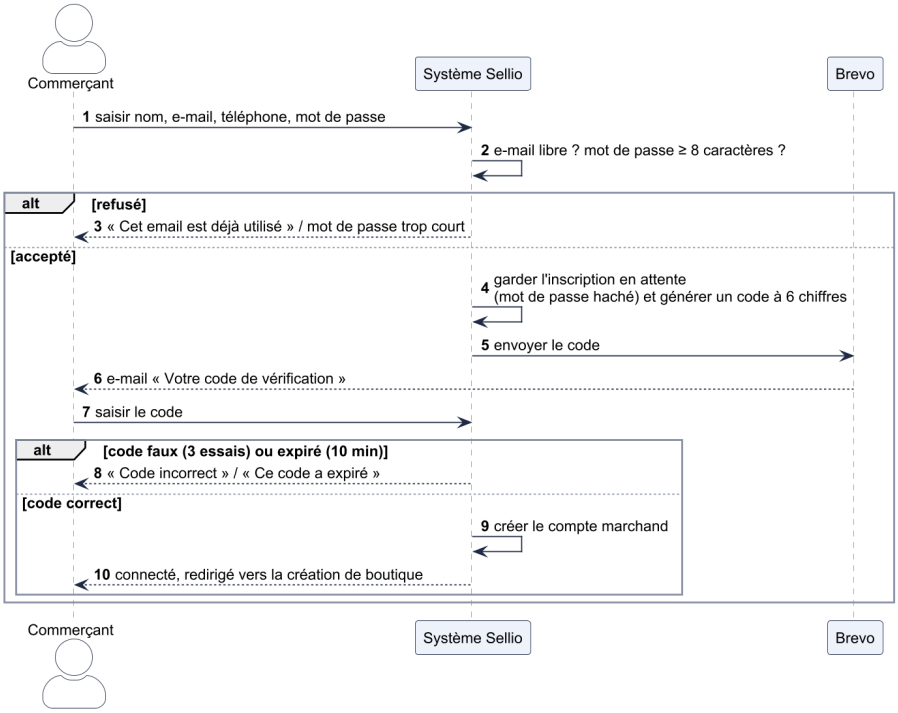


FIGURE 3.10 – Diagramme de séquence système du cas ń S'inscrire comme commerçant ż

3.9.3 Cas d'utilisation ń Supprimer un commerçant et sa boutique ż

3.9.3.1 Description textuelle

Le tableau 3.8 présente la description textuelle de ce cas d'utilisation.

Acteurs	Administrateur Sellio, Super administrateur Sellio
Objectif	Retirer définitivement un commerçant de la plateforme, avec sa boutique et toutes ses données.
Pré-condition	L'administrateur est connecté à la console.

Scénario principal	1. L'administrateur ouvre la fiche de la boutique (ou l'onglet ń Comptes ż) et choisit ń Supprimer le marchand ż. 2. Le système affiche ce qui sera supprimé et demande de saisir le nom de la boutique. 3. L'administrateur saisit le nom et confirme. 4. Le service d'identité demande à chaque microservice, par sa route interne `DELETE /internal/shops/{id}`, d'effacer les données de la boutique dans sa base : analyse, marketing, commandes, catalogue. Chaque service supprime d'abord les lignes dépendantes (lignes de commande, avis, variantes…), puis les lignes de la boutique. Le service d'identité supprime enfin, dans sa propre base, les clients, l'équipe, les rôles, la boutique et le compte du commerçant. 5. Le système affiche ń Marchand supprimé avec sa boutique et ses N clients ż.
Scénario alternatif	**Compte Sellio :** la suppression d'un administrateur n'est permise qu'au super administrateur, et Sellio garde toujours au moins un super administrateur. **Son propre compte :** le système refuse la suppression.

TABLE 3.8 – Description textuelle du cas ń Supprimer un commerçant et sa boutique ż

Chaque microservice étant le seul à connaître sa base, c'est à lui d'effacer les données de la boutique : le service d'identité n'a besoin de connaître aucune table des autres services. Les suppressions sont idempotentes ; si un service est indisponible, l'opération s'arrête avant d'effacer le compte du commerçant et peut être relancée.

3.9.3.2 Diagramme de séquence système

La figure 3.11 présente le diagramme de séquence système de ce cas.

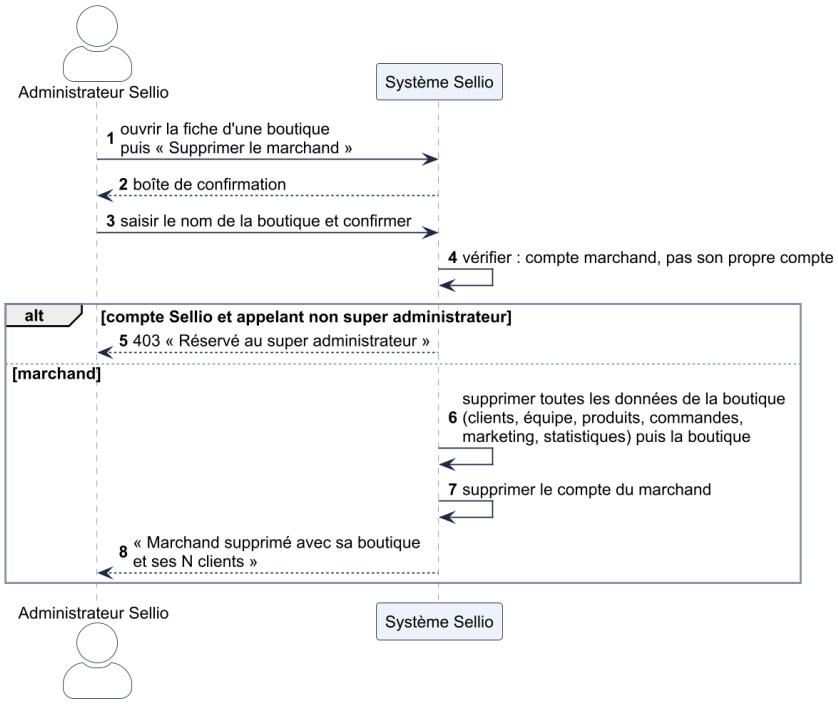


FIGURE 3.11 – Diagramme de séquence système du cas ń Supprimer un commerçant et sa boutique ż

3.9.3.3 Diagramme de séquence objet

La figure 3.12 présente le diagramme de séquence objet de ce cas.

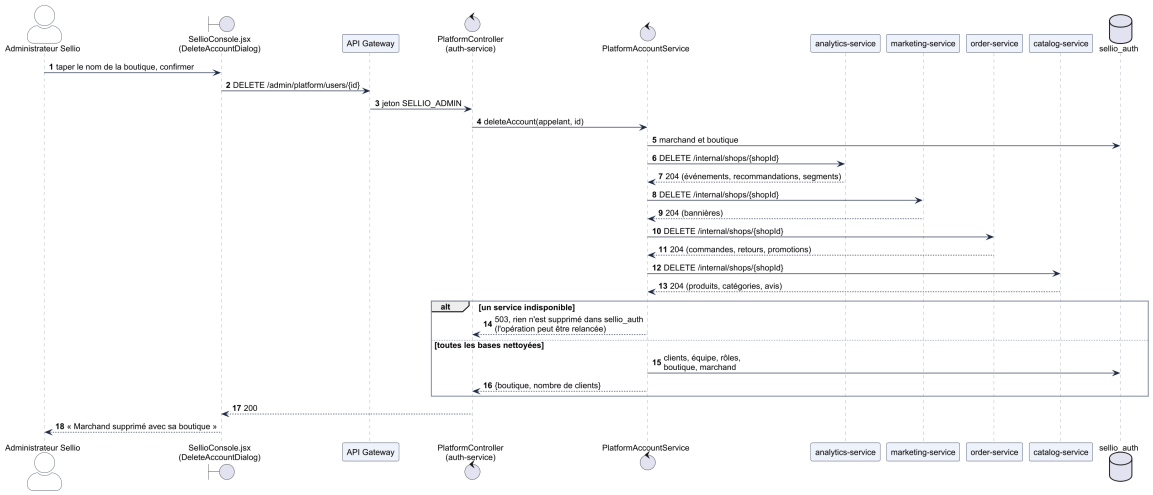


FIGURE 3.12 – Diagramme de séquence objet du cas ń Supprimer un commerçant et sa boutique ż

3.10 Réalisation du Sprint 2

Page d'accueil Sellio : la figure 3.13 présente la page d'accueil de la plateforme, qui présente l'offre aux commerçants et donne accès à l'inscription et à la connexion.

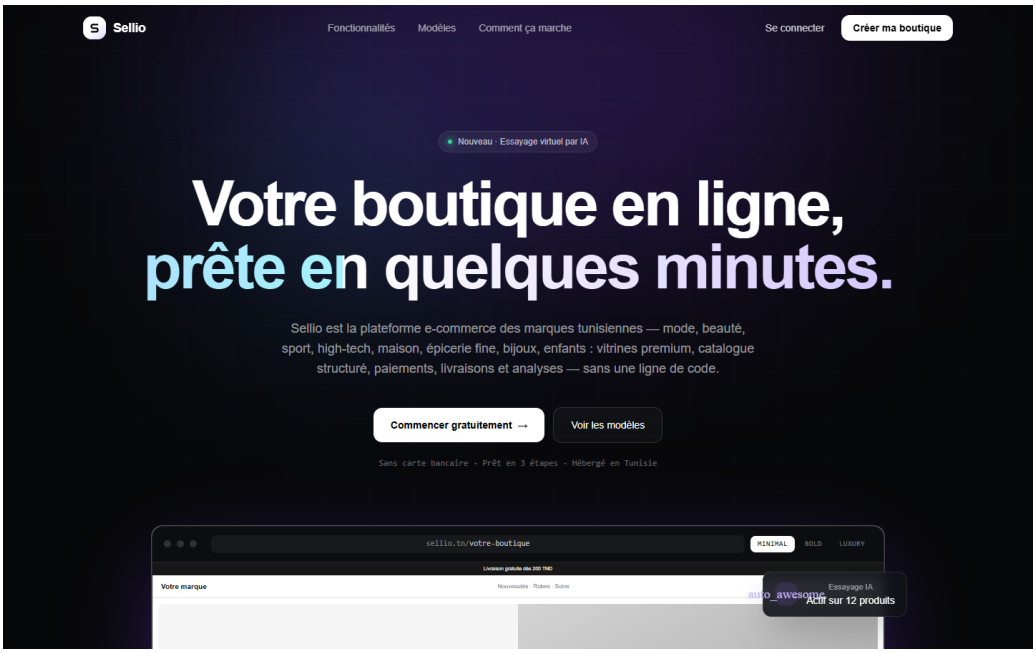


FIGURE 3.13 – Page d'accueil de Sellio

Assistant de création de boutique : la figure 3.14 présente la première étape de l'assistant : nom de la boutique et choix du secteur parmi huit, avec un aperçu de la vitrine. Les étapes suivantes enregistrent la carte bancaire et la pièce d'identité.

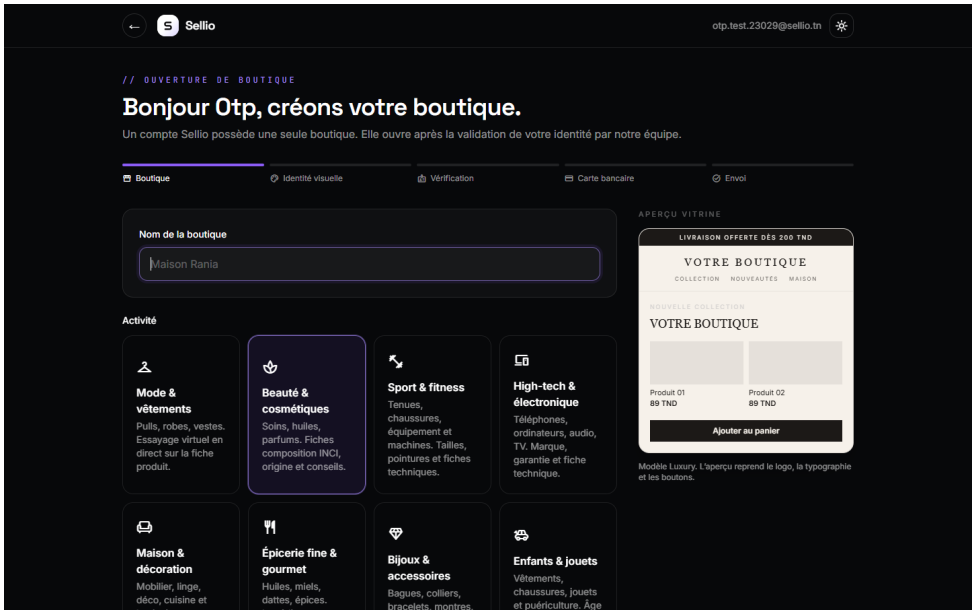


FIGURE 3.14 – Assistant de création de boutique : informations et secteur

Console d'administration Sellio : la figure 3.15 présente la console : indicateurs globaux, liste paginée des boutiques avec leur propriétaire et leur statut, et actions de suspension ou de blocage. La figure 3.16 présente la revue d'un dossier de vérification.

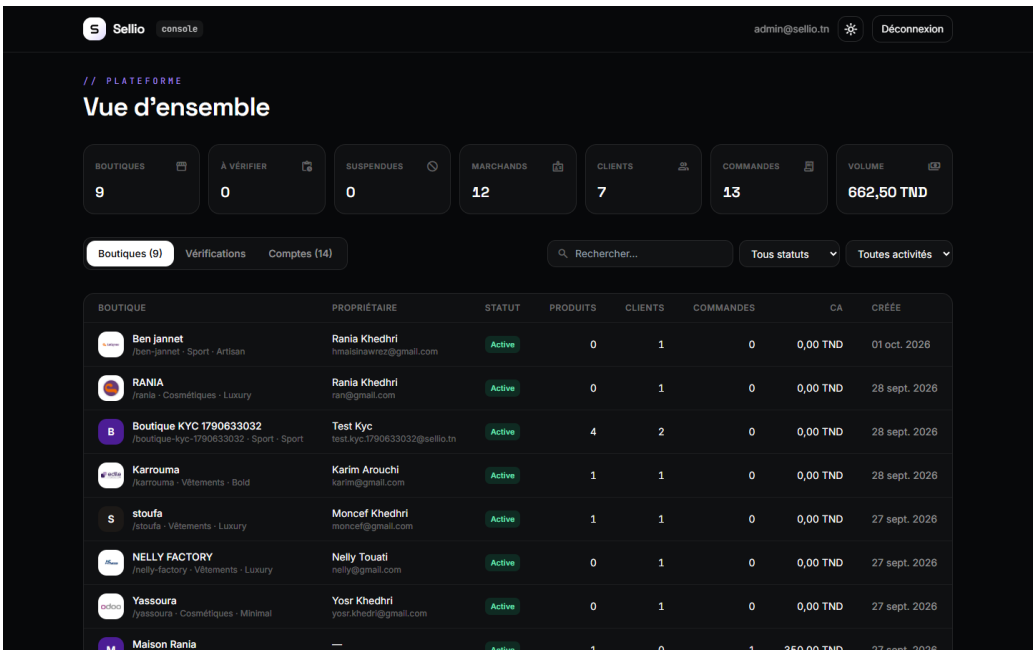


FIGURE 3.15 – Console d'administration Sellio

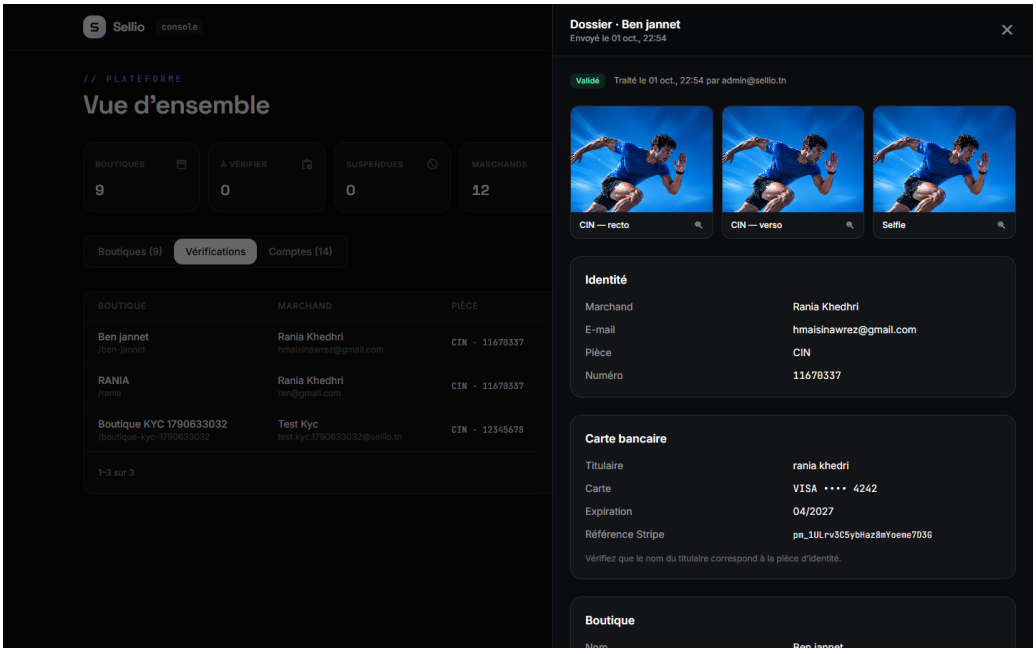


FIGURE 3.16 – Revue d'un dossier de vérification par l'administrateur

Gestion des comptes de la plateforme : la figure 3.17 présente l'onglet ń Comptes ż : seuls les commerçants et les administrateurs Sellio y figurent. Le super administrateur y ajoute un administrateur, change un rôle ou supprime un compte ; un administrateur Sellio peut bloquer ou supprimer un commerçant.

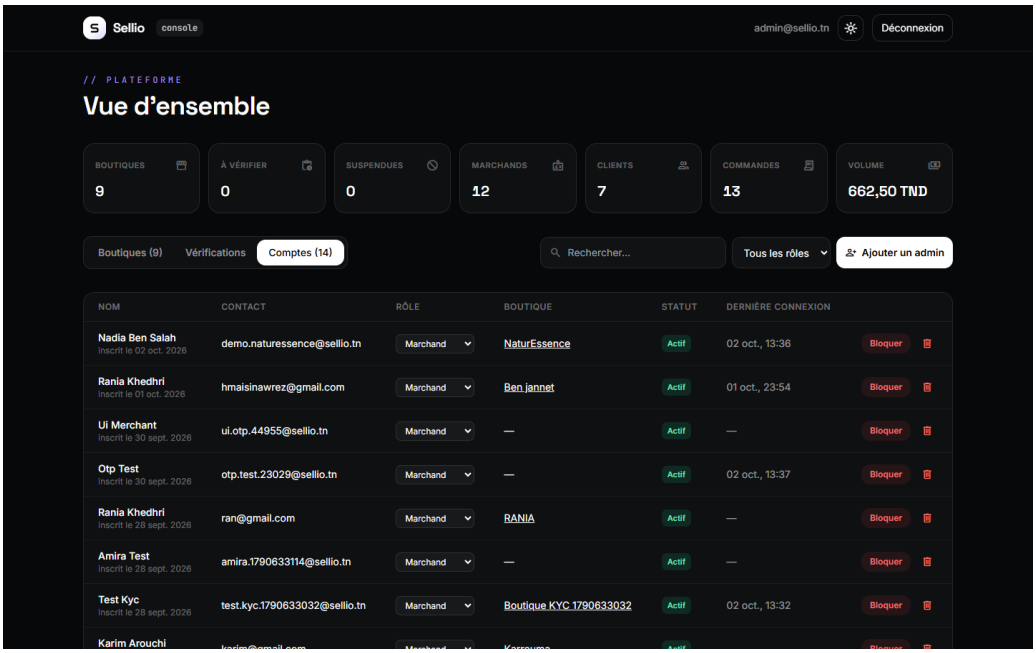


FIGURE 3.17 – Comptes de la plateforme dans la console Sellio

Conclusion

Au terme de cette release, Sellio dispose d'une architecture microservices sécurisée et d'un véritable modèle multi-boutiques : un commerçant peut s'inscrire, créer sa boutique et la faire vérifier ; chaque client appartient à une boutique ; l'équipe Sellio, organisée en administrateurs et super administrateurs, garde le contrôle sur les boutiques ouvertes sans jamais accéder à leurs clients. La release suivante porte sur le catalogue et la vitrine.

Chapitre 4

Release 2 : Catalogue produit et vitrine personnalisable

Introduction

Ce chapitre présente la deuxième release, qui donne à chaque boutique son contenu et son apparence. Le Sprint 3 met en place un catalogue capable de servir huit secteurs d'activité : la fiche d'un produit s'adapte au secteur de la boutique et au type du produit (une bague n'a pas les mêmes caractéristiques qu'une montre, une chaussure de trail pas les mêmes qu'un tapis de course). Le Sprint 4 livre huit modèles de vitrine et deux éditeurs, ń Apparence ż et ń Page d'accueil ż, qui permettent au commerçant de tout personnaliser, logo, couleurs, polices, sections et textes, avec un aperçu en direct de sa vraie vitrine.

4.1 Backlog du Sprint 3

Le tableau 4.1 présente le **backlog du Sprint 3**.

User Story	Tâches	Estimation
En tant que commerçant, je souhaite gérer mes produits	Développer les entités `Product` et `ProductVariant` (prix, stock, statut, images).	1
	Développer l'API CRUD des produits, avec archivage, désactivation et statistiques.	1
	Développer l'API de téléversement d'images.	1
	Développer les interfaces ń Produits ż, ń Ajouter un produit ż et ń Modifier un produit ż.	1
	Tester la fonctionnalité.	1
En tant que commerçant, je souhaite une fiche produit adaptée à mon secteur	Définir les huit secteurs côté serveur (`ShopCatalog`) et refuser tout secteur inconnu.	1

Suite à la page suivante

User Story	Tâches	Estimation
	Définir, pour chaque secteur générique, ses types de produits avec leurs tailles et leurs champs (registre `sectors.js`).	1
	Stocker les caractéristiques dans un objet JSON (`Product.attributes`) validé côté serveur (objet plat, 40 clés au plus).	1
	Lire le secteur depuis le serveur avant d'afficher la fiche, jamais depuis le profil gardé par le navigateur.	1
	Afficher les caractéristiques sur la vitrine : puces sous le titre, tableau technique, onglets pour les textes longs.	1
En tant que commerçant, je souhaite choisir le type de chaque produit	Proposer les types du secteur (bague, collier, montre, sac… ; vêtement, chaussures, équipement, machine, nutrition…).	1
	Adapter au type ce que le client choisit (tour de doigt, pointure, stockage, format) ou ne rien demander pour un modèle unique.	1
	Retirer les champs de l'ancien type quand le commerçant en change ; reconnaître le type des produits existants à partir de leurs tailles.	1
En tant que commerçant, je souhaite gérer mes catégories	Développer l'API des catégories hiérarchiques (parent, enfants, ordre d'affichage).	1
	Développer les interfaces de gestion et de réordonnancement des catégories.	1
En tant que commerçant, je souhaite choisir les valeurs dans des listes, et en ajouter	Définir les listes de valeurs : tailles par groupe, 28 couleurs nommées, tissus, et plus de soixante listes propres aux secteurs (matières, pierres, mouvements, connectiques, allergènes…).	1
	Développer le sélecteur de couleurs avec ajout d'une couleur libre (sélecteur RVB et nom).	1
	Stocker les valeurs ajoutées par la boutique (`customOptions`) et les fusionner avec les listes par défaut.	1
	Tester la fonctionnalité dans chaque secteur.	1
	Total	**19**

Table 4.1 – Backlog du Sprint 3

4.2 Diagramme de cas d'utilisation du Sprint 3

La figure 4.1 présente le diagramme de cas d'utilisation du troisième sprint.

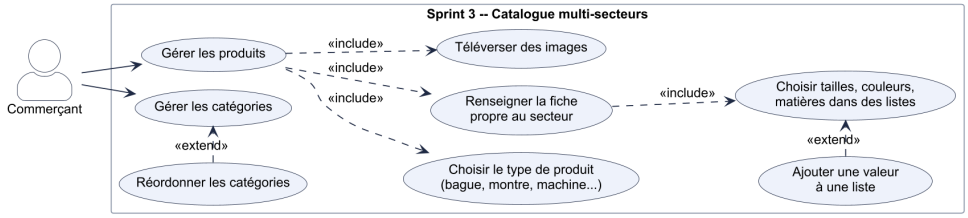


FIGURE 4.1 – Diagramme de cas d'utilisation du Sprint 3

4.3 Services web du Sprint 3

Le tableau 4.2 présente les services web du Sprint 3, exposés par le microservice `catalog-service`.

Méthode	URL	Description
Produits (administration) — /api/v1/admin/products		
GET	/api/v1/admin/products	Lister les produits de la boutique.
GET	/api/v1/admin/products/stats	Statistiques du catalogue.
POST	/api/v1/admin/products	Créer un produit, ses variantes et ses caractéristiques (`attributes`).
PUT	/api/v1/admin/products/{id}	Modifier un produit.
PATCH	/api/v1/admin/products/{id}/archive	Archiver un produit.
PATCH	/api/v1/admin/products/{id}/deactivate	Désactiver un produit.
DELETE	/api/v1/admin/products/{id}	Supprimer un produit.
POST	/api/v1/admin/upload	Téléverser une image.
Catégories (administration)		
GET, POST	/api/v1/admin/categories	Lister, créer une catégorie.
PUT, DELETE	/api/v1/admin/categories/{id}	Modifier, supprimer une catégorie.
PUT	/api/v1/admin/categories/reorder	Réordonner les catégories.
Catalogue public (vitrine)		
GET	/api/v1/public/products	Lister et filtrer les produits publiés.
GET	/api/v1/public/products/{slug}	Fiche d'un produit.
GET	/api/v1/public/products/category/{categoryId}	Produits d'une catégorie.
GET	/api/v1/public/products/by-ids	Produits à partir d'une liste d'identifiants (recommandations, sélections).
GET	/api/v1/public/categories/menu	Arborescence du menu de la vitrine.
GET	/api/v1/public/categories/homepage	Catégories mises en avant sur l'accueil.
GET	/api/v1/public/categories/footer	Catégories du pied de page.

Méthode	URL	Description

Table 4.2 – Services web du Sprint 3

4.4 Les cas d'utilisation du Sprint 3

4.4.1 Cas d'utilisation ń Ajouter un produit ż

4.4.1.1 Description textuelle

Le tableau 4.3 présente la description textuelle de ce cas.

Acteurs	Commerçant, Collaborateur disposant de la permission ń Produits ż
Objectif	Publier un produit dont les caractéristiques correspondent à son secteur et à son type, exploitables par la vitrine et par le moteur de recommandation.
Pré-condition	L'utilisateur est connecté et sa boutique est active.
Scénario principal	1. L'utilisateur ouvre ń Ajouter un produit ż ; le système lit le secteur de la boutique sur le serveur et affiche la fiche correspondante. 2. Il saisit le nom, la description, le prix, la catégorie et téléverse les images. 3. Dans une boutique de bijoux, de sport, de high-tech, de maison, d'épicerie ou pour enfants, il choisit d'abord le type du produit ; les tailles proposées et les champs de la fiche s'adaptent (pour une bague : tour de doigt, matière, pierre, largeur ; pour une montre : mouvement, boîtier, étanchéité, sans taille à choisir). 4. Il choisit les valeurs dans les listes (tailles, couleurs, matières…) et définit le stock de chaque variante. 5. L'utilisateur enregistre ; le système valide les données et les caractéristiques, puis crée le produit rattaché à sa boutique. 6. Le produit apparaît dans la liste et, s'il est publié, sur la vitrine avec ses caractéristiques.
Scénario alternatif	**Valeur absente d'une liste :** à l'étape 4, l'utilisateur ajoute une valeur (une couleur libre avec son code RVB, une nouvelle matière) ; elle est enregistrée dans les options de la boutique et proposée pour les produits suivants. **Changement de type :** les tailles et les champs du type précédent sont retirés, pour qu'aucune caractéristique incohérente ne reste sur le produit. **Données invalides :** à l'étape 5, le système signale les champs erronés (prix négatif, nom manquant, caractéristiques mal formées) sans perdre la saisie.

Table 4.3 – Description textuelle du cas ń Ajouter un produit ż

4.4.1.2 Diagramme de séquence système

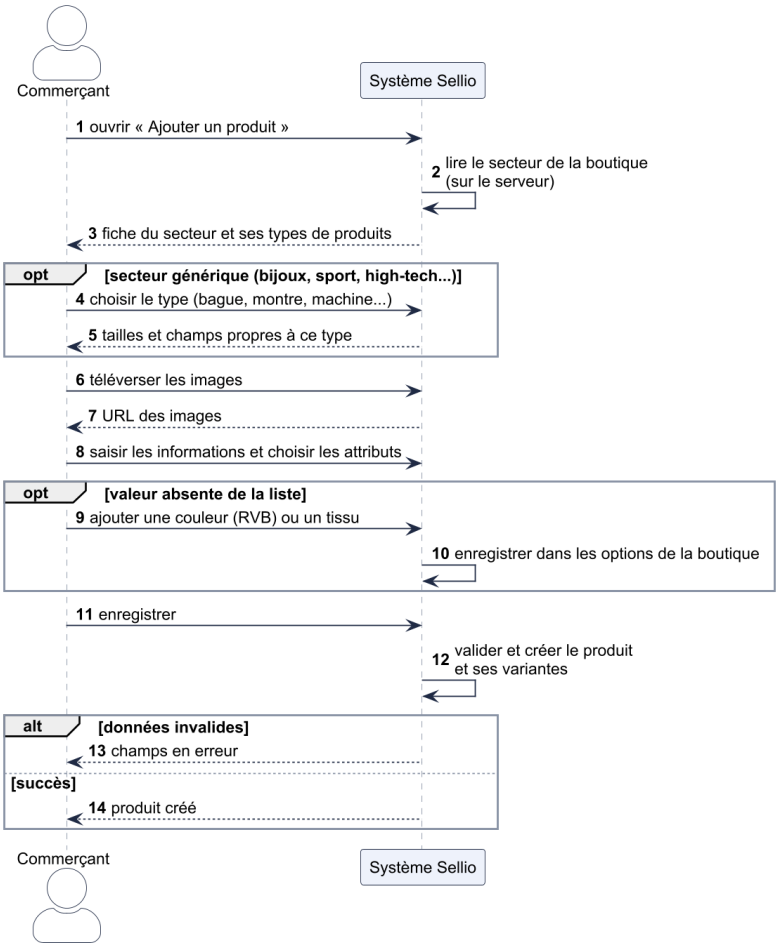


FIGURE 4.2 – Diagramme de séquence système du cas ń Ajouter un produit ż

4.4.1.3 Diagramme de séquence objet

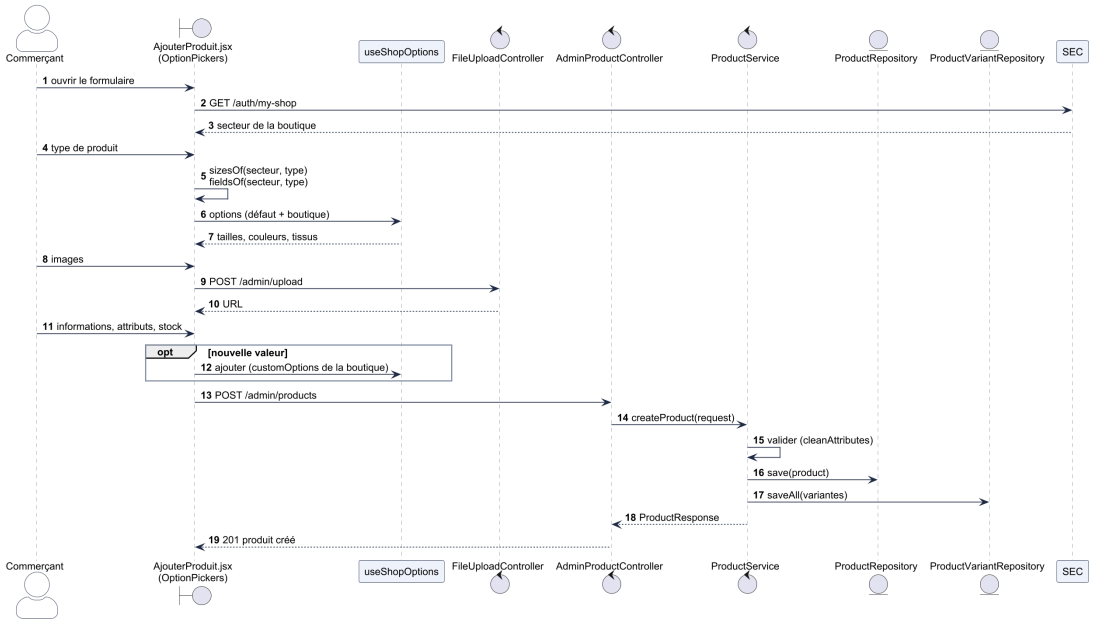


FIGURE 4.3 – Diagramme de séquence objet du cas ń Ajouter un produit ż

4.5 Réalisation du Sprint 3

Liste des produits : la figure 4.4 présente la liste des produits avec leurs statistiques, filtres et actions.

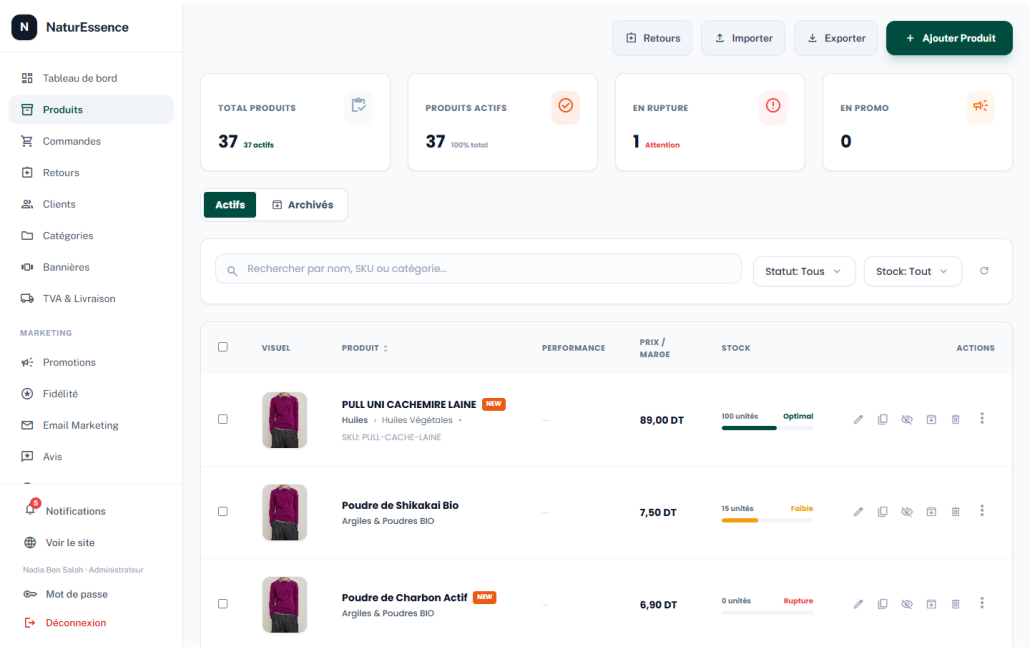


FIGURE 4.4 – Liste des produits de la boutique

Fiche selon le type de produit : la figure 4.5 présente la fiche d'un produit dans une boutique de bijoux. Le commerçant choisit le type ń Bague ż : le client devra choisir un tour de doigt, et les champs proposés sont ceux d'une bague. Le tableau 4.4 résume les types disponibles.

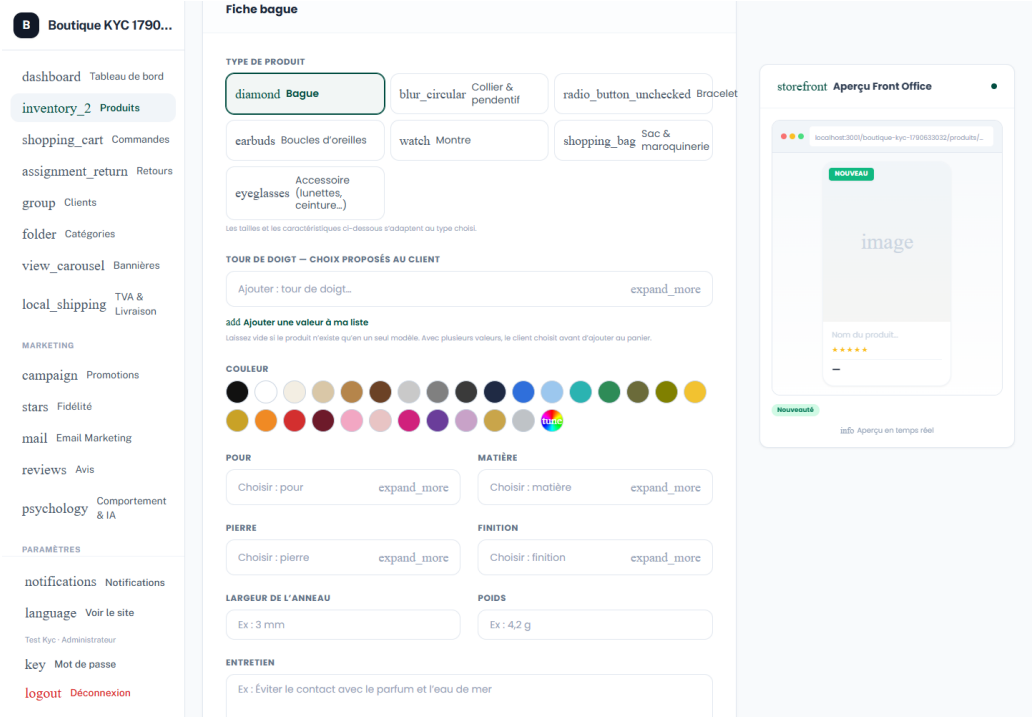


FIGURE 4.5 – Fiche d'un produit de type ń Bague ż dans une boutique de bijoux

Secteur	**Types de produits (ce que choisit le client)**
Bijoux	Bague (tour de doigt), collier (longueur), bracelet (tour de poignet), boucles d'oreilles, montre, sac, accessoire (taille)
Sport	Vêtement (taille), chaussures (pointure), équipement (taille ou poids), machine de fitness, nutrition sportive (format)
High-tech	Smartphone (stockage), ordinateur (configuration), audio, TV (taille d'écran), électroménager, accessoire
Maison	Mobilier, linge (dimensions), décoration (format), cuisine (format), luminaire
Épicerie	Huiles, douceurs, épices, boissons (format), coffrets
Enfants	Vêtement (taille), chaussures (pointure), jouet, puériculture, livres et loisirs créatifs
Mode, beauté	Fiches dédiées : tailles, tissu, coupe, couleur ; contenance, composition INCI, origine, certifications

TABLE 4.4 – Types de produits par secteur

Sélecteurs d'attributs : la figure 4.6 présente la fiche d'un produit cosmétique. Remplacer la saisie libre par des listes évite les doublons (ń bleu marine ż, ń Marine ż, ń navy ż) qui fausseraient les filtres et les recommandations.

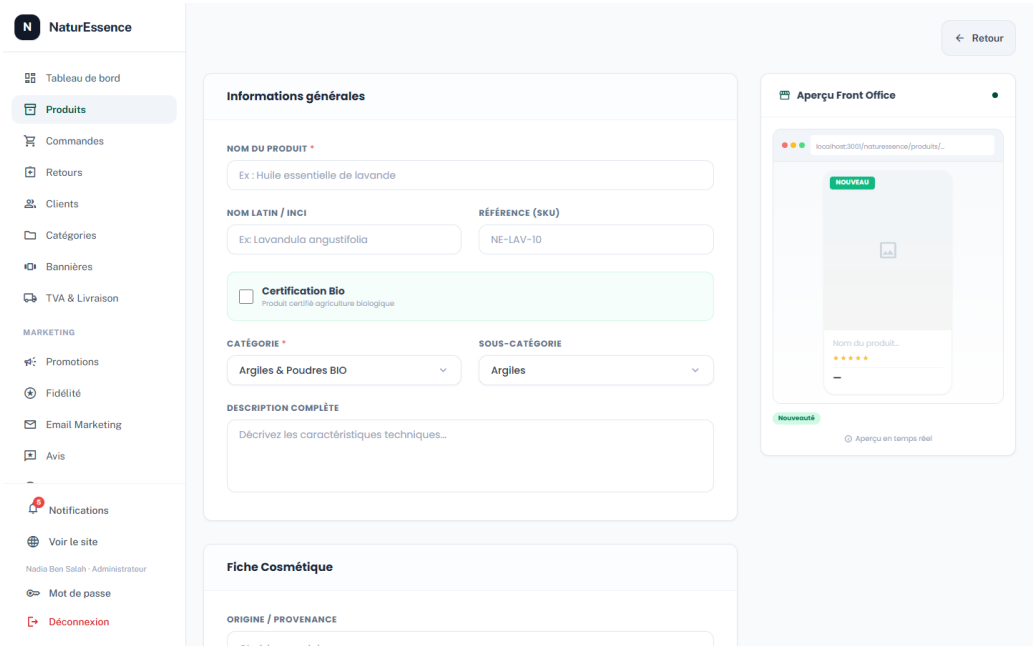


FIGURE 4.6 – Fiche d'un produit cosmétique

Catégories : la figure 4.7 présente la gestion des catégories hiérarchiques.

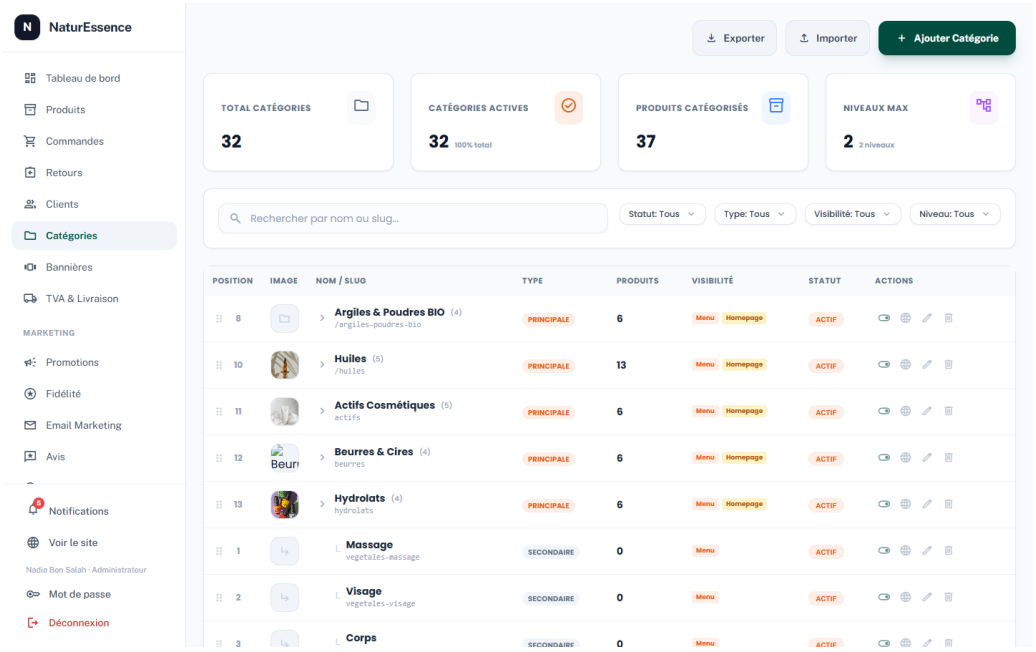


FIGURE 4.7 – Gestion des catégories

4.6 Backlog du Sprint 4

Le tableau 4.5 présente le **backlog du Sprint 4**, consacré à la vitrine et à sa personnalisation.

User Story	Tâches	Estimation
En tant que commerçant, je souhaite choisir l'un des huit modèles de vitrine	Concevoir l'architecture des modèles : en-tête, accueil et pied de page par modèle, briques partagées (navigation, données d'accueil, tiroir mobile, recherche).	1
	Développer les modèles Minimal, Bold et Luxury.	1
	Développer les modèles Sport, Tech, Artisan, Pop et Éditorial, avec leurs polices et leurs palettes.	1
	Écrire les textes par défaut de chaque secteur, en trois tons (doux, percutant, haut de gamme) selon le modèle.	1
	Recommander les modèles adaptés au secteur et permettre la prévisualisation (`?preview=`).	1
En tant que visiteur, je souhaite parcourir la vitrine et consulter un produit	Développer les menus alimentés par l'arborescence des catégories et la page de catégorie avec filtres.	1
	Développer la fiche produit (galerie, variantes, caractéristiques du secteur, avis).	1
	Afficher le logo de la boutique comme icône de l'onglet du navigateur, ou son initiale sur sa couleur.	1
En tant que commerçant, je souhaite personnaliser l'apparence de ma boutique	Stocker la personnalisation de chaque boutique (`Shop.settings`, objet JSON) et la valider côté serveur : objet, 100 Ko au plus, liens en `https` uniquement.	1
	Développer la page ń Apparence ż : logo et taille du logo, nom, coordonnées et réseaux sociaux, polices, barre d'annonce, couleurs du back-office.	1
	Développer l'éditeur de couleurs (20 réglages : marque, navigation, boutons, prix, badges, pied de page…) et leurs palettes prédéfinies.	1
	Appliquer couleurs et polices sur la vitrine (variables `-t-*`, classes d'ancrage `t-nav`, `t-btn`…) ; la couleur de navigation choisie s'applique aussi au-dessus de la bannière.	1
En tant que commerçant, je souhaite personnaliser ma page d'accueil	Découper l'accueil des huit modèles en sections nommées et définir leur ordre par défaut (`sectionRegistry`).	1

Suite à la page suivante

User Story	Tâches	Estimation
	Développer la page ń Page d'accueil ż : afficher, masquer et réordonner les sections, réécrire tous les textes (le texte du secteur est proposé).	1
	Choisir les produits mis en avant et composer une section ń Sélection ż personnalisée.	1
	Afficher la vraie vitrine dans l'éditeur et lui transmettre les modifications non enregistrées (`postMessage`).	1
	Tester la personnalisation sur les huit modèles.	1
	Total	**17**

TABLE 4.5 – Backlog du Sprint 4

4.7 Diagramme de cas d'utilisation du Sprint 4

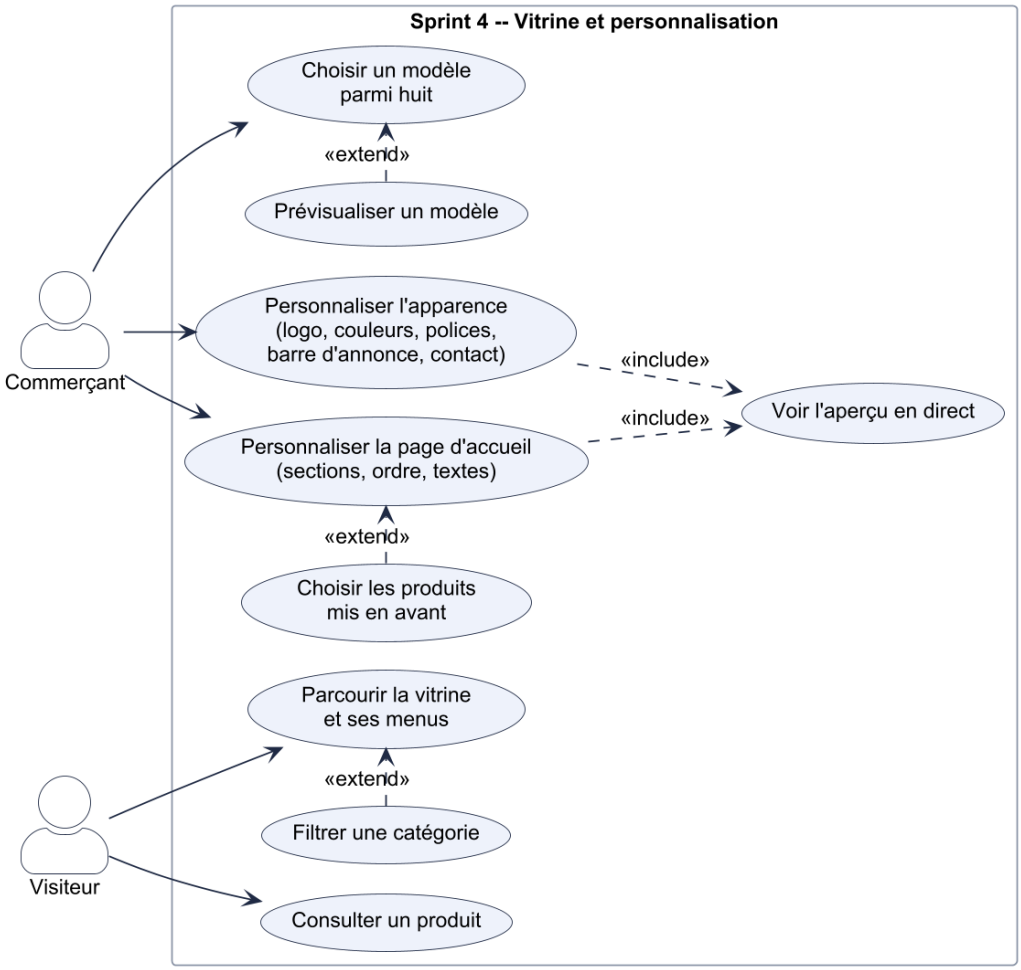


FIGURE 4.8 – Diagramme de cas d'utilisation du Sprint 4

4.8 Services web du Sprint 4

Le tableau 4.6 présente les services web du Sprint 4.

Méthode	URL	Description
Boutique et personnalisation (auth-service)		
GET	/api/v1/auth/my-shop	Lire sa boutique : modèle, secteur, couleurs, personnalisation.
PATCH	/api/v1/auth/my-shop	Enregistrer le modèle, le secteur, le nom, le logo, les couleurs (`theme`) ou la personnalisation (`settings`).
GET	/api/v1/public/shops/{slug}	Charger tout ce que la vitrine affiche d'une boutique.

Table 4.6 – Services web du Sprint 4

Toute la personnalisation passe par une seule ressource, la boutique : la vitrine la charge en une requête et le commerçant ne peut modifier que la sienne. L'ancienne table d'apparence globale, commune à toutes les boutiques, n'est plus modifiable que par le super administrateur, pour le style par défaut de Sellio.

4.9 Les cas d'utilisation du Sprint 4

4.9.1 Cas d'utilisation ń Personnaliser la page d'accueil ż

4.9.1.1 Description textuelle

Le tableau 4.7 présente la description textuelle de ce cas.

Acteurs	Commerçant, Collaborateur disposant de la permission ń Apparence ż
Objectif	Composer la page d'accueil de la vitrine sans écrire de code, en voyant le résultat avant de le publier.
Pré-condition	Le commerçant est connecté et possède une boutique.
Scénario principal	1. Le commerçant ouvre ń Page d'accueil ż. Le système affiche la liste des sections du modèle de la boutique et, à côté, la vraie vitrine. 2. Il masque une section, en déplace une autre ; l'aperçu se met à jour immédiatement, sans rien enregistrer. 3. Il ouvre une section et réécrit ses textes ; chaque champ montre le texte proposé pour son secteur, qu'il suffit de laisser vide pour le garder. 4. Il choisit les produits mis en avant ou compose une section ń Sélection ż avec ses propres produits. 5. Il enregistre ; le système valide la personnalisation et la stocke dans la boutique. 6. Les visiteurs voient la nouvelle page d'accueil dès leur prochaine visite.

Scénario alternatif	**Changement de modèle :** les sections que le nouveau modèle ne possède pas sont ignorées ; ses autres sections gardent l'ordre choisi. **Réinitialisation :** le commerçant revient à la page d'accueil d'origine du modèle. **Lien invalide :** un lien de réseau social qui ne commence pas par `https://` est refusé, pour qu'aucun lien ne puisse exécuter de script.

TABLE 4.7 – Description textuelle du cas ń Personnaliser la page d'accueil ż

4.9.1.2 Diagramme de séquence système

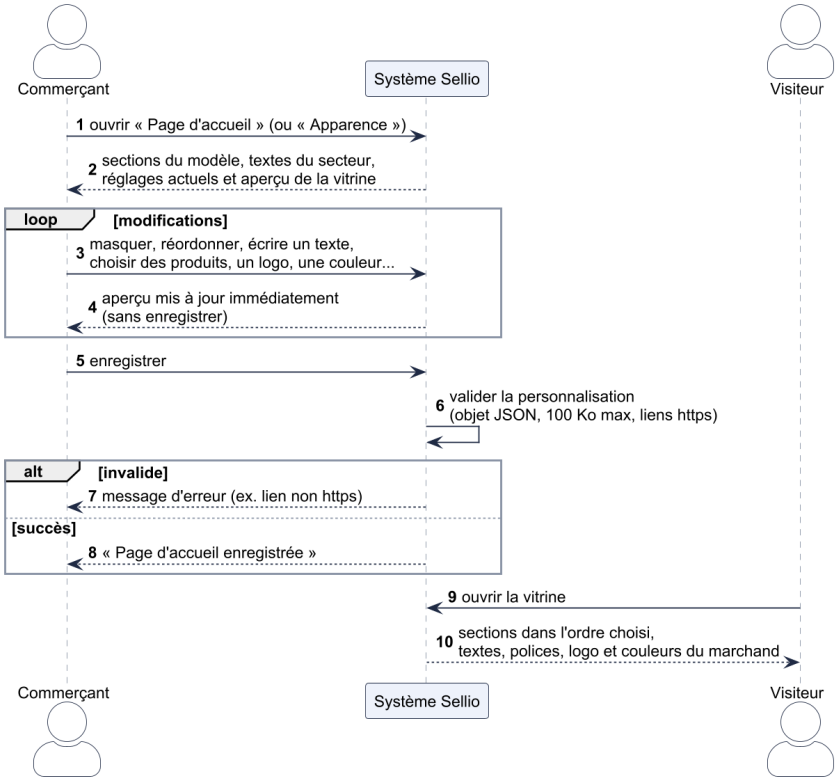


FIGURE 4.9 – Diagramme de séquence système du cas ń Personnaliser la page d'accueil ż

4.9.1.3 Diagramme de séquence objet

La figure 4.10 montre le mécanisme d'aperçu : l'éditeur affiche la vitrine dans un cadre (*iframe*) et lui envoie les réglages non enregistrés par `postMessage` ; la vitrine n'accepte ces messages que de l'origine du back-office, puis recalcule l'ordre des sections et les textes.

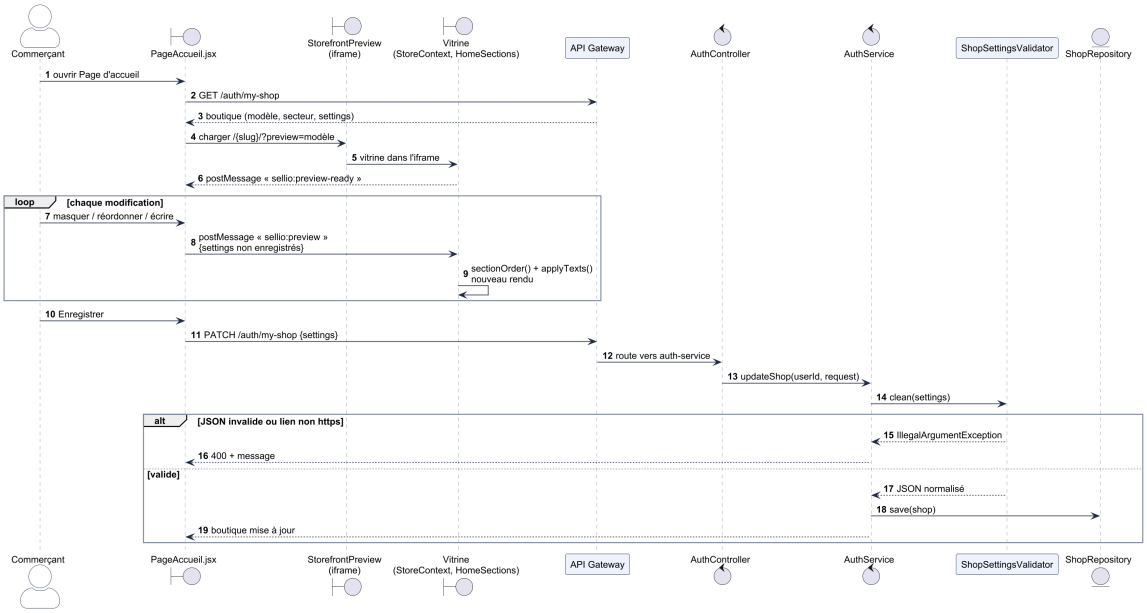
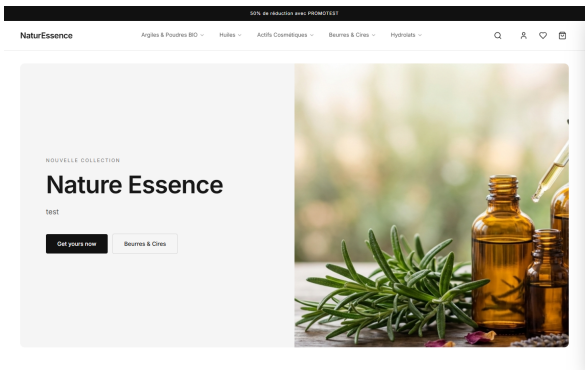


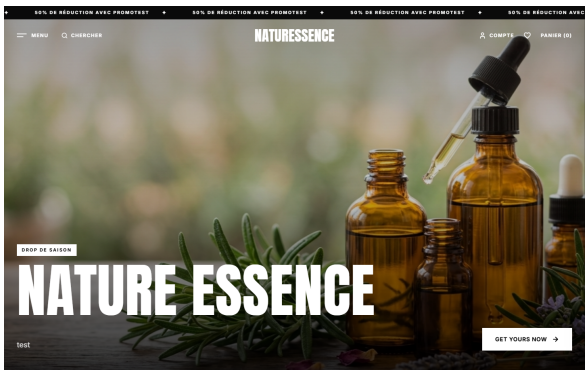
FIGURE 4.10 – Diagramme de séquence objet du cas ń Personnaliser la page d'accueil ż

4.10 Réalisation du Sprint 4

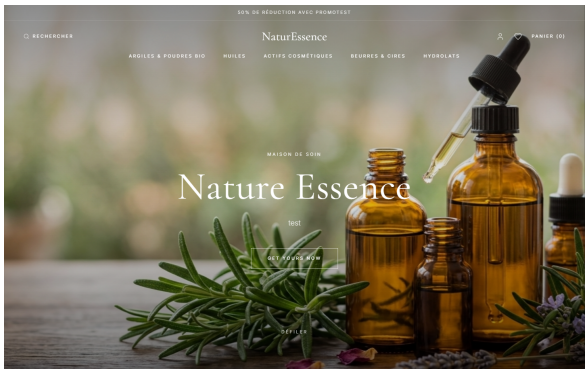
Les huit modèles : la figure 4.11 présente la même boutique affichée avec chacun des huit modèles. Le contenu (produits, catégories, bannières) est identique ; la structure, les polices et le style changent.



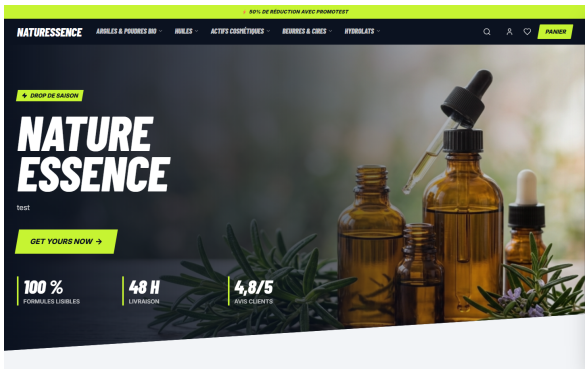
(a) Minimal



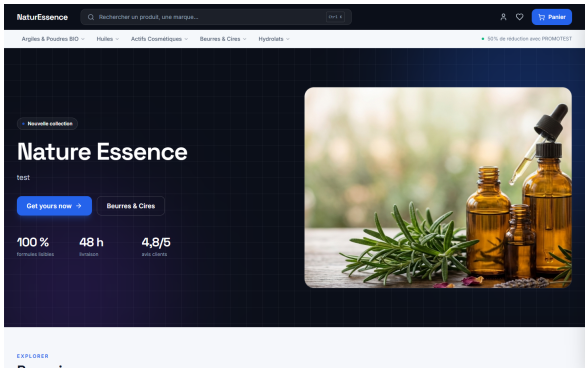
(b) Bold



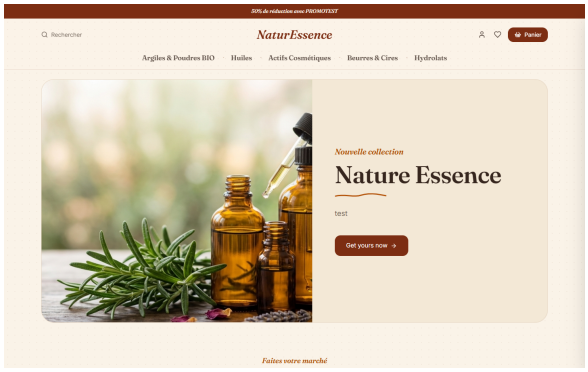
(c) Luxury



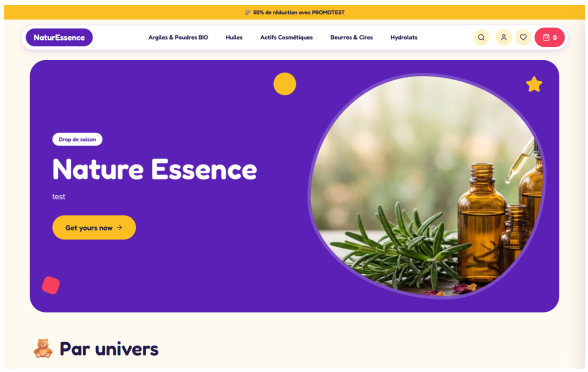
(d) Sport



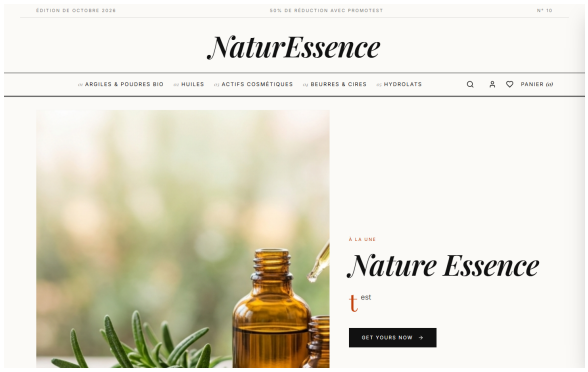
(e) Tech



(f) Artisan



(g) Pop



(h) Éditorial

FIGURE 4.11 – Une même boutique dans les huit modèles de vitrine

Type de boutique : la figure 4.12 présente la page où le commerçant change de modèle (les modèles recommandés pour son secteur sont signalés) ou de secteur.

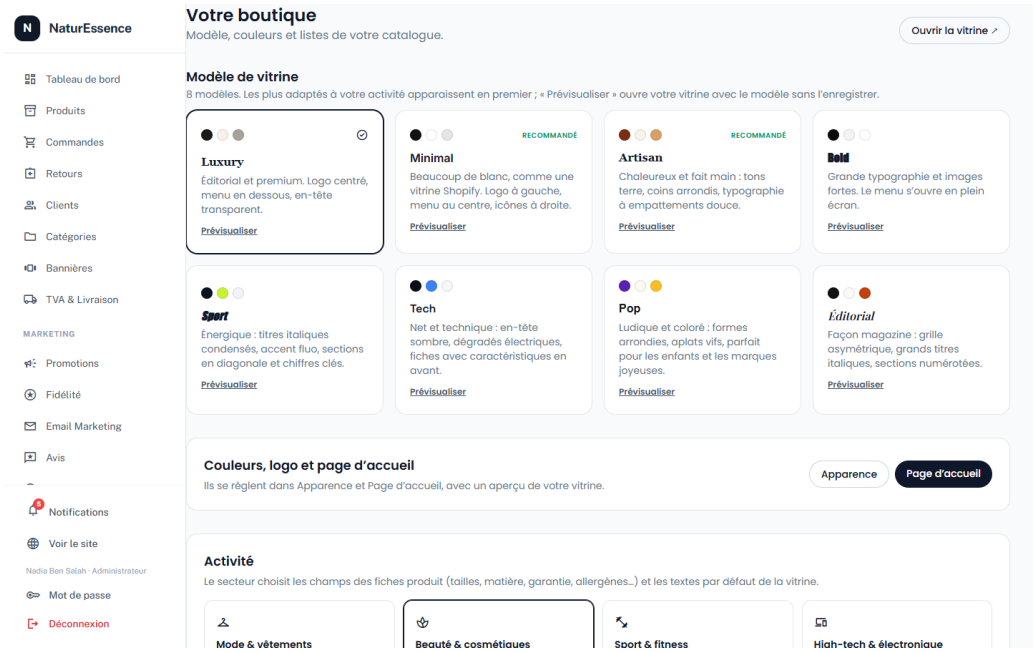


FIGURE 4.12 – Choix du modèle et du secteur

Apparence : la figure 4.13 présente la page ń Apparence ż, onglet ń Logo & identité ż : le logo, sa taille réglable, le nom de la boutique, ses coordonnées et ses réseaux sociaux, avec à droite la vitrine qui reflète chaque modification. Les onglets ń Couleurs ż (figure 4.14), ń Polices ż, ń Barre d'annonce ż et ń Back-office ż complètent la page.

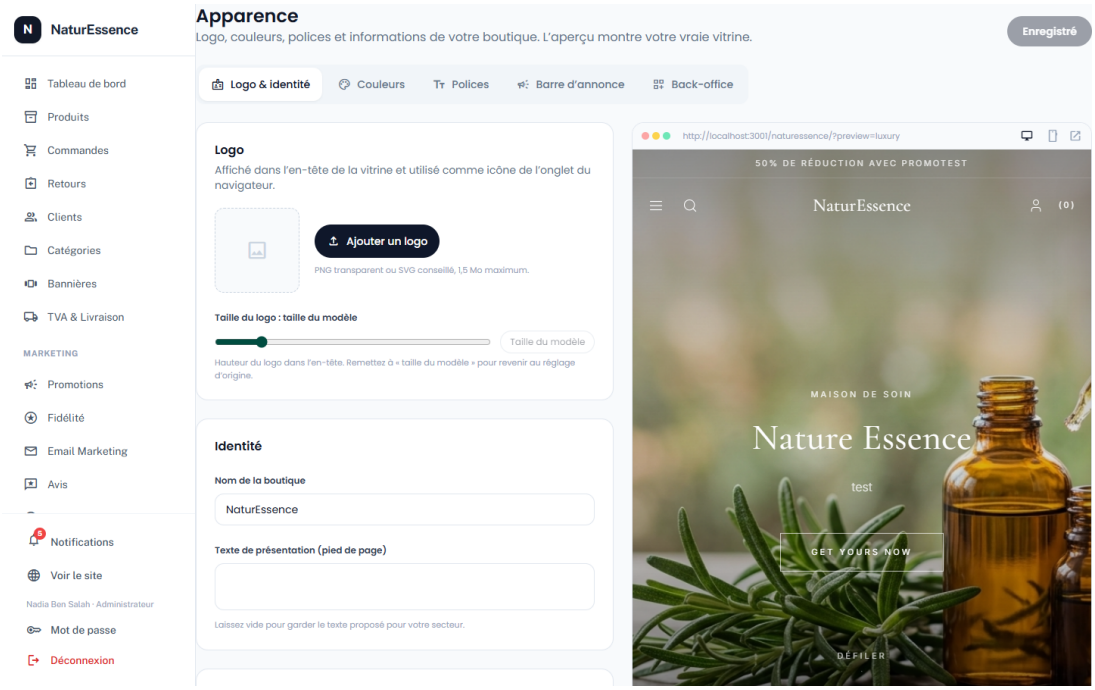


FIGURE 4.13 – Page ń Apparence ż avec l'aperçu en direct de la vitrine

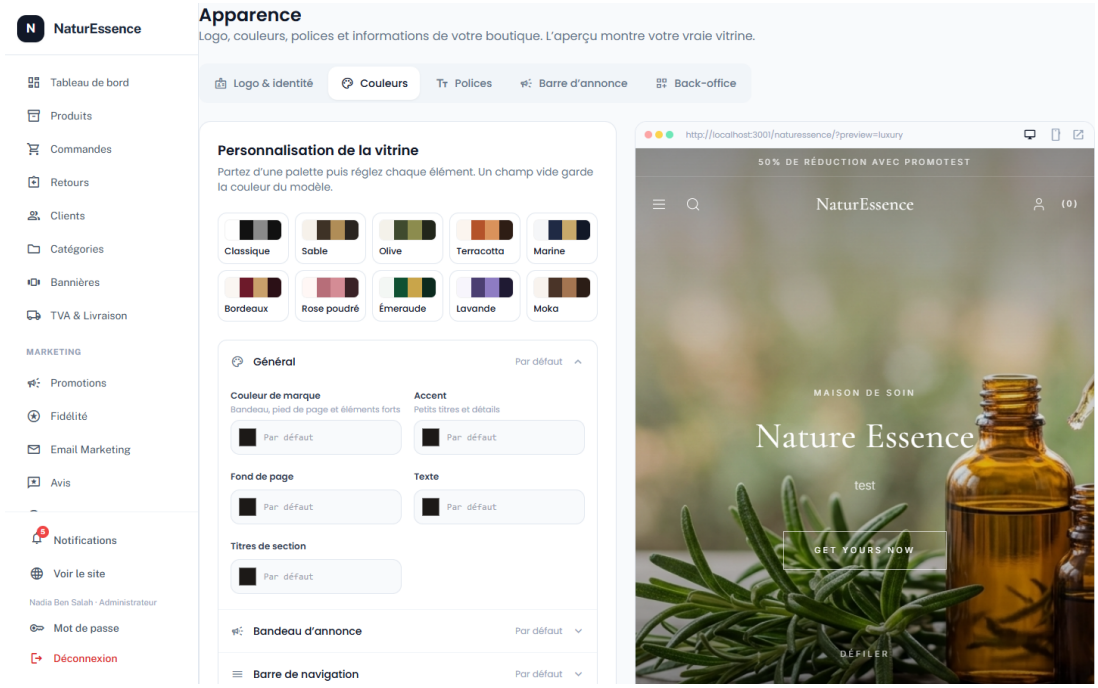


FIGURE 4.14 – Onglet ń Couleurs ż : palettes et réglages par élément

Page d'accueil : la figure 4.15 présente l'éditeur de la page d'accueil : chaque section peut être masquée, déplacée et ses textes réécrits ; l'aperçu de droite est la vraie vitrine.

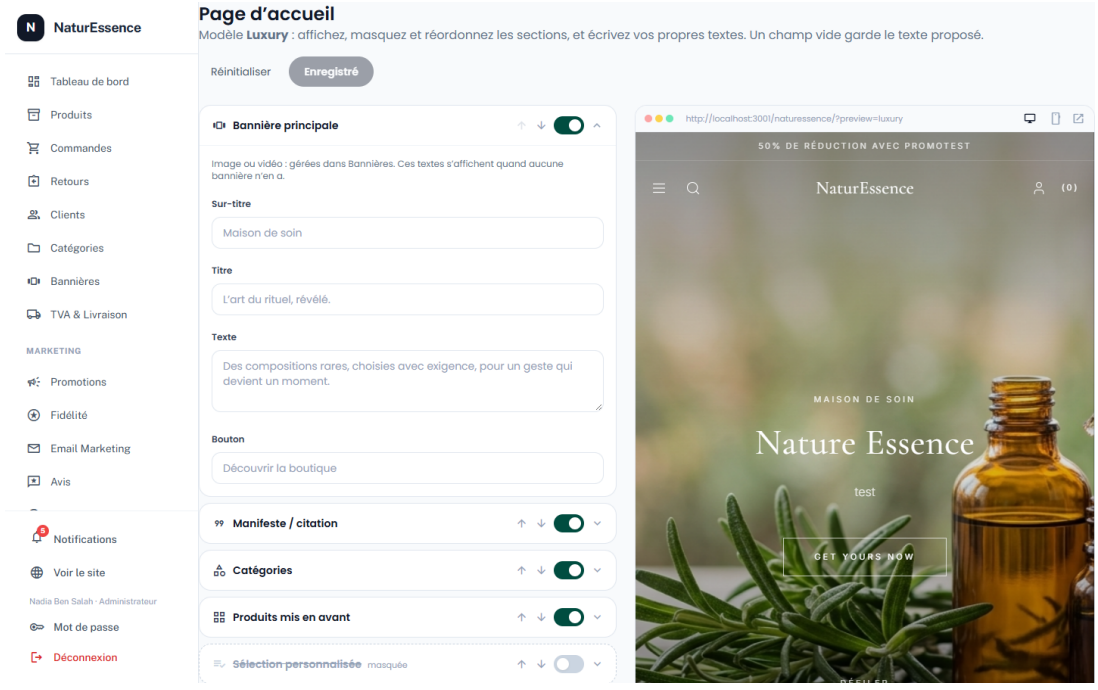


FIGURE 4.15 – Éditeur de la page d'accueil

Fiche produit : les figures 4.16 et 4.17 présentent la fiche d'un produit sur la vitrine : le client choisit la contenance d'un produit de beauté, le tour de doigt d'une bague, et retrouve les caractéristiques propres au type du produit.

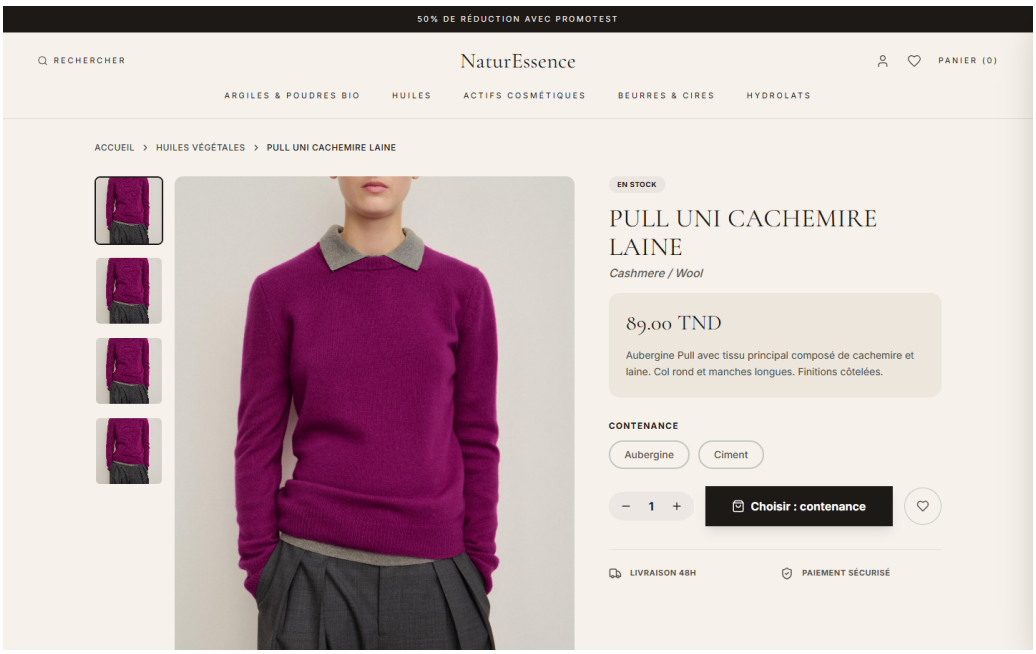


FIGURE 4.16 – Fiche produit sur la vitrine

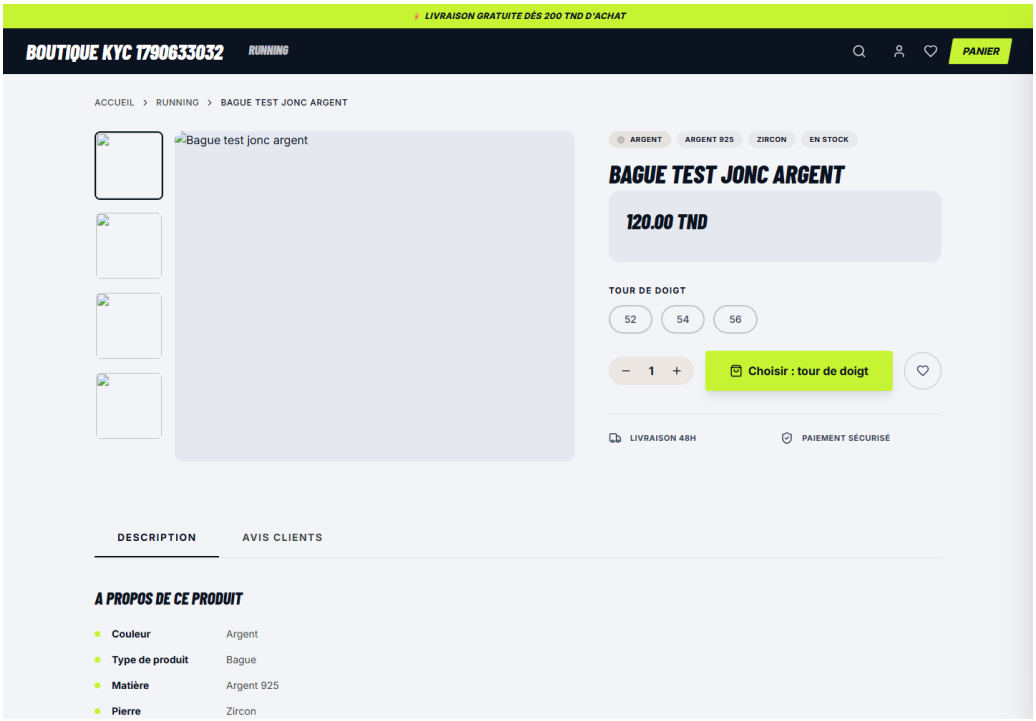


FIGURE 4.17 – Fiche d'une bague : choix du tour de doigt et caractéristiques

Conclusion

Cette release a doté chaque boutique d'un catalogue adapté à son secteur et d'une vitrine qui lui ressemble, que le commerçant compose lui-même en voyant le résultat.

Les caractéristiques des produits ne servent pas qu'à l'affichage : elles alimentent aussi la partie ń contenu ż du moteur de recommandation présenté au chapitre 8. La release suivante porte sur le parcours d'achat.

Chapitre 5

Release 3 : Parcours transactionnel

Introduction

Ce chapitre présente la troisième release, cur transactionnel de la plateforme. Le Sprint 5 couvre l'acte d'achat : panier dynamique, tunnel de commande et paiement par carte avec Stripe. Le Sprint 6 couvre ce qui suit la commande : gestion des statuts par le commerçant, tableau de bord des ventes, retours, paramétrage de la TVA et des zones de livraison, et avis clients.

5.1 Backlog du Sprint 5

Le tableau 5.1 présente le **backlog du Sprint 5**.

User Story	Tâches	Estimation
En tant que visiteur, je souhaite gérer mon panier en choisissant la taille	Développer le contexte panier (ajout, quantité, suppression, total) persistant dans le navigateur.	1
	Distinguer les lignes par produit, taille et couleur (clé de ligne).	1
	Bloquer ń Ajouter au panier ż tant que la taille n'est pas choisie, sur la fiche et sur la carte produit.	1
	Développer le tiroir de panier.	1
	Tester la fonctionnalité.	1
En tant que client, je souhaite passer commande avec livraison et code promo	Développer l'API de création de commande (sous-total, remise, livraison, TVA incluse) ; réserver le stock auprès du catalogue par son API interne.	1
	Calculer les frais de livraison selon la zone et la TVA selon la configuration.	1
	Développer l'API de validation d'un code promo (dates, montant minimum, catégories, produits).	1

Suite à la page suivante

User Story	Tâches	Estimation
	Développer le tunnel de commande (adresse, livraison, paiement, récapitulatif) et la page de confirmation.	1
	Envoyer l'e-mail de confirmation de commande.	1
	Tester la fonctionnalité.	1
En tant que client, je souhaite retrouver mon panier sur un autre appareil	Développer l'API de sauvegarde du panier du client connecté et sa restauration à la connexion.	1
En tant que client, je souhaite payer ma commande par carte	Configurer le SDK Stripe côté serveur (clé secrète en variable d'environnement).	1
	Développer l'API de création d'un *PaymentIntent*.	1
	Intégrer Stripe Elements et confirmer le paiement dans le navigateur.	1
	Conserver le paiement à la livraison comme alternative.	1
	Tester avec les cartes de test Stripe (succès, refus, authentification 3-D Secure).	1
	Total	**17**

TABLE 5.1 – Backlog du Sprint 5

5.2 Diagramme de cas d'utilisation du Sprint 5

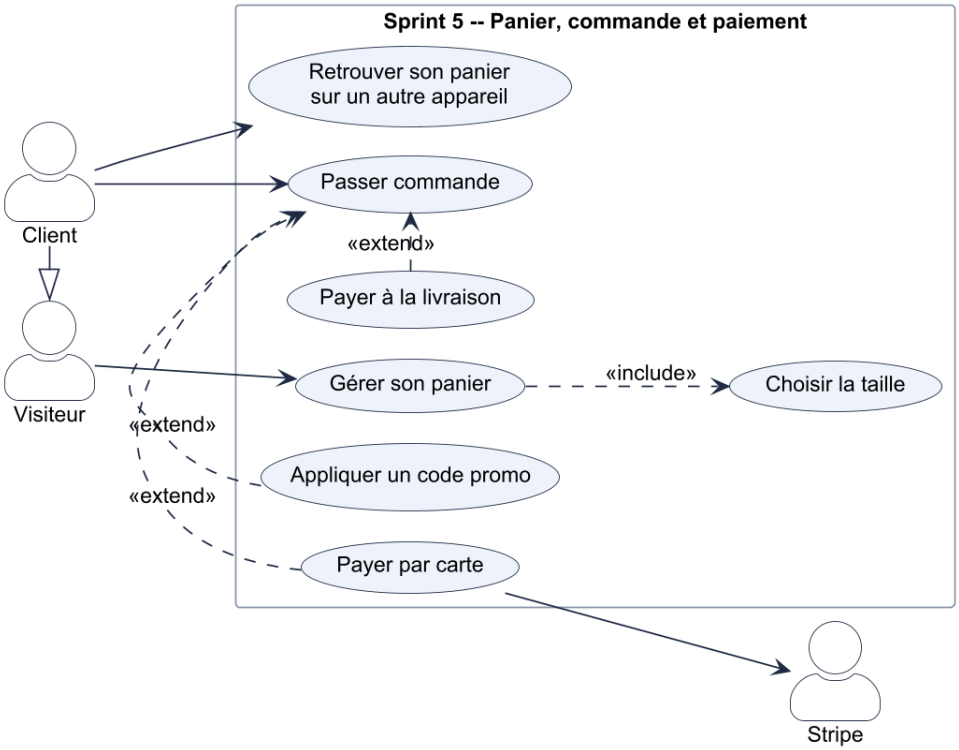


FIGURE 5.1 – Diagramme de cas d'utilisation du Sprint 5

5.3 Services web du Sprint 5

Le tableau 5.2 présente les services web du Sprint 5.

Méthode	URL	Description
Tunnel de commande — /api/v1/public/checkout (order-service)		
GET	/api/v1/public/checkout/shipping-zones	Zones de livraison et tarifs.
GET	/api/v1/public/checkout/tva-config	Configuration de la TVA.
POST	/api/v1/public/checkout/orders	Créer une commande.
GET	/api/v1/public/coupons/validate	Valider un code promo pour un panier.
GET	/api/v1/public/coupons/announcement	Promotion à afficher dans le bandeau d'annonce.
Paiement — /api/v1/public/stripe		
POST	/api/v1/public/stripe/payment-intent	Créer un *PaymentIntent* et retourner son *client secret*.
Panier du client — /api/v1/profile (auth-service)		
GET	/api/v1/profile/cart	Récupérer le panier sauvegardé.
PUT	/api/v1/profile/cart	Sauvegarder le panier.

TABLE 5.2 – Services web du Sprint 5

5.4 Les cas d'utilisation du Sprint 5

5.4.1 Cas d'utilisation ń Passer commande et payer par carte ż

5.4.1.1 Description textuelle

Le tableau 5.3 présente la description textuelle de ce cas.

Acteurs	Client, Stripe
Objectif	Transformer un panier en commande payée.
Pré-condition	Le panier contient au moins un article ; la taille, la pointure ou le format de chaque article qui en propose est choisi ; la boutique est active.
Scénario principal	1. Le client ouvre le tunnel de commande et saisit ou choisit son adresse. 2. Il choisit sa ville ; le système affiche les frais de la zone de livraison correspondante et la TVA. 3. Il saisit un code promo ; le système le valide et affiche la remise. 4. Il choisit ń Carte bancaire ż ; le système crée un *PaymentIntent* Stripe et renvoie son *client secret*. 5. Le client saisit sa carte dans le champ sécurisé Stripe, qui confirme le paiement (avec authentification 3-D Secure si la banque l'exige). 6. Après confirmation, l'interface envoie la commande ; le système calcule le sous-total, applique la remise, la livraison (gratuite au-delà du seuil) et la TVA incluse dans les prix, demande au catalogue de réserver le stock, puis enregistre la commande au statut `EN_ATTENTE` avec le mode de paiement `CARTE`, en conservant le nom et le prix de chaque article au moment de l'achat. 7. Le système envoie l'e-mail de confirmation ; le client est redirigé vers la page de confirmation et un événement d'achat est enregistré pour l'analyse comportementale.
Scénario alternatif	**Code promo invalide :** à l'étape 3, le système en donne la raison (expiré, montant minimum non atteint, réservé à d'autres catégories). **Paiement refusé :** à l'étape 5, le message de Stripe est affiché ; aucune commande n'est créée et le client peut réessayer. **Paiement à la livraison :** à l'étape 4, si le client choisit ce mode, la commande est créée directement, sans passer par Stripe.

Table 5.3 – Description textuelle du cas ń Passer commande et payer par carte ż

5.4.1.2 Diagramme de séquence système

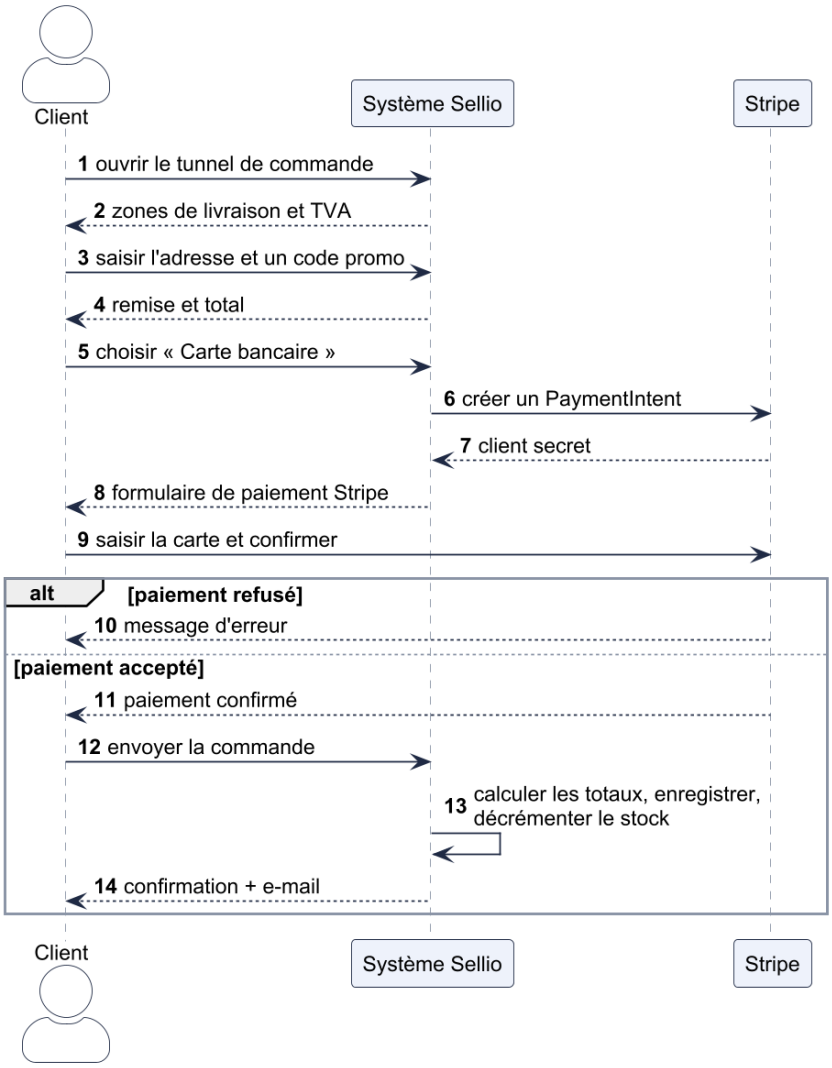


FIGURE 5.2 – Diagramme de séquence système du cas ń Passer commande et payer par carte ż

5.4.1.3 Diagramme de séquence objet

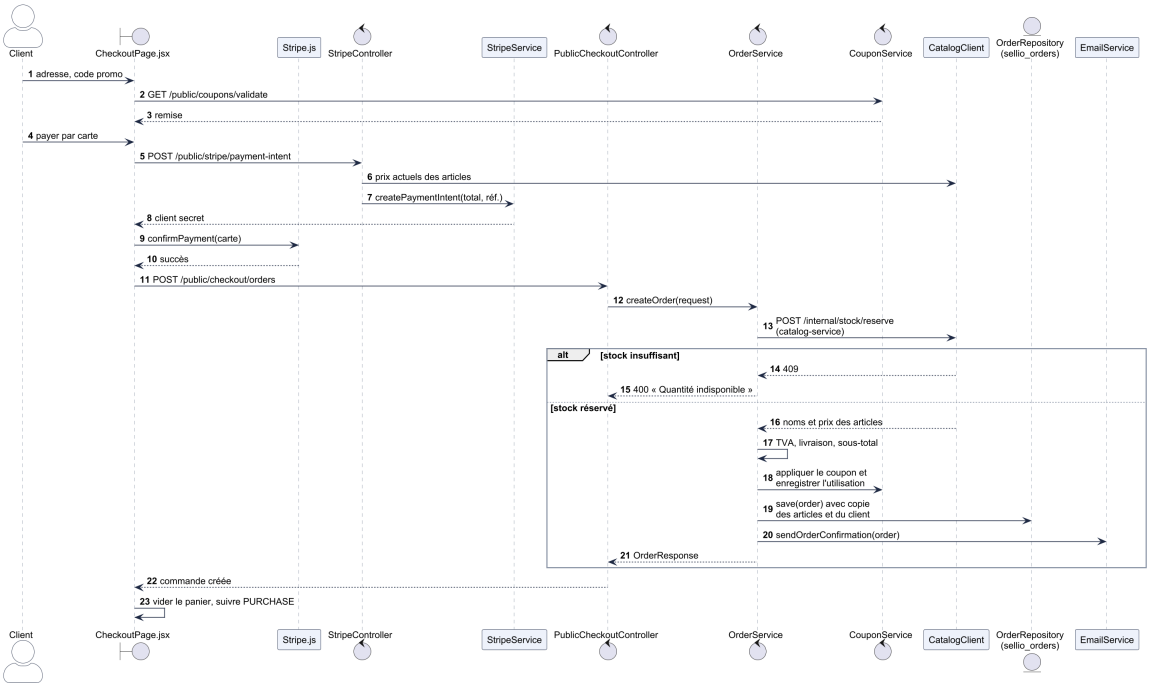


FIGURE 5.3 – Diagramme de séquence objet du cas ń Passer commande et payer par carte ż

5.5 Réalisation du Sprint 5

Panier : la figure 5.4 présente le tiroir de panier. Chaque ligne correspond à une combinaison produit / taille / couleur ; la section ń Souvent achetés ensemble ż, ajoutée au Sprint 10, y propose des articles complémentaires.

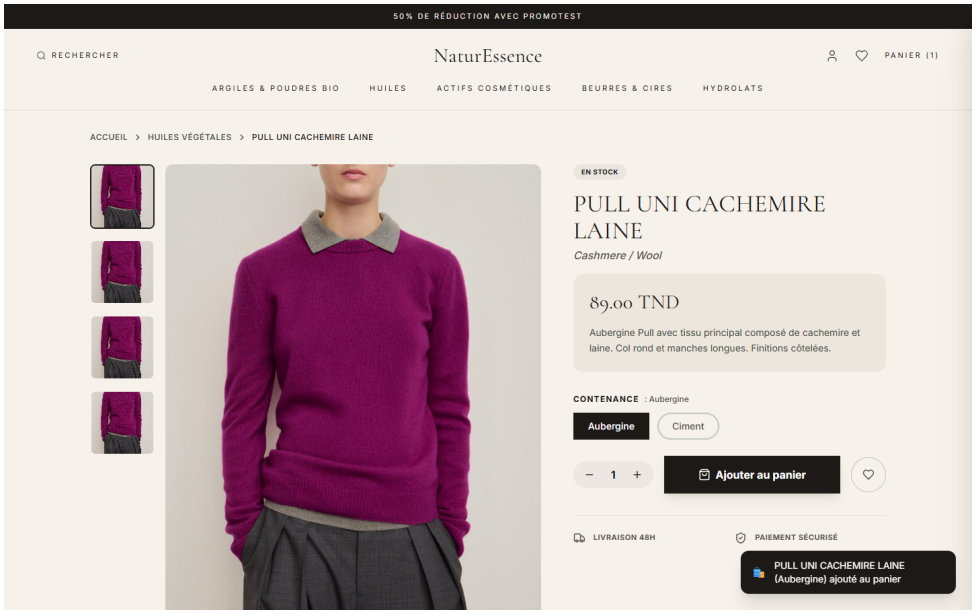


FIGURE 5.4 – Tiroir de panier

Tunnel de commande et paiement : les figures 5.5 et 5.6 présentent le tunnel de commande puis le formulaire de paiement Stripe.

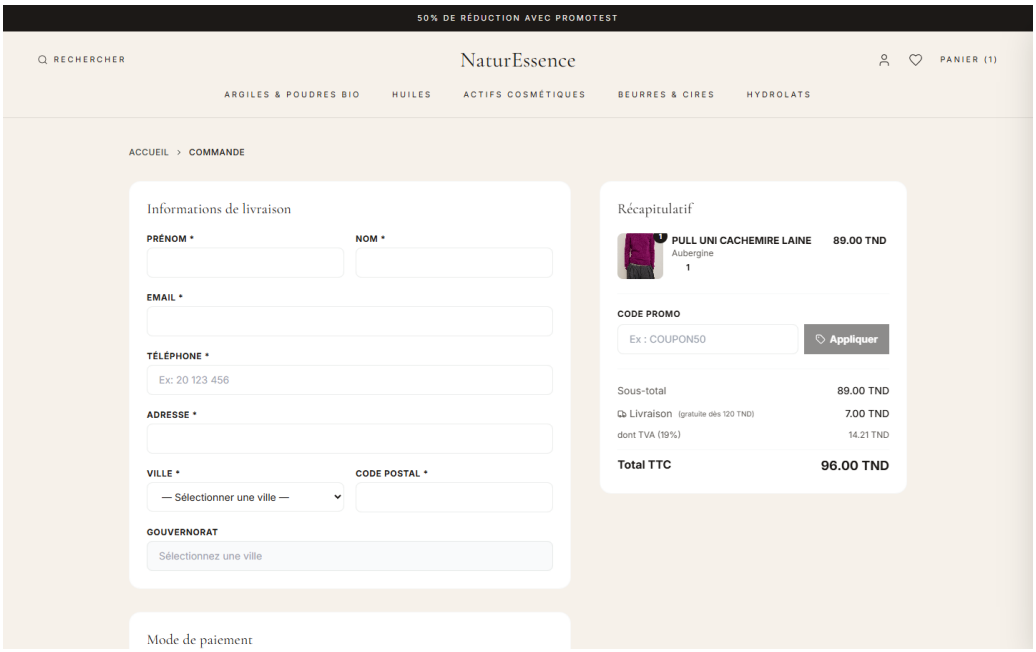


FIGURE 5.5 – Tunnel de commande : adresse, livraison et code promo

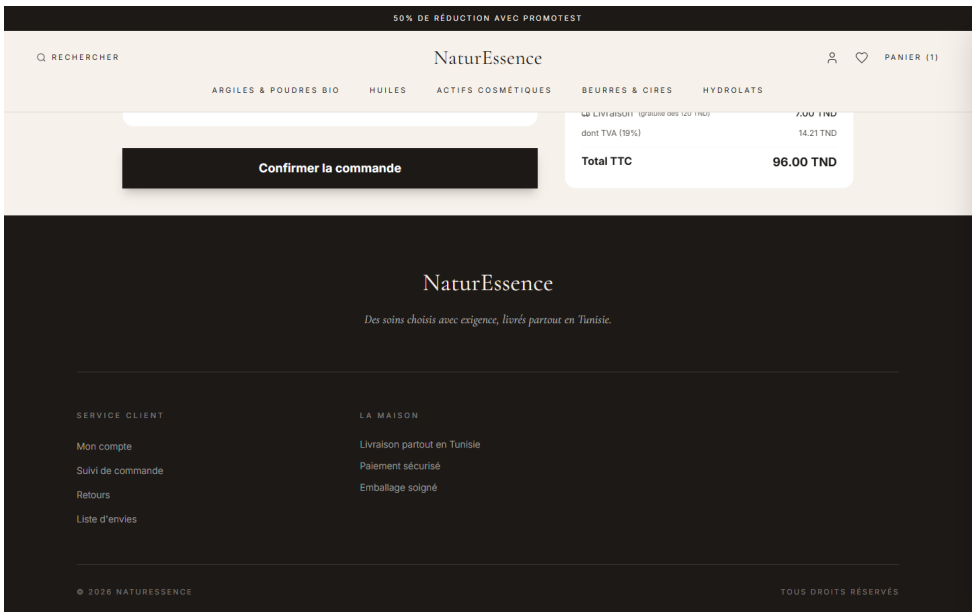


FIGURE 5.6 – Paiement par carte avec Stripe

Remarque sur la devise : Stripe ne prend pas en charge le dinar tunisien ; en mode test, les montants sont donc débités en euros. Une mise en production en Tunisie passerait par un prestataire de paiement local ou par la conversion des montants, et la confirmation de la commande devrait alors s'appuyer sur un *webhook* du prestataire plutôt que sur le seul retour du navigateur. De même, les prix unitaires transmis par le panier devraient être revérifiés côté serveur à partir du catalogue, et le stock contrôlé avant la validation. Ces points figurent dans les perspectives.

5.6 Backlog du Sprint 6

Le tableau 5.4 présente le **backlog du Sprint 6**.

User Story	Tâches	Estimation
En tant que commerçant, je souhaite gérer mes commandes et leurs statuts	Développer l'API de liste, de détail et de recherche par référence des commandes.	1
	Développer le changement de statut (en attente, confirmée, en préparation, expédiée, livrée, annulée, remboursée), avec remise en stock en cas d'annulation et e-mail au client à la livraison.	1
	Développer les interfaces ń Commandes ż et ń Détail commande ż.	1
	Tester la fonctionnalité.	1
En tant que client, je souhaite suivre mes commandes	Développer l'API et la page ń Mes commandes ż.	1
En tant que commerçant, je souhaite consulter un tableau de bord des ventes	Développer l'API d'agrégats (chiffre d'affaires, commandes, panier moyen, meilleurs produits).	1
	Développer le tableau de bord avec graphiques.	1
En tant que client, je souhaite demander un retour	Développer l'API de demande de retour (articles, motif) dans le délai de la politique.	1
	Développer la page ń Mes retours ż.	1
En tant que commerçant, je souhaite traiter les retours et définir ma politique de retour	Développer l'API de traitement des retours et de la politique de retour.	1
	Développer les interfaces ń Retours ż, ń Détail retour ż et ń Historique des remboursements ż.	1

Suite à la page suivante

User Story	Tâches	Estimation
	Tester la fonctionnalité.	1
En tant que commerçant, je souhaite paramétrer la TVA et les zones de livraison	Développer l'API de configuration de la TVA et des taux.	1
	Développer l'API des zones de livraison (villes, tarif, seuil de gratuité, délai).	1
	Développer l'interface ń TVA et livraison ż.	1
En tant que client, je souhaite publier un avis sur un produit acheté	Développer l'API de publication (un avis par produit et par commande) et l'affichage sur la fiche produit.	1
En tant que commerçant, je souhaite modérer les avis et y répondre	Développer l'API et l'interface de modération (statut, réponse, suppression).	1
	Total	**17**

TABLE 5.4 – Backlog du Sprint 6

5.7 Diagramme de cas d'utilisation du Sprint 6

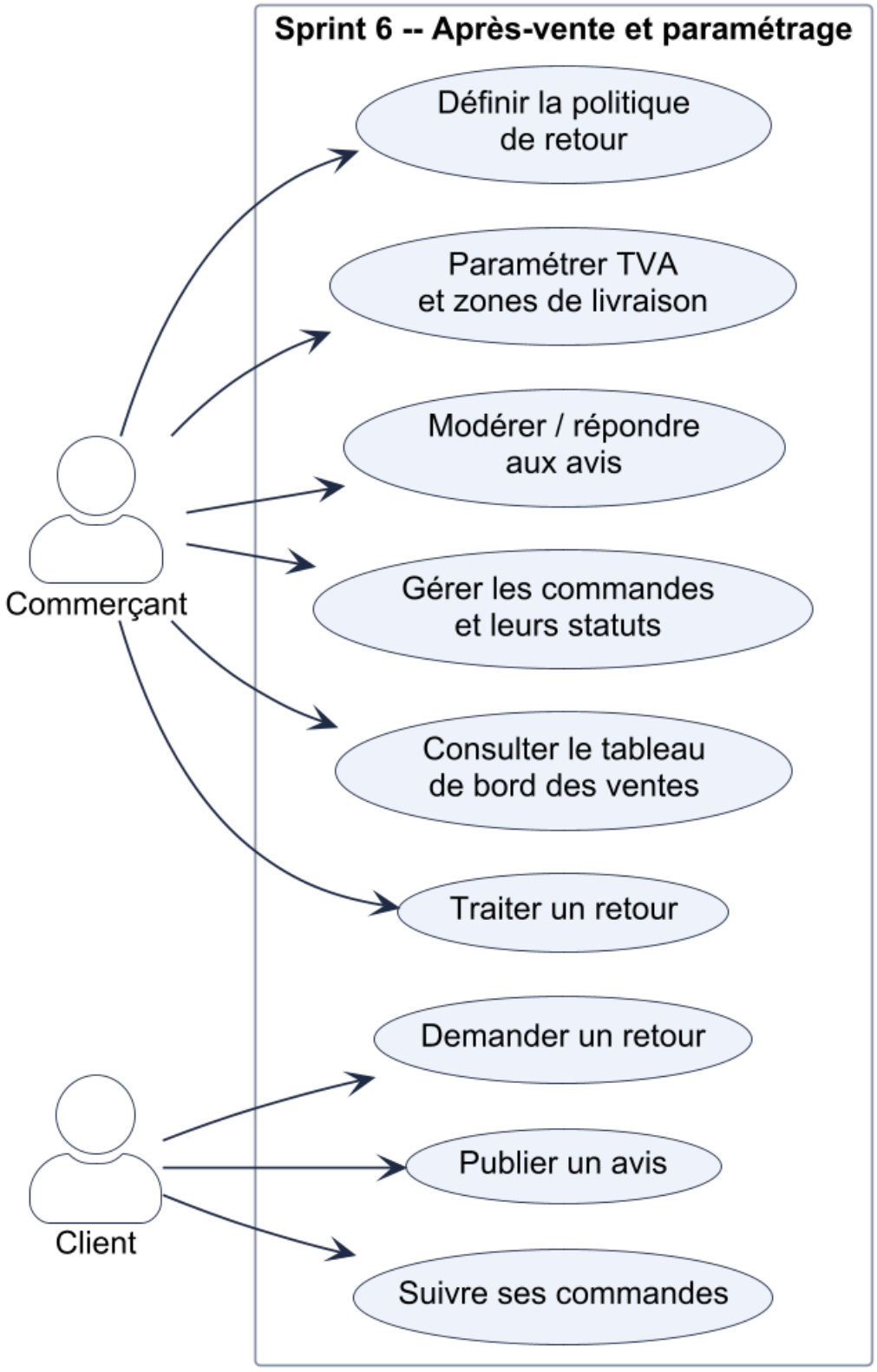


FIGURE 5.7 – Diagramme de cas d'utilisation du Sprint 6

5.8 Services web du Sprint 6

Le tableau 5.5 présente les services web du Sprint 6.

Méthode	URL	Description
Commandes — order-service		
GET	/api/v1/admin/orders	Lister les commandes de la boutique.
GET	/api/v1/admin/orders/{id}	Détail d'une commande.
GET	/api/v1/admin/orders/reference/{reference}	Rechercher une commande par référence.
GET	/api/v1/admin/orders/user/{userId}	Commandes d'un client.
PATCH	/api/v1/admin/orders/{id}/status	Changer le statut d'une commande.
GET	/api/v1/profile/orders	Commandes du client connecté.
GET	/api/v1/admin/dashboard	Indicateurs du tableau de bord.
Retours		
GET, POST	/api/v1/profile/returns	Lister, créer ses demandes de retour.
GET	/api/v1/admin/returns	Lister les retours.
PATCH	/api/v1/admin/returns/{id}/status	Accepter, refuser ou rembourser un retour.
GET, PUT	/api/v1/admin/returns/policy	Consulter, modifier la politique de retour.
GET	/api/v1/public/return-policy	Politique de retour publique.
TVA et livraison — /api/v1/admin/tva-shipping		
GET, PUT	/api/v1/admin/tva-shipping/config	Configuration générale de la TVA.
GET, POST	/api/v1/admin/tva-shipping/rates	Lister, créer un taux de TVA.
PUT, DELETE	/api/v1/admin/tva-shipping/rates/{id}	Modifier, supprimer un taux.
GET, POST	/api/v1/admin/tva-shipping/zones	Lister, créer une zone de livraison.
PUT, DELETE	/api/v1/admin/tva-shipping/zones/{id}	Modifier, supprimer une zone.
Avis — catalog-service		
GET	/api/v1/public/reviews/product/{productId}	Avis publiés d'un produit.
POST	/api/v1/profile/reviews	Publier un avis.
GET	/api/v1/reviews/has-reviewed	Savoir si le client a déjà noté le produit.
GET	/api/v1/admin/reviews	Lister les avis à modérer.
PATCH	/api/v1/admin/reviews/{id}/statut	Publier ou masquer un avis.
PATCH	/api/v1/admin/reviews/{id}/reponse	Répondre à un avis.

Table 5.5 – Services web du Sprint 6

5.9 Les cas d'utilisation du Sprint 6

5.9.1 Cas d'utilisation ń Traiter une demande de retour ż

5.9.1.1 Description textuelle

Le tableau 5.6 présente la description textuelle de ce cas.

Acteurs	Client, Commerçant
Objectif	Permettre au client de retourner un article et au commerçant de traiter la demande jusqu'au remboursement.
Pré-condition	La commande est livrée et le délai de retour de la politique de la boutique n'est pas dépassé.
Scénario principal	1. Le client ouvre ń Mes commandes ż, choisit une commande livrée et clique sur ń Retourner ż. 2. Il sélectionne les articles, indique le motif et valide. 3. Le système vérifie le délai et crée la demande au statut ń en attente ż. 4. Le commerçant consulte la demande dans ń Retours ż et l'accepte. 5. À la réception des articles, il marque le retour comme remboursé ; le système met à jour la commande et l'historique des remboursements. 6. Le client suit l'avancement de sa demande dans ń Mes retours ż.
Scénario alternatif	**Délai dépassé :** à l'étape 3, le système refuse la demande et rappelle la politique de retour. **Retour refusé :** à l'étape 4, le commerçant refuse la demande en indiquant la raison, communiquée au client.

Table 5.6 – Description textuelle du cas ń Traiter une demande de retour ż

5.9.1.2 Diagramme de séquence système

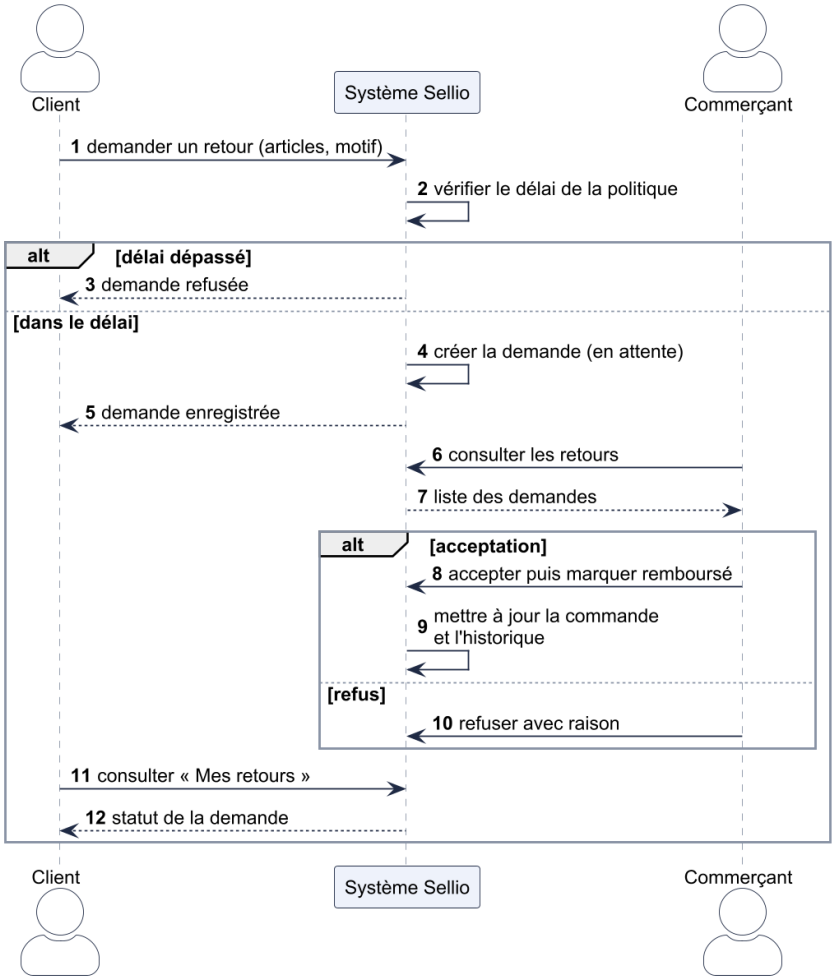


FIGURE 5.8 – Diagramme de séquence système du cas ń Traiter une demande de retour ż

5.9.1.3 Diagramme de séquence objet

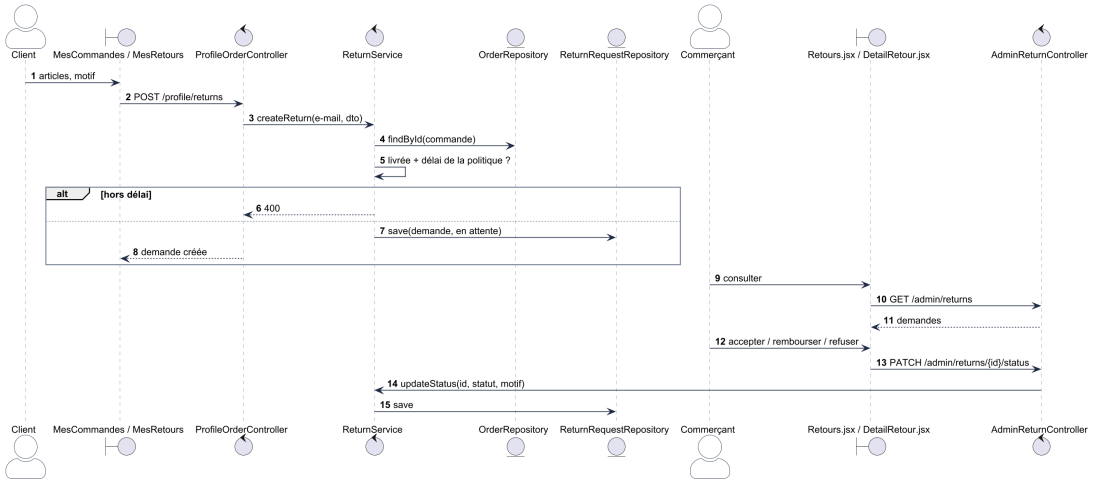


FIGURE 5.9 – Diagramme de séquence objet du cas ń Traiter une demande de retour ż

5.10 Réalisation du Sprint 6

Gestion des commandes : la figure 5.10 présente la liste des commandes et la figure 5.11 le tableau de bord des ventes.

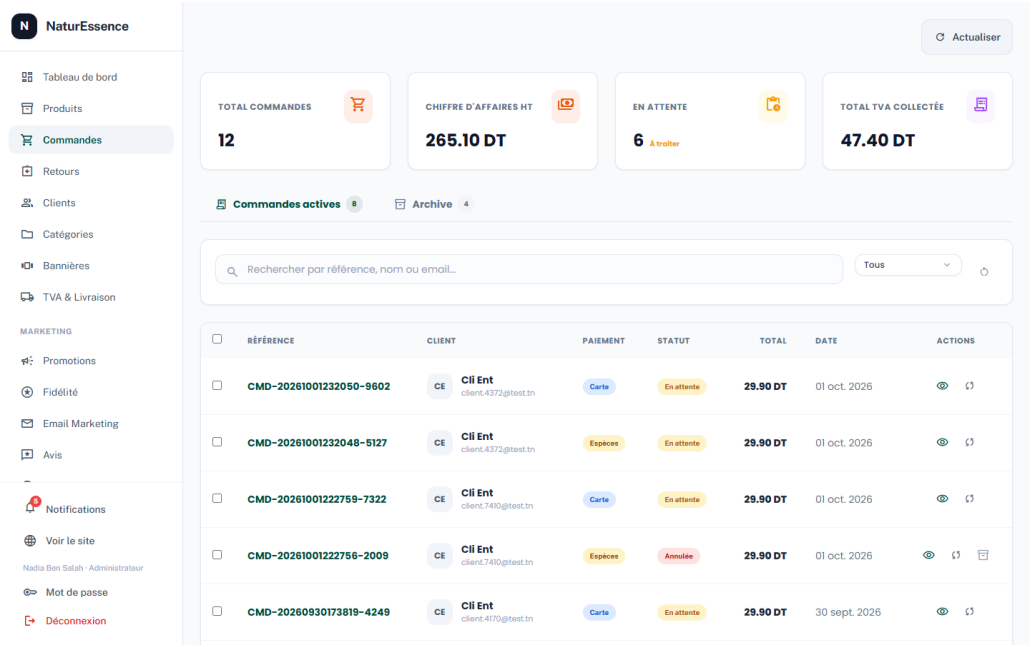


FIGURE 5.10 – Gestion des commandes

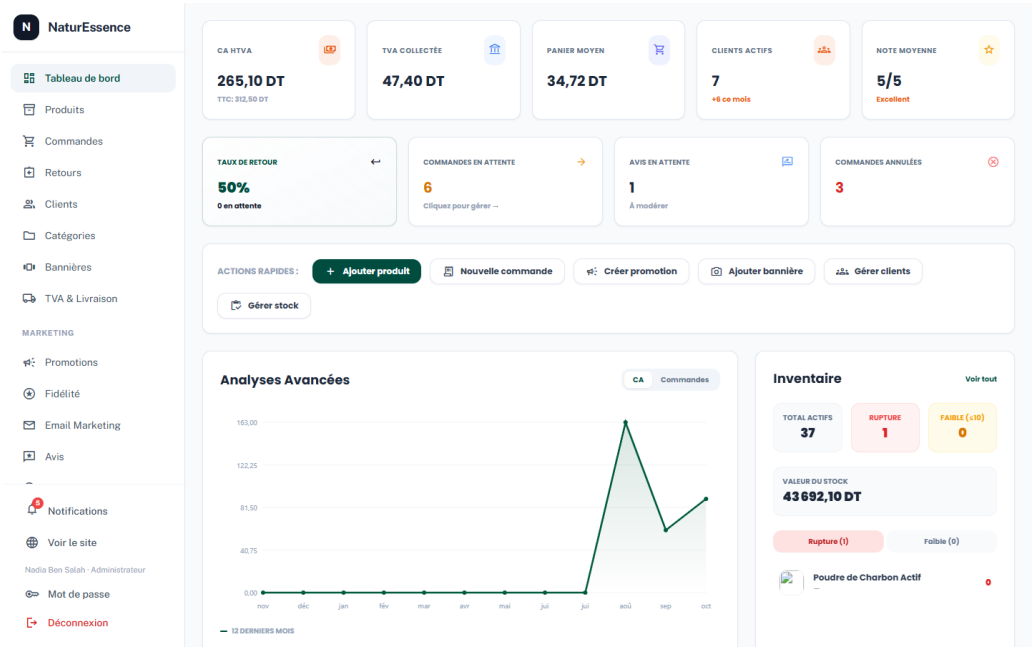


FIGURE 5.11 – Tableau de bord des ventes

Retours, TVA et livraison : les figures 5.12 et 5.13 présentent le traitement des retours et le paramétrage de la TVA et des zones de livraison.

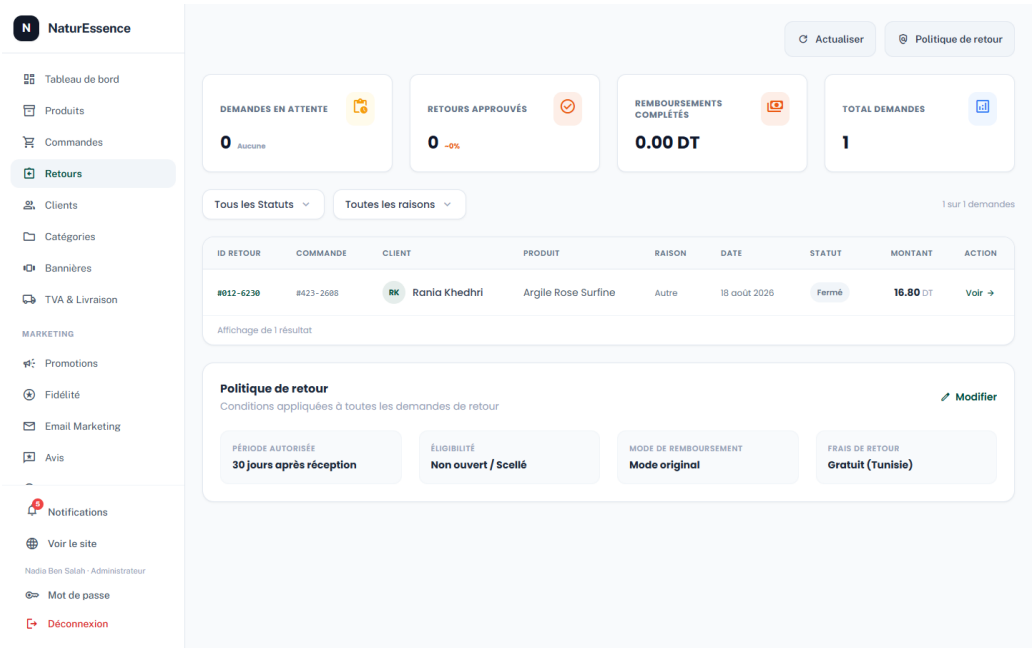


FIGURE 5.12 – Traitement des retours

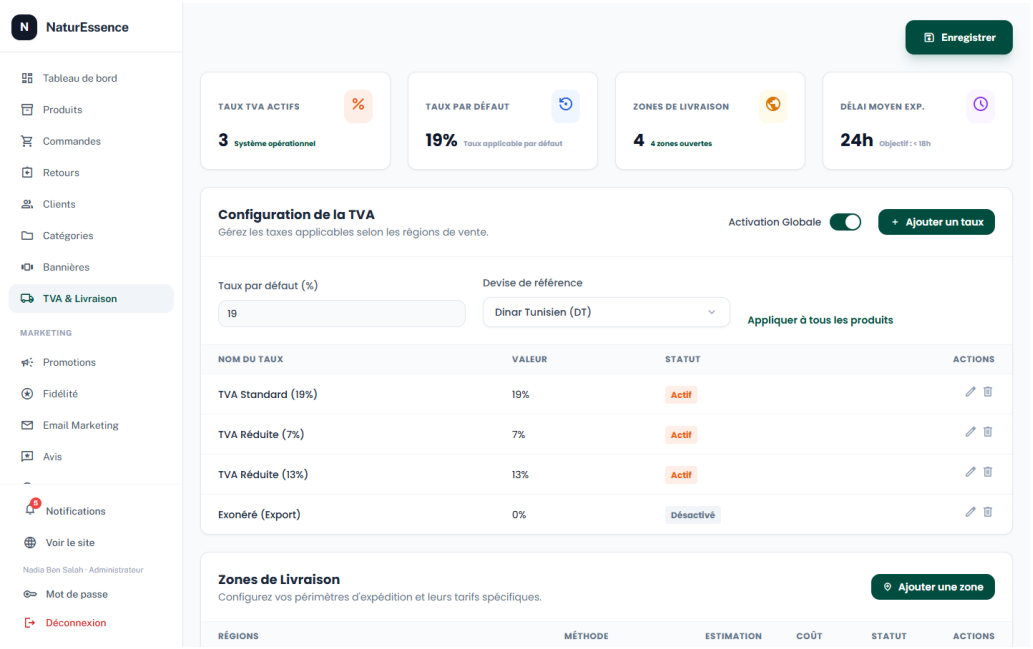


FIGURE 5.13 – Paramétrage de la TVA et des zones de livraison

Avis clients : la figure 5.14 présente la modération des avis.

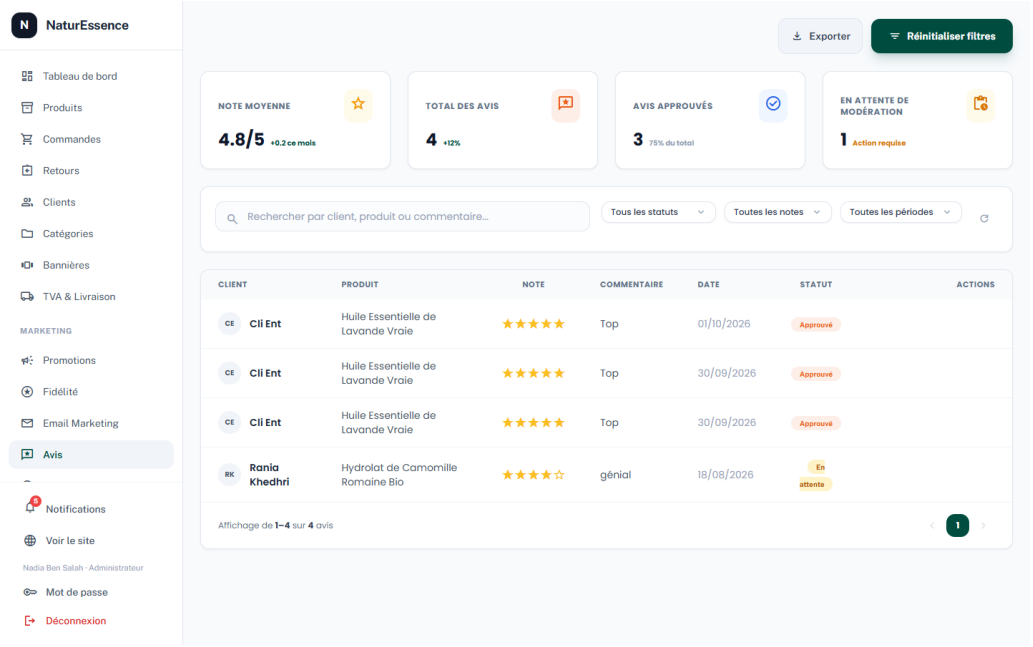


FIGURE 5.14 – Modération des avis clients

Conclusion

Cette release a rendu la plateforme pleinement transactionnelle : un client peut acheter, payer et retourner un article, et le commerçant pilote ses ventes et son paramétrage fiscal et logistique. La release suivante apporte les outils marketing et de fidélisation.

Chapitre 6

Release 4 : Marketing et fidélisation

Introduction

Vendre ne suffit pas : un commerçant doit aussi attirer de nouveaux clients et faire revenir les anciens. Ce chapitre présente la quatrième release, qui lui en donne les moyens. Le Sprint 7 couvre les promotions, les bannières ciblées et l'e-mailing ; le Sprint 8 couvre le programme de fidélité à points et ses niveaux.

6.1 Backlog du Sprint 7

Le tableau 6.1 présente le **backlog du Sprint 7**.

User Story	Tâches	Estimation
En tant que commerçant, je souhaite créer des codes promo	Développer l'entité `Coupon` : type (pourcentage, montant fixe, livraison offerte, cadeau, ń un acheté, un offert ż), valeur, montant minimum, période et plage horaire, limites globale et par client, segment, catégories et produits ciblés.	1
	Développer l'API CRUD, l'activation / désactivation et le suivi des utilisations.	1
	Calculer les statistiques par code (commandes, revenus, taux de conversion).	1
	Développer l'interface ń Promotions ż.	1
	Tester la fonctionnalité.	1
En tant que commerçant, je souhaite créer des remises automatiques	Développer l'API des remises sur un produit ou une catégorie, avec période de validité.	1
	Afficher le prix barré et le badge de remise sur la vitrine.	1
	Tester la fonctionnalité.	1

Suite à la page suivante

User Story	Tâches	Estimation
En tant que commerçant, je souhaite publier des bannières ciblées	Développer l'API des bannières : titre, images ordinateur et mobile, vidéo, bouton d'action, alignement, animation, durée, planning.	1
	Cibler une bannière par segment de clientèle et par type d'appareil.	1
	Développer le formulaire de bannière avec aperçu ordinateur et mobile.	1
	Afficher les bannières dans le carrousel d'accueil de chacun des huit modèles.	1
En tant que commerçant, je souhaite envoyer des e-mails à mes clients	Intégrer l'API d'envoi transactionnel de Brevo (clé en variable d'environnement).	1
	Développer l'envoi d'une newsletter à un segment et le contact individuel d'un client.	1
	Développer l'interface ń E-mail marketing ż et ses statistiques.	1
	Total	**15**

TABLE 6.1 – Backlog du Sprint 7

6.2 Diagramme de cas d'utilisation du Sprint 7

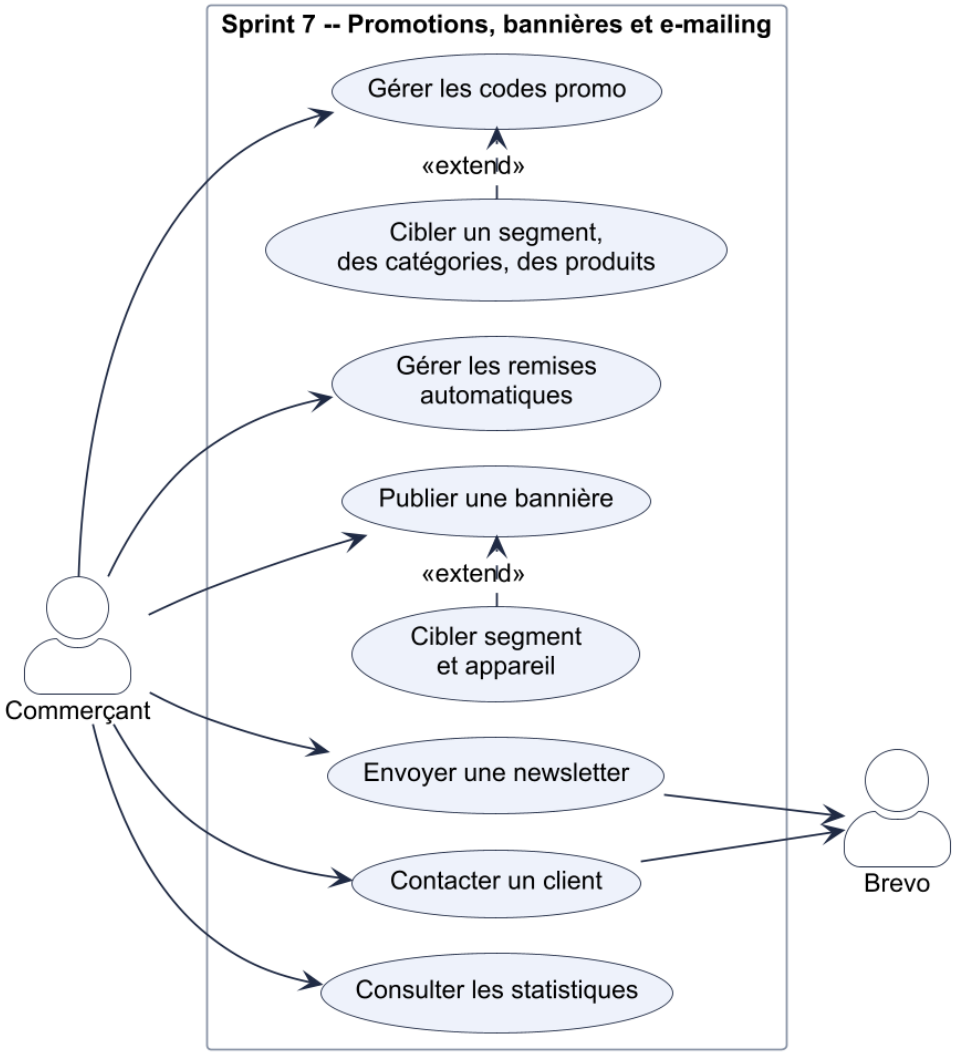


FIGURE 6.1 – Diagramme de cas d'utilisation du Sprint 7

6.3 Services web du Sprint 7

Le tableau 6.2 présente les services web du Sprint 7.

Méthode	URL	Description
Promotions — /api/v1/admin/promotions (order-service)		
GET	/api/v1/admin/promotions/stats	Statistiques des promotions.
GET, POST	/api/v1/admin/promotions/coupons	Lister, créer un code promo.
PUT, DELETE	/api/v1/admin/promotions/coupons/{id}	Modifier, supprimer un code promo.
PATCH	/api/v1/admin/promotions/coupons/{id}/toggle	Activer ou désactiver un code.
GET, POST	/api/v1/admin/promotions/discounts	Lister, créer une remise.

Méthode	URL	Description
PUT, DELETE	/api/v1/admin/promotions/discounts/{id}	Modifier, supprimer une remise.
PATCH	/api/v1/admin/promotions/discounts/{id}/toggle	Activer ou désactiver une remise.
Bannières — marketing-service		
GET, POST	/api/v1/admin/banners	Lister, créer une bannière.
PUT, DELETE	/api/v1/admin/banners/{id}	Modifier, supprimer une bannière.
PATCH	/api/v1/admin/banners/{id}/toggle	Activer ou désactiver une bannière.
GET	/api/v1/public/banners ?position=&segment=	Bannières actives pour une position et un segment.
E-mail marketing — /api/v1/admin/email		
GET	/api/v1/admin/email/stats	Statistiques d'envoi.
POST	/api/v1/admin/email/newsletter	Envoyer une newsletter.
POST	/api/v1/admin/email/contact/{userId}	Contacter un client.

TABLE 6.2 – Services web du Sprint 7

6.4 Les cas d'utilisation du Sprint 7

6.4.1 Cas d'utilisation ń Créer un code promo ciblé ż

6.4.1.1 Description textuelle

Le tableau 6.3 présente la description textuelle de ce cas.

Acteurs	Commerçant
Objectif	Créer une offre réservée à une cible et en mesurer l'efficacité.
Pré-condition	Le commerçant est connecté et sa boutique est active.

Scénario principal	1. Le commerçant ouvre ń Promotions ż puis ń Nouveau code ż. 2. Il saisit le code (par exemple `VIP20`), le type et la valeur de la remise, et le montant minimum de commande. 3. Il définit la période, éventuellement une plage horaire, et les limites d'utilisation. 4. Il restreint le code à un segment (par exemple VIP) et, s'il le souhaite, à des catégories ou des produits. 5. Le système vérifie que le code n'existe pas déjà, calcule son statut (programmé, actif ou expiré) à partir des dates, et l'enregistre. 6. Lorsqu'un client l'utilise, le système enregistre l'utilisation et met à jour le nombre de commandes, les revenus et le taux de conversion du code.
Scénario alternatif	**Code déjà existant :** à l'étape 5, le système demande un autre code. **Limite atteinte :** à l'étape 6, si le client a déjà utilisé le code le nombre maximal de fois autorisé, le système refuse la remise avec un message explicite.

TABLE 6.3 – Description textuelle du cas ń Créer un code promo ciblé ż

6.4.1.2 Diagramme de séquence système

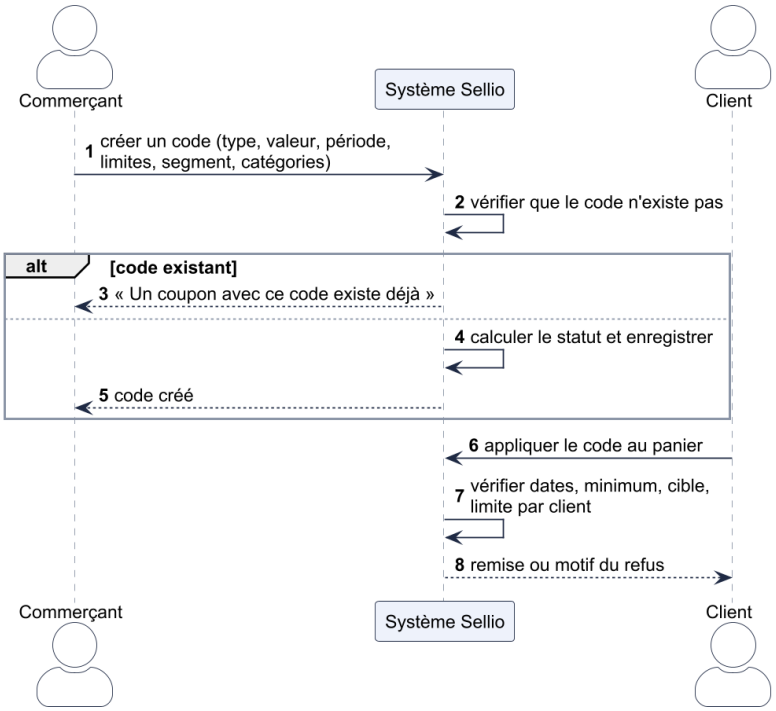


FIGURE 6.2 – Diagramme de séquence système du cas ń Créer un code promo ciblé ż

6.4.1.3 Diagramme de séquence objet

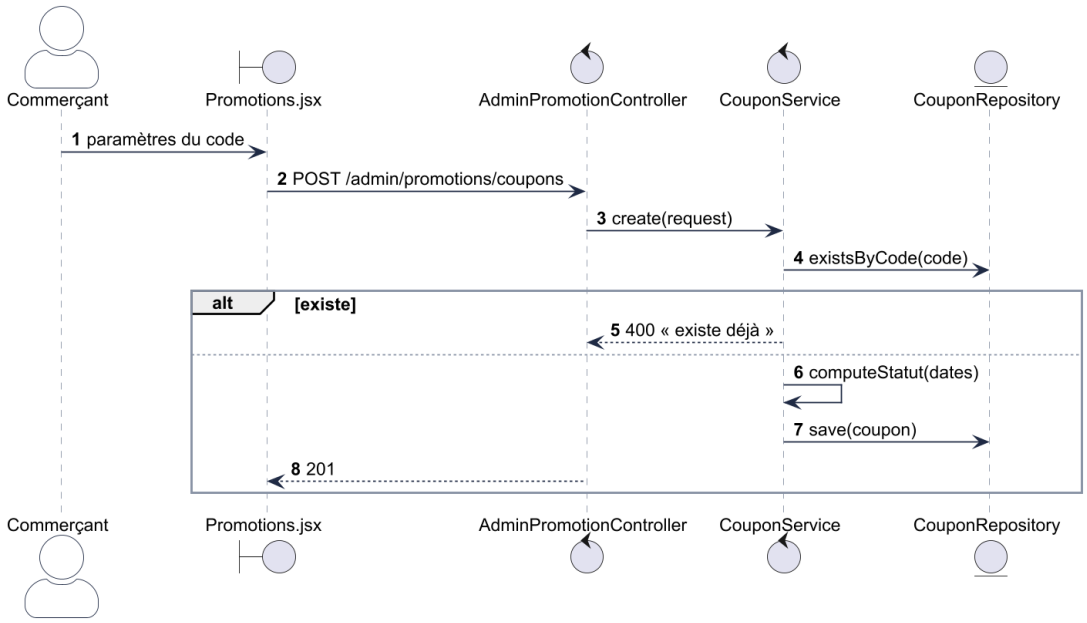


FIGURE 6.3 – Diagramme de séquence objet du cas ń Créer un code promo ciblé ż

6.5 Réalisation du Sprint 7

Promotions : la figure 6.4 présente la gestion des codes promo et des remises, avec leurs indicateurs.

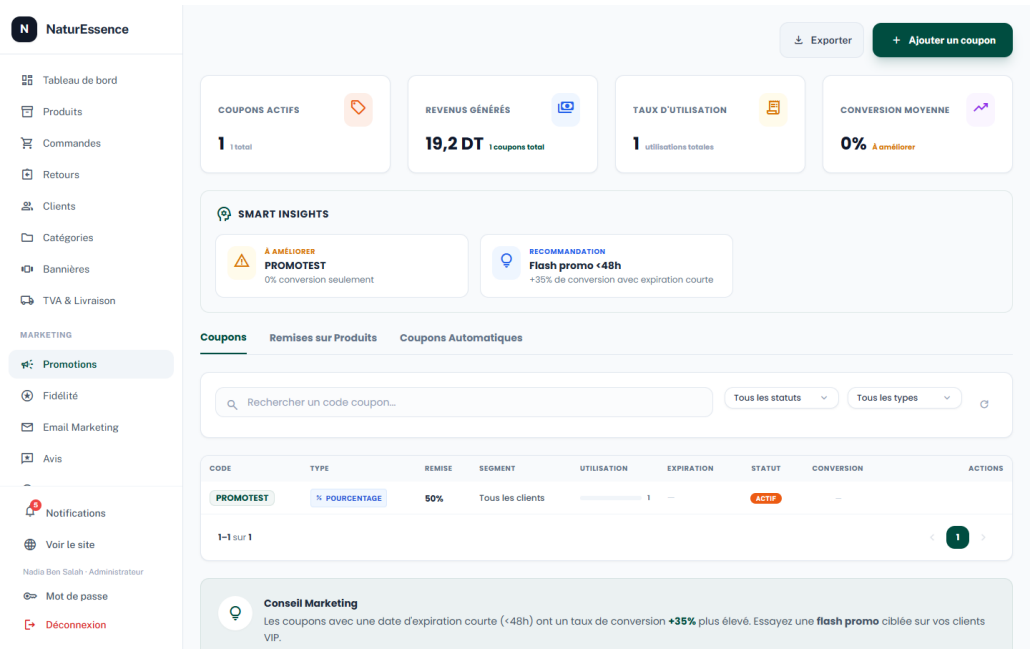


FIGURE 6.4 – Gestion des promotions

Bannières : la figure 6.5 présente le formulaire de bannière, avec aperçu sur ordinateur et sur mobile.

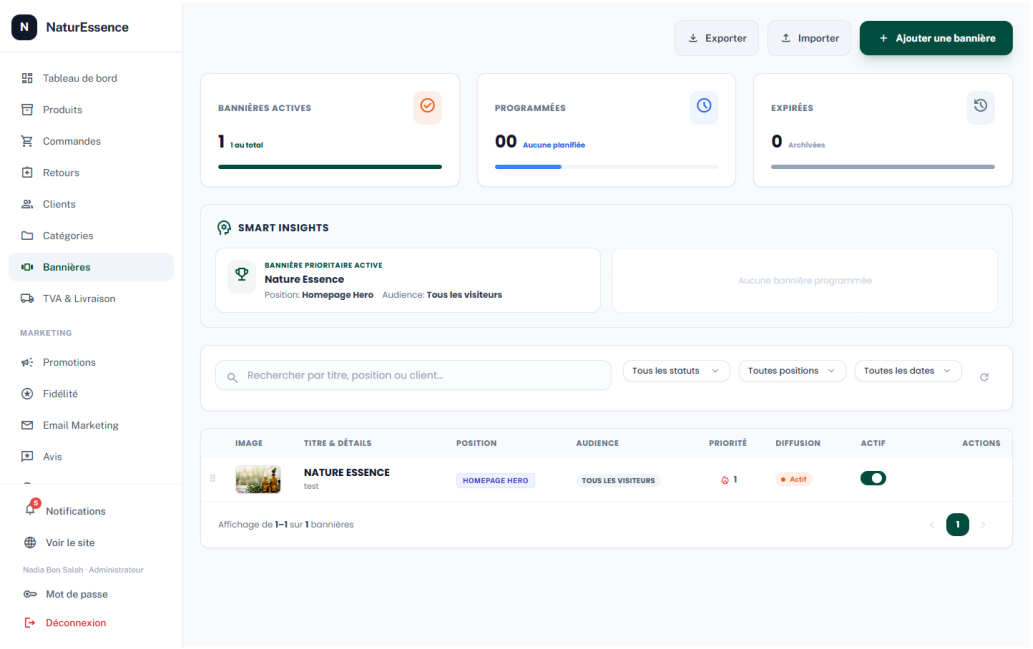


FIGURE 6.5 – Création d'une bannière ciblée

E-mail marketing : la figure 6.6 présente l'envoi d'une newsletter.

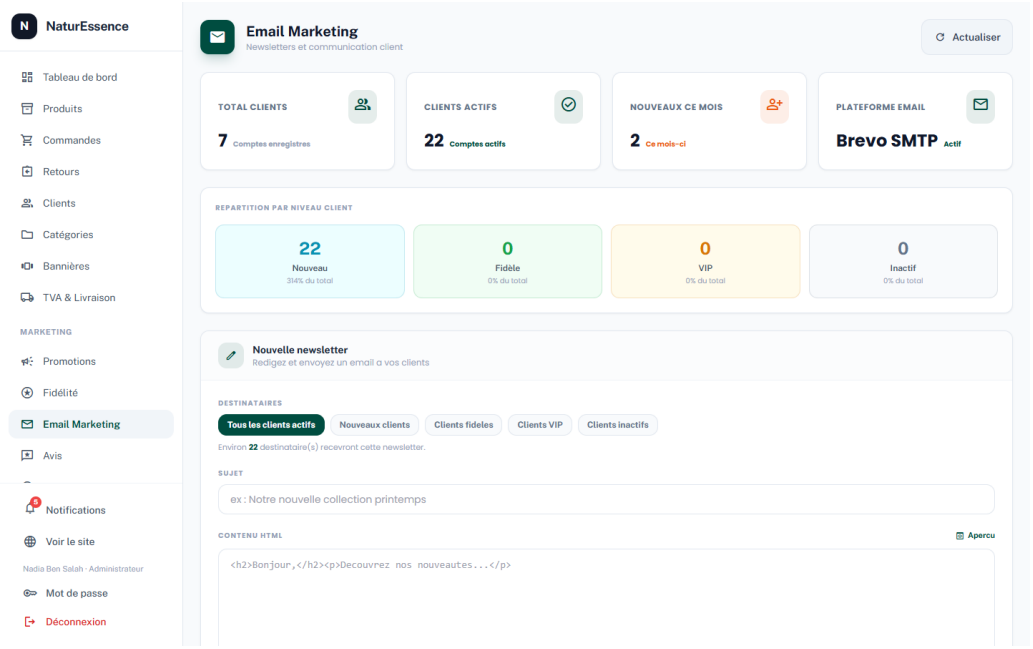


FIGURE 6.6 – E-mail marketing

6.6 Backlog du Sprint 8

Le tableau 6.4 présente le **backlog du Sprint 8**, consacré au programme de fidélité.

User Story	Tâches	Estimation
En tant que commerçant, je souhaite configurer le gain de points	Développer l'entité `LoyaltyConfig` (points par dinar, bienvenue, avis, anniversaire, promotion automatique).	1
	Attribuer les points à la livraison d'une commande, à l'inscription et à la publication d'un avis, avec le multiplicateur du niveau.	1
	Journaliser chaque mouvement de points (`PointsTransaction`).	1
	Développer l'onglet ń Configuration ż.	1
En tant que commerçant, je souhaite gérer les niveaux et ajuster les points d'un client	Développer l'API CRUD des segments (seuil de points, multiplicateur, avantages).	1
	Développer la montée de niveau automatique après chaque attribution de points.	1
	Développer l'ajustement manuel (le solde ne peut pas devenir négatif) et le classement des clients.	1
	Développer les onglets ń Avantages / Niveaux ż, ń Classement ż et ń Ajustement manuel ż.	1
	Tester la fonctionnalité.	1
En tant que client, je souhaite consulter mes points et mon niveau	Développer l'API `/profile/loyalty` (solde, niveau, historique, progression vers le niveau suivant).	1
	Afficher la carte de fidélité dans ń Mon profil ż.	1
	Total	**11**

Table 6.4 – Backlog du Sprint 8

6.7 Diagramme de cas d'utilisation du Sprint 8

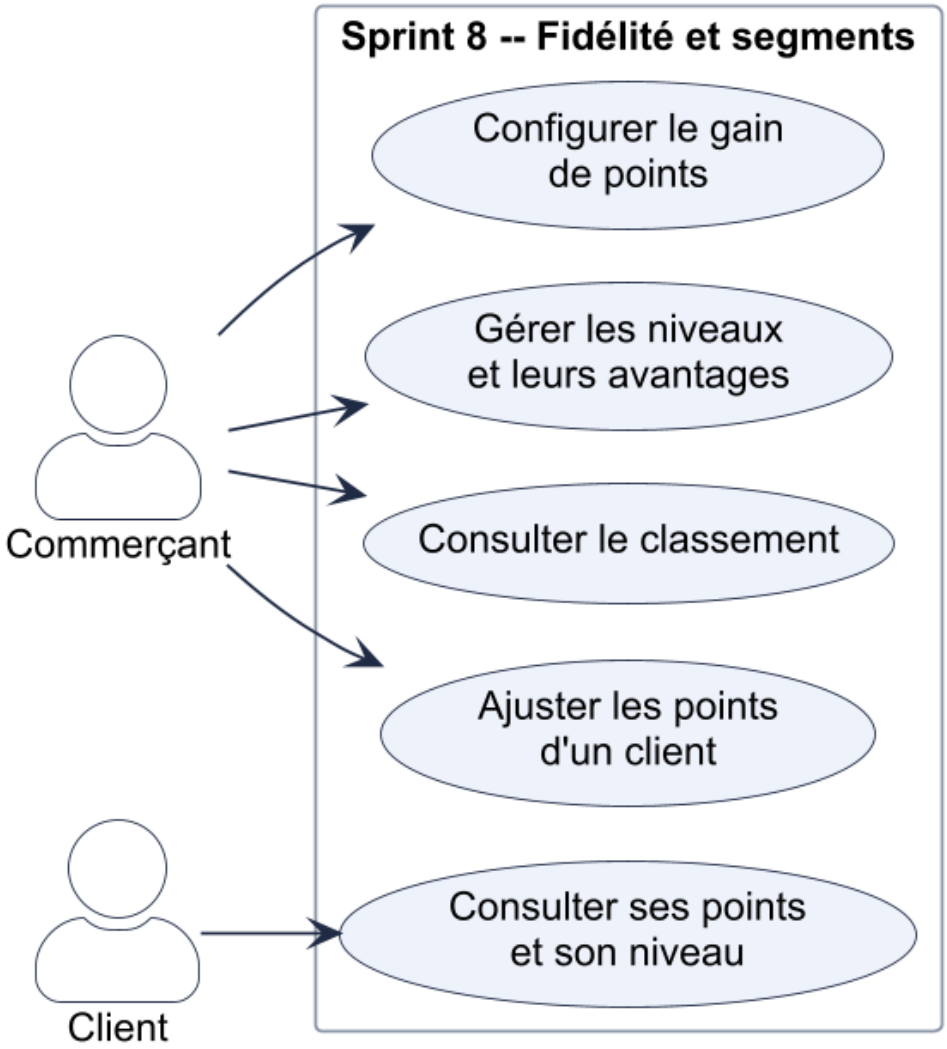


FIGURE 6.7 – Diagramme de cas d'utilisation du Sprint 8

6.8 Services web du Sprint 8

Le tableau 6.5 présente les services web du Sprint 8.

Méthode	URL	Description
Fidélité — /api/v1/admin/loyalty (auth-service)		
GET	/api/v1/admin/loyalty/config	Consulter la configuration du programme.
PUT	/api/v1/admin/loyalty/config	Modifier la configuration.
GET	/api/v1/admin/loyalty/leaderboard	Classement des clients par points.
POST	/api/v1/admin/loyalty/users/{userId}/adjust-points	Ajuster les points d'un client (positif ou négatif).
Segments — /api/v1/admin/segments		
GET, POST	/api/v1/admin/segments	Lister, créer un niveau.

Méthode	URL	Description
PUT, DELETE	/api/v1/admin/segments/{id}	Modifier, supprimer un niveau.
GET	/api/v1/admin/users/by-segment/{segmentName}	Clients d'un niveau.
Espace client		
GET	/api/v1/profile/loyalty	Solde, niveau et historique du client connecté.

Table 6.5 – Services web du Sprint 8

6.9 Les cas d'utilisation du Sprint 8

6.9.1 Cas d'utilisation ń Gagner des points et changer de niveau ż

6.9.1.1 Description textuelle

Le tableau 6.6 présente la description textuelle de ce cas.

Acteurs	Client, Commerçant
Objectif	Récompenser automatiquement les achats et faire progresser le client dans les niveaux du programme.
Pré-condition	Le client est inscrit ; le programme de fidélité est configuré.
Scénario principal	1. Le commerçant passe une commande du client au statut ń livrée ż. 2. Le système calcule les points : points = arrondi(montant × points par dinar × multiplicateur du niveau). Par exemple, 120 DT avec 1 point par dinar et un multiplicateur de 1,5 donnent 180 points. 3. Le système crédite le solde et journalise le mouvement. 4. Si la promotion automatique est active, le système cherche le niveau le plus élevé dont le seuil est atteint et y fait passer le client. 5. Le client consulte son solde, son niveau et sa progression dans ń Mon profil ż.
Scénario alternatif	**Ajustement manuel :** le commerçant retire des points ; le solde est borné à zéro. **Pas de rétrogradation automatique :** une baisse du solde ne fait pas redescendre le client ; le niveau ń Inactif ż n'est attribué que manuellement.

Table 6.6 – Description textuelle du cas ń Gagner des points et changer de niveau ż

6.9.1.2 Diagramme de séquence système

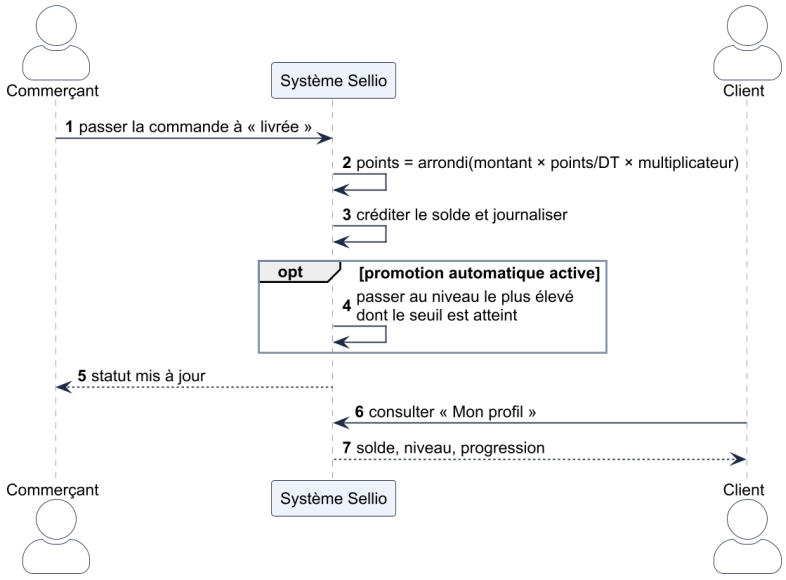


FIGURE 6.8 – Diagramme de séquence système du cas ń Gagner des points et changer de niveau ż

6.9.1.3 Diagramme de séquence objet

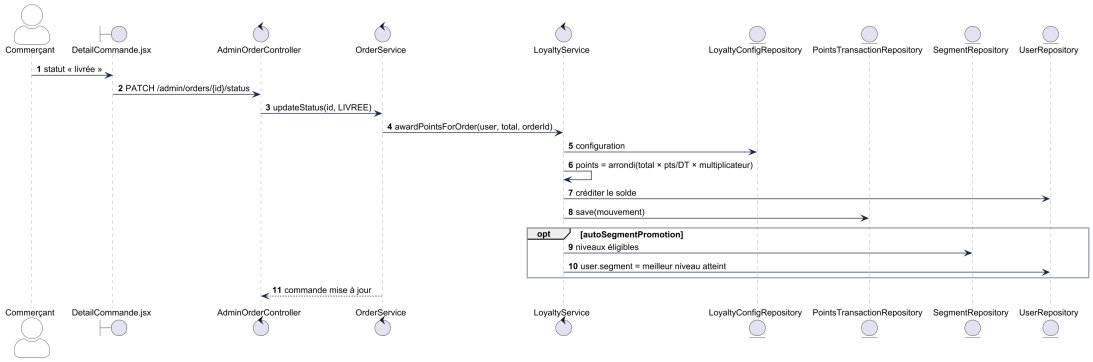


FIGURE 6.9 – Diagramme de séquence objet du cas ń Gagner des points et changer de niveau ż

6.10 Réalisation du Sprint 8

Programme de fidélité : les figures 6.10 et 6.11 présentent la configuration du programme et la gestion des niveaux (Nouveau, Fidèle, VIP, Inactif).

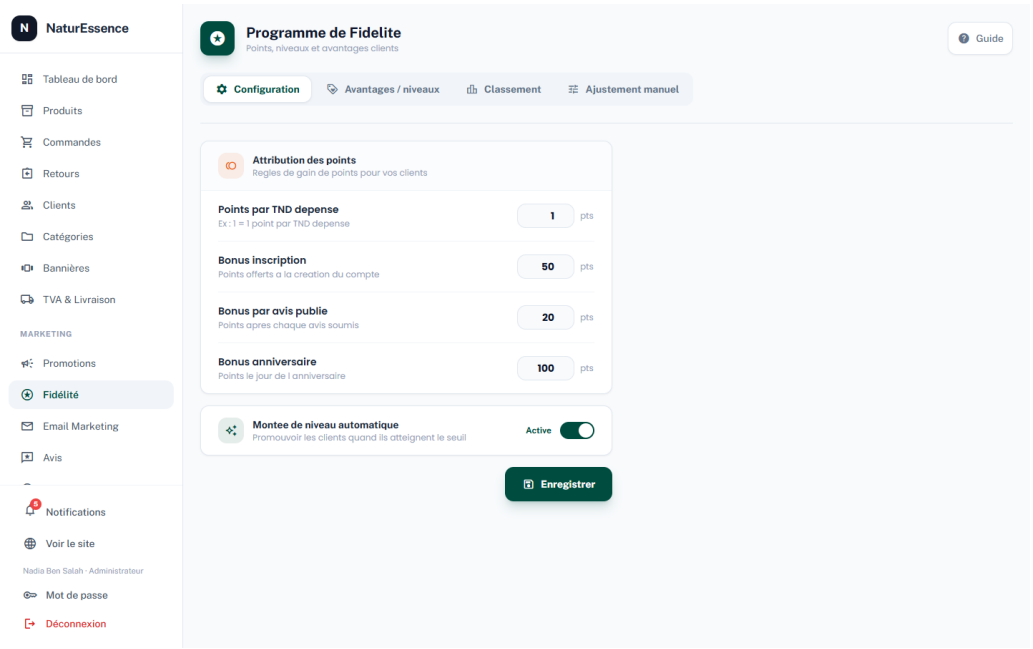


FIGURE 6.10 – Configuration du programme de fidélité

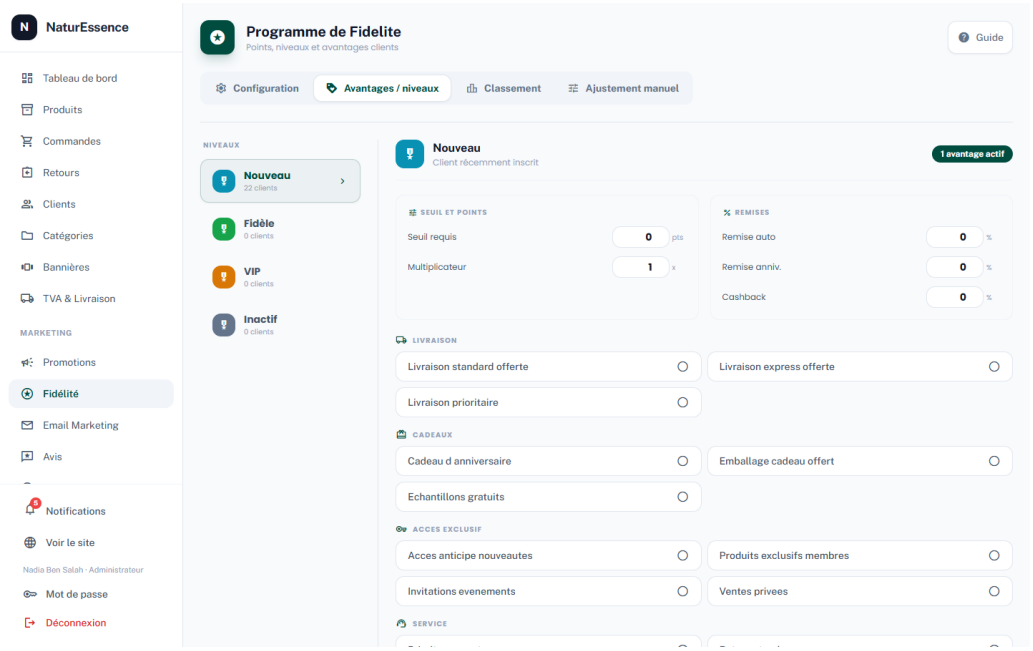


FIGURE 6.11 – Niveaux et avantages du programme de fidélité

Espace client : la figure 6.12 présente la carte de fidélité du client dans son profil.

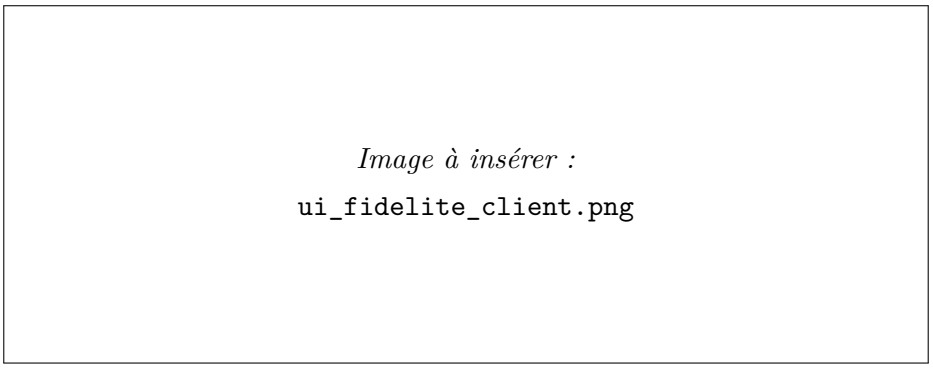


FIGURE 6.12 – Carte de fidélité dans l'espace client

Conclusion

Cette release a donné au commerçant les leviers classiques du marketing en ligne : promotions ciblées, bannières, e-mails et fidélité. Ces outils reposent toutefois sur des segments définis à la main (Nouveau, Fidèle, VIP). Les deux dernières releases vont plus loin grâce à l'intelligence artificielle : l'essayage virtuel d'abord, puis l'analyse automatique du comportement des visiteurs.

Chapitre 7

Release 5 : Essayage virtuel en temps réel par IA générative

Introduction

Dans la mode en ligne, la première source d'hésitation est simple : le client ne sait pas à quoi ressemblera le vêtement sur lui. Cette incertitude freine l'achat et multiplie les retours. Ce chapitre présente la cinquième release, qui répond à ce problème par une cabine d'essayage virtuel : depuis la fiche d'un vêtement, le client active sa caméra et se voit, en direct, portant l'article. Nous présentons d'abord l'architecture de la fonctionnalité, puis le choix du modèle et sa configuration, avant de détailler le Sprint 9.

7.1 Architecture de l'essayage virtuel

7.1.1 Flux d'architecture

La figure 7.1 présente le flux de l'essayage virtuel. Il fait intervenir trois parties : le navigateur du client, notre microservice catalogue et l'API de Decart.

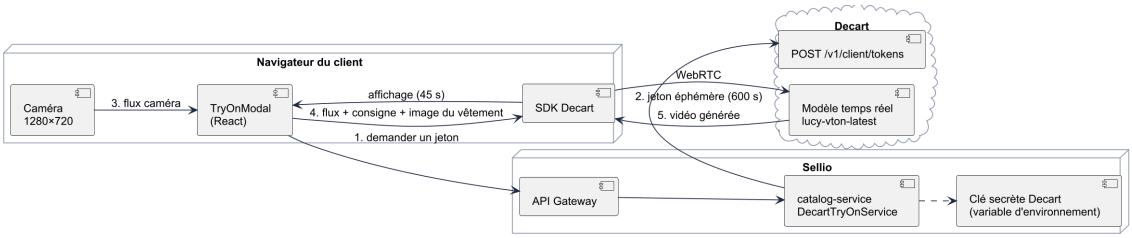


FIGURE 7.1 – Architecture de l'essayage virtuel

Le flux se déroule en quatre temps :

1. **Obtention d'un jeton :** le navigateur demande un jeton à `catalog-service` (`POST /api/v1/public/tryon/token`). Le service, seul détenteur de la clé secrète Decart, demande à Decart un **jeton client éphémère** valable dix minutes et le renvoie au navigateur. La clé secrète n'est donc jamais exposée côté client.
2. **Préparation des entrées :** le navigateur ouvre la caméra (1280 Œ 720) et télécharge l'image du vêtement, qu'il redimensionne (768 pixels au plus sur le grand côté) et compresse en JPEG pour limiter la latence.

3. **Connexion temps réel :** le SDK Decart ouvre une connexion **WebRTC** vers le modèle *Lucy VTON*. Le flux caméra part vers Decart ; l'image du vêtement et une consigne textuelle sont envoyées sur la même session.
4. **Restitution :** Decart renvoie un flux vidéo dans lequel le haut porté par le client est remplacé par le vêtement ; le navigateur l'affiche à côté de l'image de la caméra.

7.1.2 Couche de présentation

Le composant React `TryOnModal` gère l'ensemble de la session : états successifs (ń Jeton Decart ż, ń Caméra ż, ń Connexion au modèle ż, ń Essayage en direct ż), affichage des deux vidéos, compte à rebours de la session et messages d'erreur compréhensibles. Le bouton ń Essayer virtuellement ż n'apparaît que sur les fiches des boutiques du secteur vêtements.

7.1.3 Couche d'intermédiation

Le service `DecartTryOnService` du microservice catalogue appelle `POST https://api.decart.ai/v` avec la clé secrète lue dans une variable d'environnement, et renvoie au navigateur le jeton, sa date d'expiration et la durée de session autorisée. Si la clé n'est pas configurée, il répond `503`, et en cas d'échec de Decart, `502`, ce qui permet à l'interface d'afficher un message adapté.

7.1.4 Couche de génération

La génération est entièrement réalisée par Decart. Le modèle *Lucy VTON* est un modèle vidéo génératif temps réel : il transforme chaque image du flux caméra en conservant la personne, sa posture et l'arrière-plan, et en remplaçant le vêtement ciblé par celui de l'image de référence.

7.2 Mise en uvre de l'essayage virtuel

7.2.1 Critères de choix

Le choix de la technologie a été guidé par cinq critères :

— **Réalisme :** le rendu doit être crédible (plis, éclairage, morphologie), faute de quoi il dessert la vente ;

— **Interactivité :** un essayage en direct, où le client bouge et se tourne, est bien plus convaincant qu'une photo retouchée ;

— **Infrastructure :** la plateforme ne dispose pas de serveur GPU ;

— **Intégration :** la solution doit s'intégrer à une application web, sans installation côté client ;

— **Coût et sécurité :** coût maîtrisé par session et clé secrète jamais exposée.

7.2.2 Comparaison des solutions candidates

Le tableau 7.1 compare les quatre familles de solutions étudiées.

Solution	Avantages	Contraintes	Décision
Superposition 2D par détection de pose (MediaPipe [30])	Gratuit, exécuté dans le navigateur, temps réel.	Rendu peu réaliste : le vêtement est ń collé ż sur l'image, sans plis ni occlusions ; image détourée à préparer pour chaque produit.	Écartée (réalisme insuffisant).
Modèles de diffusion open source auto-hébergés (IDM-VTON [31], OOTDiffusion [32])	Très bon réalisme sur photo ; aucun coût de licence ; données maîtrisées.	Nécessitent un GPU puissant ; plusieurs secondes par image ; photo statique, pas de vidéo.	Écartée (pas de GPU, pas de temps réel).
API d'essayage image par image hébergées (par exemple FASHN [33])	Bon réalisme ; aucune infrastructure ; intégration simple par API REST.	Photo statique : le client doit téléverser une photo et attendre le résultat ; facturation à l'image.	Écartée (expérience moins interactive).
Decart *Lucy VTON* temps réel [26]	Vidéo en direct via WebRTC ; aucune infrastructure ; SDK JavaScript ; jetons clients éphémères.	Facturation à la seconde de génération ; dépendance à un fournisseur ; WebRTC bloqué par certains VPN ou réseaux.	**Retenue.**

Table 7.1 – Comparaison des solutions d'essayage virtuel

7.2.3 Solution retenue

Nous avons retenu **Decart** et son modèle ***Lucy VTON***. C'est la seule option qui combine un rendu génératif réaliste et un essayage **en direct**, sans serveur GPU de notre côté. Le mécanisme de jetons clients éphémères répond en outre au critère de sécurité : la clé secrète reste sur notre serveur. Ses contraintes, coût à la seconde et dépendance au fournisseur, ont été encadrées par la configuration décrite ci-dessous.

7.2.4 Configuration du modèle

Le tableau 7.2 résume la configuration retenue et sa justification.

Paramètre	Valeur	Justification
Modèle	`lucy-vton-latest`	Modèle d'essayage virtuel temps réel de Decart.
Résolution de la caméra	1280 Œ 720	Résolution attendue par le modèle ; lue dans sa description pour s'y adapter.

Paramètre	Valeur	Justification
Image du vêtement	JPEG, 768 px max., qualité 0,85	Réduit le volume envoyé sans perte visible de détails.
Consigne (*prompt*)	ń Substitute the current top with a *<nom du produit>*, matching the color, knit, round neck and long sleeves of the reference garment image ż	Indique au modèle quelle pièce remplacer et quelles caractéristiques respecter ; le nom du produit est inséré automatiquement.
Amélioration du prompt	désactivée	Garder une consigne prévisible et identique d'un produit à l'autre.
Durée d'une session	45 secondes	Plafonne le coût par essai ; le compte à rebours ne démarre qu'à la première image générée.
Validité du jeton client	600 secondes	Suffisant pour une session, inutilisable au-delà.
Délai de détection d'absence de vidéo	20 secondes	Au-delà, un message explique la cause probable (VPN bloquant WebRTC, crédits épuisés).

TABLE 7.2 – Configuration de l'essayage virtuel

7.2.5 Gestion des erreurs

Une fonctionnalité qui dépend d'une caméra, d'un réseau temps réel et d'un service externe peut échouer de nombreuses manières. Chaque cause est traduite en un message clair : caméra refusée, crédits Decart épuisés, image refusée par la modération, session coupée pour violation de politique, ou absence de vidéo (souvent due à un VPN, WebRTC ayant besoin d'UDP). Un problème rencontré pendant le développement mérite d'être signalé : un module interne du SDK, chargé seulement de mesurer la latence, faisait échouer le flux vidéo avec le serveur de développement Vite ; il a été neutralisé par un module de substitution (*stub*) sans effet sur la génération.

7.3 Backlog du Sprint 9

Le tableau 7.3 présente le **backlog du Sprint 9**.

User Story	Tâches	Estimation
En tant que visiteur, je souhaite essayer un vêtement en direct avec ma caméra	Étudier et comparer les solutions d'essayage virtuel.	1
	Ouvrir la caméra à la résolution du modèle et gérer le refus d'autorisation.	1
	Préparer l'image du vêtement (téléchargement, redimensionnement, compression).	1
	Connecter le SDK Decart en WebRTC et afficher le flux généré.	1
	Construire la consigne à partir du nom du produit.	1
	Limiter la session à 45 secondes avec compte à rebours et fermeture automatique.	1
	Traduire les erreurs (caméra, crédits, modération, réseau) en messages clairs.	1
	Afficher le bouton ń Essayer virtuellement ż sur les fiches des boutiques de mode et de sport.	1
	Tester sur plusieurs vêtements et conditions d'éclairage.	1
En tant que système, je veux délivrer des jetons Decart à durée limitée sans exposer la clé	Développer `DecartTryOnService` et l'API `/api/v1/public/tryon/token`.	1
	Ajouter la route à la passerelle et un intermédiaire de secours dans le serveur de développement.	1
	Tester la fonctionnalité (clé absente, erreur Decart).	1
	Total	**12**

Table 7.3 – Backlog du Sprint 9

7.4 Diagramme de cas d'utilisation du Sprint 9

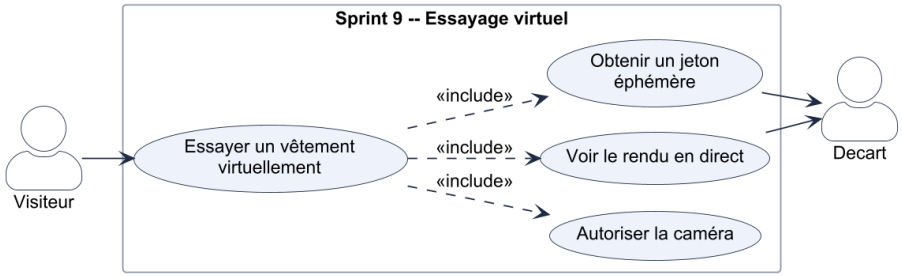


FIGURE 7.2 – Diagramme de cas d'utilisation du Sprint 9

7.5 Services web du Sprint 9

Le tableau 7.4 présente les services web du Sprint 9, y compris les appels à l'API externe de Decart.

Méthode	**URL**	**Description**
API Sellio — catalog-service		
POST	/api/v1/public/tryon/token	Obtenir un jeton client Decart éphémère, sa date d'expiration et la durée de session.
API externe Decart		
POST	https ://api.decart.ai/v1/client/tokens	Créer un jeton client (appelé par le serveur avec la clé secrète).
WebRTC	`client.realtime.connect(stream, {model})` (SDK)	Ouvrir la session temps réel et envoyer le flux caméra.
SDK	`realtime.set({prompt, image})`	Envoyer la consigne et l'image du vêtement.

TABLE 7.4 – Services web du Sprint 9

7.6 Les cas d'utilisation du Sprint 9

7.6.1 Cas d'utilisation ń Essayer un vêtement virtuellement ż

7.6.1.1 Description textuelle

Le tableau 7.5 présente la description textuelle de ce cas.

Acteurs	Visiteur ou Client, Decart
Objectif	Voir en direct le vêtement porté sur soi avant de l'acheter.
Pré-condition	La boutique est du secteur mode ou sport ; le produit a une image ; l'appareil dispose d'une caméra.

Scénario principal	1. Le visiteur clique sur ń Essayer virtuellement ż depuis la fiche produit. 2. Le système demande un jeton éphémère au service catalogue, qui l'obtient auprès de Decart. 3. Le navigateur demande l'accès à la caméra ; le visiteur l'autorise. 4. Le système prépare l'image du vêtement et ouvre la session WebRTC avec le modèle *Lucy VTON*. 5. Le système envoie la consigne et l'image du vêtement. 6. Decart renvoie la vidéo générée ; le système l'affiche et lance le compte à rebours de 45 secondes. 7. Le visiteur bouge et se tourne pour observer le rendu. 8. À la fin du compte à rebours ou à la fermeture, le système coupe la session et libère la caméra.
Scénario alternatif	**Caméra refusée :** à l'étape 3, le système affiche ń Caméra refusée. Autorisez-la dans le navigateur, puis réessayez. ż **Service indisponible :** à l'étape 2, si la clé n'est pas configurée ou si Decart échoue, le système affiche que l'essayage est indisponible. **Aucune vidéo après 20 secondes :** à l'étape 6, le système suggère de désactiver le VPN ou signale des crédits épuisés.

Table 7.5 – Description textuelle du cas ń Essayer un vêtement virtuellement ż

7.6.1.2 Diagramme de séquence système

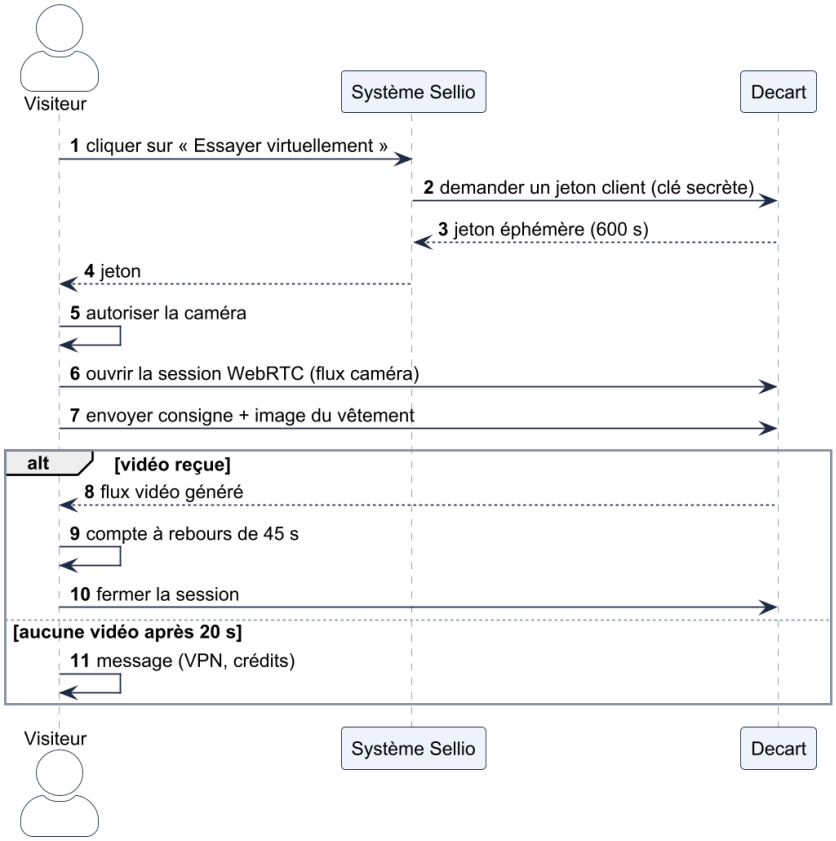


FIGURE 7.3 – Diagramme de séquence système du cas ń Essayer un vêtement virtuellement ż

7.6.1.3 Diagramme de séquence objet

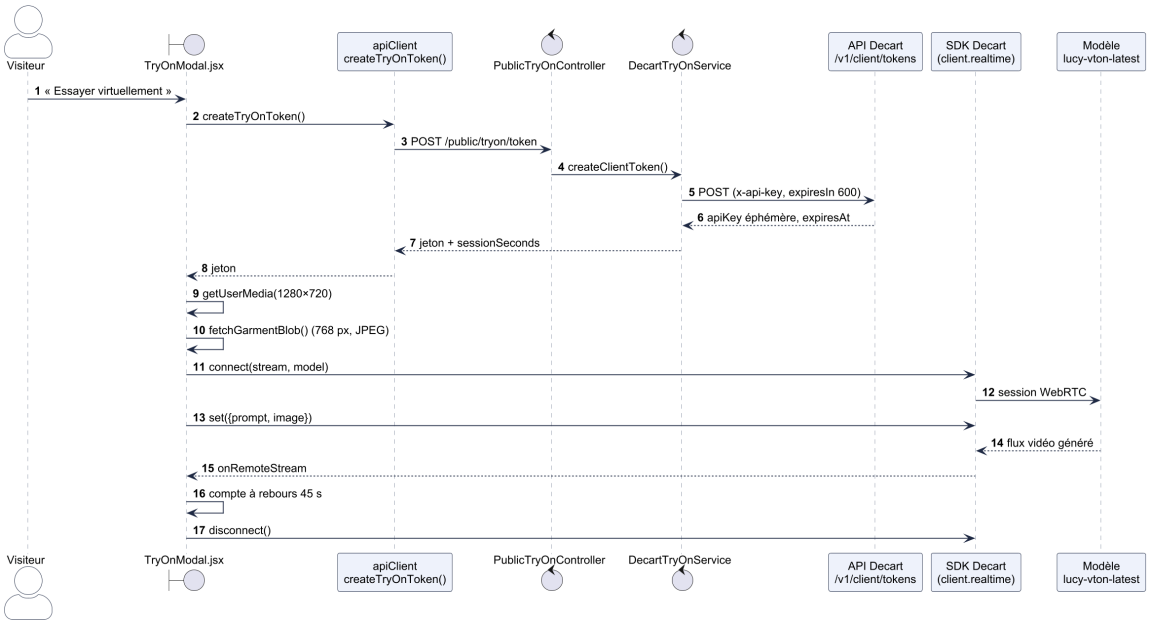


FIGURE 7.4 – Diagramme de séquence objet du cas ń Essayer un vêtement virtuellement ż

7.7 Réalisation du Sprint 9

Bouton d'essayage : la figure 7.5 présente la fiche d'un vêtement avec le bouton ń Essayer virtuellement ż.

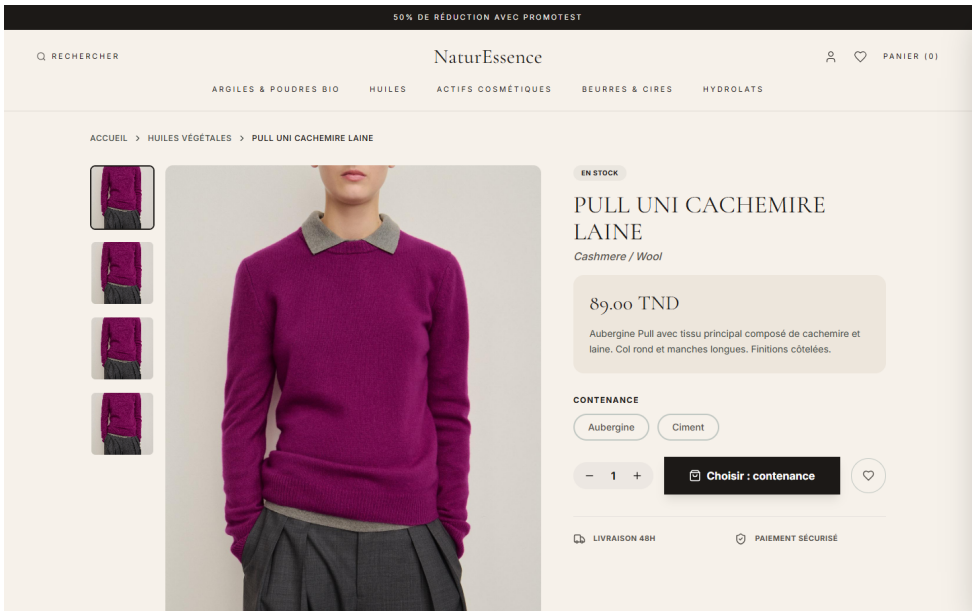


FIGURE 7.5 – Accès à l'essayage virtuel depuis la fiche produit

Cabine d'essayage : la figure 7.6 présente la cabine en fonctionnement : à gauche l'image de la caméra, à droite la vidéo générée par Decart, avec le compte à rebours.

Image à insérer :
`ui_tryon_session.png`

FIGURE 7.6 – Essayage virtuel en direct

Conclusion

Cette release a ajouté à Sellio une fonctionnalité rarement proposée par défaut par les plateformes de boutiques en ligne : un essayage virtuel en direct, fondé sur un modèle génératif temps réel et intégré de façon sécurisée grâce aux jetons éphémères. Le chapitre suivant présente l'autre volet intelligent de la plateforme : l'analyse du comportement des visiteurs par apprentissage automatique.

Chapitre 8

Release 6 : Intelligence comportementale et apprentissage automatique

Introduction

Ce dernier chapitre répond au cur du sujet : analyser les interactions des utilisateurs et exploiter ces données par apprentissage automatique pour personnaliser l'expérience d'achat. Nous présentons d'abord l'architecture de ce volet, puis les données utilisées pour entraîner et évaluer les modèles et le protocole d'évaluation. Nous détaillons ensuite les trois modèles, recommandation, segmentation et prédiction de l'attrition, avec le choix de chaque algorithme, sa configuration et ses résultats. Nous terminons par les deux sprints de la release.

8.1 Architecture de l'intelligence comportementale

8.1.1 Flux d'architecture

La figure 8.1 présente le flux complet, de la collecte d'un clic jusqu'à l'affichage d'une recommandation ou d'un score d'attrition.

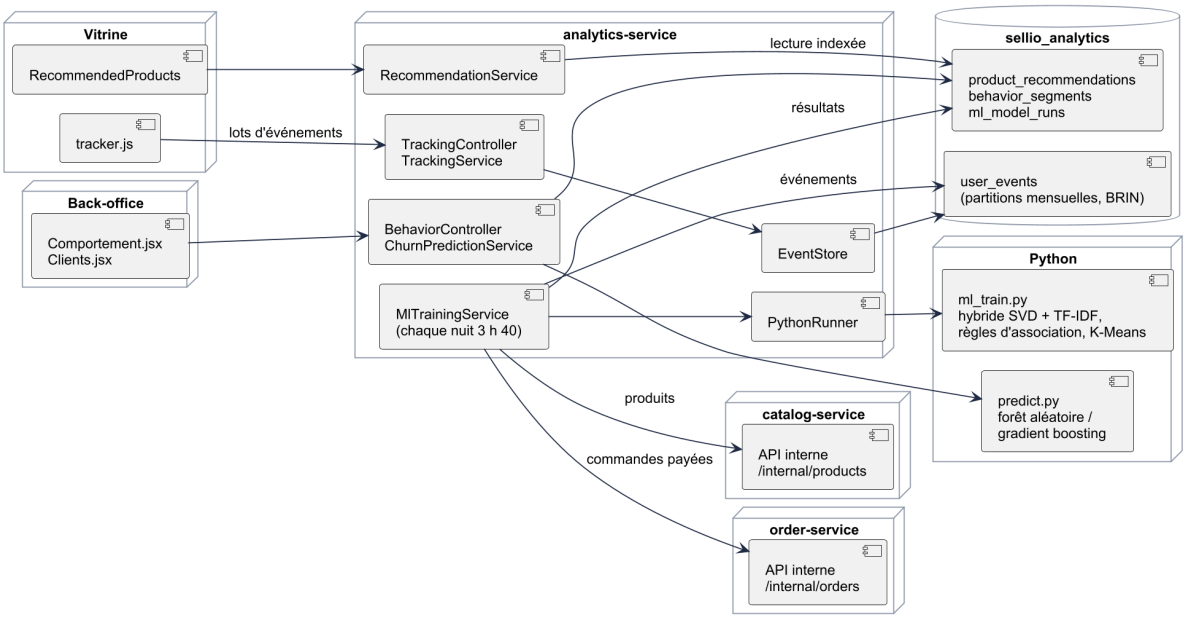


FIGURE 8.1 – Architecture de l'intelligence comportementale

8.1.2 Couche de collecte

Un module JavaScript de la vitrine (`tracker.js`) enregistre douze types d'événements : pages vues, catégories et produits consultés, clics, recherches, ajouts et retraits du panier, ajouts aux favoris, début de commande, achats et clics sur une recommandation. Chaque événement porte l'identifiant de la boutique, un **identifiant de visiteur anonyme** conservé dans le navigateur, un identifiant de session (une nouvelle session commence après 30 minutes d'inactivité) et, si le client est connecté, son identifiant. Les événements sont regroupés en lots et envoyés à `POST /api/v1/analytics/events`, pour ne pas ralentir la navigation. Le serveur n'accepte que les types connus, limite la taille des lots (50 événements) et le débit par visiteur, afin qu'un script ne puisse pas fausser les données.

8.1.3 Couche de stockage

Le volume d'événements croît beaucoup plus vite que celui des commandes. La table `user_events` est donc conçue pour ce volume :

— **partitionnement mensuel** par date : les requêtes sur une période ne lisent que les mois concernés, et la suppression d'un mois ancien est instantanée ;

— **index BRIN** sur la date : quelques kilo-octets suffisent à indexer des millions de lignes insérées dans l'ordre chronologique ;

— **index B-tree partiels** sur les seuls événements utiles aux modèles (produits consultés, achats) ;

— **agrégats quotidiens** pour le tableau de bord et **rétention de 13 mois**, suffisante pour comparer une saison à celle de l'année précédente.

8.1.4 Couche d'entraînement

Chaque nuit à 3 h 40, et à la demande du commerçant, le service `MlTrainingService` rassemble les données de chaque boutique (ses événements, dans sa propre base ; les

produits, demandés au catalogue, et les commandes payées, demandées au service de commandes, par leurs API internes), les transmet à des scripts Python (`ml_train.py`) exécutés en sous-processus, puis enregistre les résultats : voisins de chaque produit et règles d'association dans `product_recommendations`, segment de chaque visiteur dans `behavior_segments`, et métriques de chaque entraînement dans `ml_model_runs`. Les modèles d'attrition, entraînés hors ligne sur des données réelles, sont chargés par `predict.py`.

8.1.5 Couche de service et de restitution

Les recommandations sont lues dans des tables pré-calculées : servir une recommandation revient à une simple requête indexée, sans calcul au moment de l'affichage. La vitrine les affiche sur la fiche produit, sur l'accueil et dans le panier ; le back-office restitue segments, entonnoir de conversion et risques d'attrition dans la page ń Comportement ż.

8.2 Données d'entraînement et d'évaluation

8.2.1 Pourquoi des jeux de données publics réels

Une plateforme nouvelle n'a pas d'historique : ses premières boutiques ne comptent que quelques centaines d'événements, trop peu pour mesurer la qualité d'un modèle. Générer des données artificielles aurait été une impasse : un modèle évalué sur des données fabriquées retrouve surtout les règles qui ont servi à les fabriquer, et ses scores ne disent rien de son comportement face à de vrais clients. Nous avons donc entraîné et évalué les modèles sur **trois jeux de données publics réels**, choisis pour correspondre aux deux secteurs de Sellio, et nettoyés par des scripts Python reproductibles. Le tableau 8.1 les présente.

Jeu de données	**Contenu**	**Volume brut**	**Usage dans Sellio**
Online Retail II (UCI, licence CC BY 4.0) [34]	Transactions d'un e-commerçant britannique, déc. 2009 – déc. 2011.	1 067 371 lignes de factures	Attrition (modèle général), recommandation, ń souvent achetés ensemble ż.
eCommerce Events History in Cosmetics Shop (Kaggle, REES46) [35]	Événements de navigation d'une boutique de cosmétiques en ligne (vue, panier, retrait, achat).	3 533 286 événements (décembre 2019)	Segmentation des visiteurs, recommandation (secteur cosmétiques).
H&M Personalized Fashion Recommendations (Kaggle) [36]	Achats, articles et clients d'une enseigne de mode, sept. 2018 – sept. 2020.	31,8 millions d'achats, 105 542 articles	Attrition (modèle vêtements), recommandation (secteur vêtements).

Table 8.1 – Jeux de données publics utilisés

8.2.2 Nettoyage des données

Aucun de ces jeux n'est directement exploitable : chacun contient des doublons, des valeurs manquantes, des valeurs aberrantes ou du trafic non humain. Chaque script de nettoyage produit un rapport qui détaille, étape par étape, le nombre de lignes avant et après. Le tableau 8.2 en résume les principales étapes.

Jeu	**Étape (lignes supprimées)**	**Résultat**
Online Retail II	Doublons exacts (34 335) ; factures annulées ń C ż et ajustements (19 110) ; quantité ou prix ≤ 0 (6 014) ; codes qui ne sont pas des produits : frais de port, remises, frais bancaires (4 698) ; client inconnu (226 637) ; annotations internes ń damaged ż, ń wet ż (479) ; valeurs au-delà du 99,9^{e} centile (1 507).	774 591 lignes de vente
Cosmetics	Doublons exacts, par exemple double clic (183 860) ; prix $\leq$ 0 (7 537) ; session manquante (714) ; **robots** : plus de 500 événements par jour ou 60 par minute, 136 comptes (78 998) ; prix aberrant par rapport à la médiane du produit (696) ; échantillon aléatoire de 60 000 visiteurs pour tenir en mémoire.	525 253 événements, 60 000 visiteurs
H&M	Échantillon d'un client sur seize ; achats en magasin retirés, Sellio étant une boutique en ligne (588 289) ; prix au-delà du 99,9^{e} centile (1 394) ; achats identiques du même jour regroupés en une ligne avec quantité (167 798). Traduction des articles en fiches Sellio (genre, couleur, tissu) et mise à l'échelle des prix anonymisés (article médian $\approx$ 50 DT).	1 224 099 achats, 68 874 clients, 71 735 articles

Table 8.2 – Principales étapes de nettoyage

La traduction du jeu H&M vers le modèle de données de Sellio a aussi servi de test d'adéquation : la couleur a pu être renseignée pour 98 % des articles et le tissu pour 80 %, ce qui confirme que les attributs définis au Sprint 3 couvrent les données d'une vraie enseigne de mode.

8.2.3 Protocole d'évaluation

Deux règles ont guidé toutes les évaluations :

— **Découpage temporel :** un modèle est toujours entraîné sur le passé et évalué sur la période qui suit, comme en production. Pour l'attrition, trois dates d'observation T1, T2 et T3 sont utilisées : entraînement à T1, choix du modèle et des réglages à T2, puis évaluation finale une seule fois à T3, le modèle retenu étant ré-entraîné sur T1 et T2. Un découpage aléatoire aurait laissé le modèle ń voir le futur ż et gonflé ses scores.

— **Comparaison à une référence simple :** chaque modèle est comparé à une règle naïve (produits les plus populaires, ou ancienneté du dernier achat pour l'attrition).

Un modèle qui ne bat pas sa référence n'apporte rien et n'est pas retenu.

Le tableau 8.3 définit les métriques utilisées.

Métrique	**Signification**
ROC-AUC	Probabilité qu'un client parti reçoive un score plus élevé qu'un client resté (0,5 = hasard, 1 = parfait).
Précision	Parmi les clients signalés ń à risque ż, part de ceux qui partent vraiment.
Rappel	Parmi les clients qui partent vraiment, part de ceux que le modèle a signalés.
F1	Moyenne harmonique de la précision et du rappel.
HR@10 (*hit rate*)	Part des cas où au moins un produit réellement acheté ensuite figure dans les 10 produits recommandés.
NDCG@10	Comme HR@10, mais récompense davantage un bon produit placé en haut de la liste.
Couverture	Part du catalogue qui apparaît au moins une fois dans les recommandations.
Silhouette	Qualité d'un regroupement : proximité d'un visiteur avec son groupe comparée au groupe voisin (de -1 à 1).
Davies-Bouldin	Rapport entre dispersion interne et séparation des groupes (plus bas = mieux).
ARI	Accord entre deux regroupements ; utilisé pour mesurer la stabilité des segments d'un entraînement à l'autre (1 = identiques).

Table 8.3 – Métriques d'évaluation

8.3 Moteur de recommandation

8.3.1 Choix du modèle

Le moteur doit fonctionner pour des boutiques de toutes tailles, y compris une boutique qui vient d'ouvrir (problème du ń démarrage à froid ż), et produire des recommandations rapides à servir. Le tableau 8.4 compare les approches candidates.

Approche	**Avantages**	**Contraintes**	**Décision**
Popularité	Simple, robuste, forte sur les produits saisonniers.	Identique pour tous : aucune personnalisation.	Référence, et composante du modèle retenu.
Filtrage par contenu (TF-IDF)	Fonctionne dès le premier jour, à partir des fiches produit.	Recommande des produits trop semblables ; ignore les goûts réels.	Composante du modèle retenu.

Approche	Avantages	Contraintes	Décision
Filtrage collaboratif (factorisation SVD)	Apprend des comportements réels ; découvre des associations inattendues.	Inutilisable pour un produit sans interactions.	Composante du modèle retenu.
Réseaux de neurones (modèles séquentiels)	Très performants sur de grands volumes.	Exigent beaucoup de données et un GPU ; peu explicables.	Écartée.
Hybride pondéré	Combine les forces des trois premières ; s'adapte au volume de données de chaque produit.	Deux paramètres à régler par boutique.	**Retenue.**

TABLE 8.4 – Comparaison des approches de recommandation

8.3.2 Modèle retenu et configuration

Le modèle hybride se construit en quatre étapes :

1. **Signal implicite pondéré :** chaque événement reçoit un poids selon l'intérêt qu'il traduit (vue 1, clic depuis une recherche 1,5, favori 3, ajout au panier et début de commande 4, achat 8), atténué avec le temps (demi-vie de 30 jours).
2. **Similarité collaborative :** la matrice visiteurs Œ produits est pondérée à la manière de l'IDF en recherche documentaire, pour éviter que les produits vus par tout le monde ne dominent, puis factorisée par **SVD tronquée** (64 à 128 facteurs latents). La similarité cosinus entre les vecteurs des produits donne la similarité collaborative.
3. **Similarité de contenu :** les fiches produit (nom, catégorie, attributs) sont vectorisées par **TF-IDF**.
4. **Mélange adaptatif :** pour deux produits i et j, avec n le plus petit nombre d'interactions des deux,

$$\mathrm{sim}(i,j) = \beta \cdot \mathrm{sim}_{\mathrm{CF}}(i,j) + (1-\beta) \cdot \mathrm{sim}_{\mathrm{contenu}}(i,j), \qquad \beta = \frac{n}{n+\lambda}.$$

   Un produit riche en historique est recommandé d'après les comportements ; un produit nouveau, d'après sa description.

Au moment de servir, le score personnel d'un produit est la somme de ses similarités avec les produits que le visiteur a consultés, pondérées par l'intérêt ; il est ensuite mélangé à la popularité :

$$\mathrm{score} = (1-\alpha) \cdot \mathrm{personnel} + \alpha \cdot \mathrm{popularité}.$$

Les paramètres $\lambda \in \{10, 30, 100, 300, 1000\}$ et $\alpha \in \{0; 0{,}2; 0{,}4; 0{,}6; 0{,}8\}$ sont **réglés automatiquement pour chaque boutique** en cachant, pour chaque visiteur, son dernier produit fortement signalé (ajout au panier ou achat) et en retenant les valeurs qui maximisent le NDCG@10 ; sans au moins 30 cas, les valeurs par défaut ($\alpha = 0{,}4$) s'appliquent.

Pour la section ń souvent achetés ensemble ż, nous utilisons des **règles d'association** $A \to B$ extraites des paniers, classées par *lift* $= P(B \mid A)/P(B)$ et ne conservant que les paires observées au moins deux fois.

8.3.3 Résultats

La tâche évaluée est la plus exigeante : prédire les **nouveaux** produits qu'un client achètera (Online Retail II, H&M) ou le prochain produit mis au panier (Cosmetics), parmi plusieurs milliers. Le tableau 8.5 présente les résultats sur la période de test.

Modèle (HR@10 / NDCG@10)	**Online Retail II**	**Cosmetics**	**H&M**
Popularité (référence)	0,308 / 0,052	0,159 / 0,044	0,053 / 0,015
Contenu seul (TF-IDF)	0,259 / 0,044	0,069 / 0,019	0,038 / 0,014
Filtrage collaboratif seul (SVD)	0,240 / 0,044	0,084 / 0,020	0,020 / 0,008
Hybride	0,337 / 0,062	0,079 / 0,022	0,039 / 0,015
Hybride + popularité (retenu)	**0,357 / 0,069** ($\alpha = 0{,}4$)	**0,178 / 0,052** ($\alpha = 0{,}8$)	**0,059 / 0,022** ($\alpha = 0{,}4$)
Cas évalués	1 500	1 373	2 000

TABLE 8.5 – Résultats du moteur de recommandation sur la période de test

Trois constats se dégagent :

— **Le modèle retenu bat la référence sur les trois jeux**, de +16 % (Online Retail II), +12 % (Cosmetics) et +10 % (H&M) en HR@10, et de +34 %, +17 % et +41 % en NDCG@10 : il place les bons produits plus haut dans la liste.

— **La popularité est une référence difficile à battre.** Sur Cosmetics, les produits consommables rachetés par tous pèsent lourd, d'où un α élevé (0,8) choisi automatiquement ; c'est précisément pour cela qu'α est réglé par boutique plutôt que fixé.

— **Les scores absolus restent modestes**, notamment sur H&M (5,9 %), où il faut deviner un article parmi 5 000 dans un catalogue de mode très renouvelé. Ces valeurs sont cohérentes avec la difficulté de la tâche ; elles ne doivent pas être comparées aux scores de 90 % et plus que l'on obtient sur des données artificielles.

Enfin, pour ń souvent achetés ensemble ż, les règles d'association placent un produit réellement acheté dans le même panier parmi les 5 suggestions dans **47,1 %** des cas, contre 43,0 % pour la popularité, et couvrent 90 % des produits.

8.4 Segmentation des visiteurs

8.4.1 Choix du modèle

Les niveaux de fidélité du chapitre 6 sont définis à la main à partir des points. La segmentation comportementale cherche au contraire des groupes dans les données, sans étiquette préalable : c'est un problème d'apprentissage **non supervisé**. Le tableau 8.6 compare les algorithmes candidats.

Algorithme	Avantages	Contraintes	Décision
K-Means	Rapide sur des dizaines de milliers de visiteurs ; centres de groupes directement interprétables.	Nombre de groupes à choisir ; sensible aux valeurs extrêmes.	**Retenu.**
DBSCAN	Trouve des groupes de forme libre et isole le bruit.	Paramètres délicats en dimension 9 ; beaucoup de visiteurs classés ń bruit ż.	Écarté.
Mélange gaussien (GMM)	Appartenance probabiliste.	Plus lent, moins stable ; interprétation moins directe.	Écarté.
Classification hiérarchique	Visualisation en dendrogramme.	Coût quadratique en mémoire : inadapté à 60 000 visiteurs.	Écartée.

TABLE 8.6 – Comparaison des algorithmes de segmentation

8.4.2 Configuration

Chaque visiteur est décrit par neuf variables : nombre de sessions, profondeur des sessions, produits vus, part des recherches, taux d'ajout au panier, taux d'achat, taux d'ajout aux favoris, ancienneté de la dernière visite et nombre de jours actifs. Trois traitements rendent K-Means fiable sur ces données :

— **transformation logarithmique** ($\log(1+x)$) des comptages, très asymétriques ;

— **standardisation**, pour que chaque variable pèse autant ;

— **écrêtage à ± 3 écarts-types** : sans lui, quelques visiteurs extrêmes formaient à eux seuls un groupe d'une seule personne.

Le nombre de groupes K est choisi entre 3 et 6 par le meilleur score de silhouette ; $K = 2$ est exclu car deux groupes (en général ń dormants ż et ń tous les autres ż) sont trop grossiers pour agir. Chaque groupe reçoit enfin un nom et une recommandation d'action déduits de son centre (par exemple ń Abandonnistes de panier : relance panier, code livraison offerte ż).

8.4.3 Résultats

Sur les 60 000 visiteurs du jeu Cosmetics, le meilleur score de silhouette est obtenu pour $K = 3$ (0,468, contre 0,385 à 0,418 pour K de 4 à 6). L'indice de Davies-Bouldin vaut 1,26 et la **stabilité** mesurée par l'ARI moyen entre entraînements vaut 0,951 : les segments ne changent pas d'un entraînement à l'autre. Le tableau 8.7 décrit les trois segments.

Segment	Part	Profil moyen	Action suggérée
Visiteurs occasionnels	76 % (45 694)	1,1 session, 1,4 produit vu, quasiment aucun achat.	Capter l'e-mail (newsletter, première commande).
Acheteurs décidés	13 % (7 615)	1,4 session, 15 événements par session, 8 produits vus, 1 achat pour 4 ajouts au panier.	Mettre en avant les nouveautés et le réassort.
Explorateurs	11 % (6 691)	5,6 sessions, 22 produits vus sur 4 jours actifs, achètent moins que les décidés.	Recommandations et contenus inspirants.

TABLE 8.7 – Segments de visiteurs obtenus sur le jeu Cosmetics

Une silhouette de 0,47 indique des groupes nettement séparés sans l'être parfaitement, ce qui est attendu pour des comportements humains qui forment un continuum.

8.5 Prédiction de l'attrition

8.5.1 Définition du problème

Un client est considéré comme **perdu** s'il ne passe aucune commande dans les **90 jours** qui suivent la date d'observation. Le modèle estime cette probabilité pour chaque client ayant déjà acheté, à partir de six variables calculées sur son historique : ancienneté, nombre de commandes, montant total dépensé, panier moyen, fréquence d'achat et nombre de jours depuis la dernière commande. Le jeu H&M ajoute l'âge du client.

8.5.2 Choix du modèle

Le tableau 8.8 compare les modèles candidats. Tous ont été entraînés à T1, puis comparés à T2 selon l'AUC, pour plusieurs réglages (profondeur des arbres, taille minimale des feuilles, taux d'apprentissage).

Modèle	Avantages	Contraintes	Décision
Régression logistique	Simple, rapide, coefficients lisibles.	Relations linéaires uniquement (après transformation logarithmique).	Référence forte.
Forêt aléatoire	Capture les effets non linéaires et les interactions ; robuste au surapprentissage grâce à la moyenne de nombreux arbres.	Moins lisible qu'une régression ; modèle plus volumineux.	**Retenue (secteur général).**

Modèle	Avantages	Contraintes	Décision
Gradient boosting	Souvent le plus précis sur données tabulaires.	Plus sensible aux réglages.	**Retenu (secteur vêtements).**
Réseau de neurones	Très flexible.	Excessif pour six variables ; peu explicable.	Écarté.

TABLE 8.8 – Comparaison des modèles de prédiction de l'attrition

8.5.3 Résultats

Le tableau 8.9 présente les résultats sur la période de test T3, jamais utilisée pour choisir le modèle.

Modèle	AUC	Précision	Rappel	F1
Online Retail II (5 239 clients, taux d'attrition 56,5 %)				
Référence : ancienneté du dernier achat	0,762	0,844	0,375	0,519
Régression logistique	0,791	0,737	0,804	0,769
Forêt aléatoire (profondeur 6), retenue	**0,801**	0,728	**0,849**	**0,784**
H&M (64 871 clients, taux d'attrition 73,1 %)				
Référence : ancienneté du dernier achat	0,738	0,915	0,349	0,506
Modèle général appliqué tel quel	0,787	0,831	0,887	0,858
Gradient boosting sans l'âge	0,799	0,878	0,724	0,794
Gradient boosting (taux 0,05), retenu	**0,799**	**0,880**	0,718	0,791

TABLE 8.9 – Résultats de la prédiction de l'attrition sur la période de test

Ces résultats appellent plusieurs remarques :

— **Un gain réel mais mesuré.** Avec une AUC d'environ 0,80, le modèle classe correctement un client parti devant un client resté quatre fois sur cinq. Il dépasse nettement la règle naïve, surtout en rappel : sur Online Retail II, il repère 85 % des clients qui partent, contre 38 % pour la règle.

— **Des variables explicables.** L'importance par permutation montre que le nombre de jours depuis la dernière commande et la fréquence d'achat dominent sur Online Retail II ; sur H&M, c'est le nombre de commandes. Le modèle confirme donc l'intuition métier, en la quantifiant.

— **Un modèle par secteur.** Le modèle général fonctionne déjà sur la mode (AUC 0,787), ce qui montre que les comportements d'achat se ressemblent, mais le modèle entraîné sur H&M fait mieux (0,799) et, surtout, il est plus précis (88 % contre 83 %) : les alertes envoyées au commerçant sont plus fiables. Sellio choisit donc le modèle selon le secteur de la boutique.

— **L'âge n'apporte rien** (0,7988 sans, 0,7993 avec) : nous n'avons pas besoin de demander cette donnée personnelle aux clients.

Le score est enfin traduit en trois niveaux de risque, lisibles par un commerçant : **élevé** pour les 30 % de clients les plus à risque, **moyen** pour les 30 % suivants, **faible** pour les autres. Les seuils sont calculés pour chaque modèle à partir de la distribution de ses scores.

8.5.4 Limites

Ces modèles ont été entraînés sur des données européennes antérieures à 2021. Le comportement des clients tunisiens peut différer (paiement à la livraison dominant, saisonnalité du Ramadan et des soldes). Le système est conçu pour cela : la recommandation et la segmentation sont ré-entraînées chaque nuit sur les données propres de chaque boutique, et l'attrition pourra l'être dès que les boutiques auront accumulé plusieurs mois d'historique.

8.5.5 Fonctionnement d'une boutique à l'autre : les huit secteurs

Sellio héberge des boutiques de huit secteurs. Les trois briques d'apprentissage ne s'y adaptent pas de la même manière, ce que résume le tableau 8.10.

Brique	**Apprentissage**	**Données**	**Quantité minimale**
Recommandation	Non supervisé (SVD) + contenu (TF-IDF)	Celles de la boutique : 180 jours d'événements et ses commandes, chaque nuit	2 produits actifs
Segmentation	Non supervisé (K-Means)	Visiteurs de la boutique sur 90 jours, chaque nuit	8 visiteurs
Attrition	Supervisé, pré-entraîné	Jeux publics réels (Online Retail II, H&M)	1 commande par client

TABLE 8.10 – Ce qui est appris par boutique et ce qui est pré-entraîné

La recommandation et la segmentation n'utilisent **aucun jeu public en production** : elles apprennent sur la boutique elle-même, quel que soit son secteur, et les données d'une boutique ne servent jamais à une autre. Le seul élément propre au secteur est le texte utilisé par la similarité de contenu : il reprend les caractéristiques de la fiche produit (discipline et niveau en sport, marque et connectique en high-tech, matière et pierre en bijoux, allergènes et origine en épicerie…). Le réglage automatique du poids de la popularité (α) adapte en outre le moteur au rythme du secteur : une épicerie où tout le monde rachète les mêmes produits obtient un α élevé, une boutique de bijoux où chaque client a son style un α plus faible.

L'attrition, elle, demande des clients observés pendant des mois : une boutique qui ouvre n'en a pas. Le service choisit donc le modèle selon le secteur, comme le montre le tableau 8.11.

Secteur	**Modèle**	**Justification**	**Confiance**
Mode	Gradient boosting (H&M)	Même secteur	Élevée
Sport, enfants	Gradient boosting (H&M)	H&M couvre ses lignes sport et enfants ; achats saisonniers et tailles	Moyenne
Beauté	Forêt aléatoire (Online Retail II)	Petits paniers fréquents, proches d'un commerce de détail généraliste	Moyenne
Maison	Forêt aléatoire (Online Retail II)	Online Retail II vend de la décoration et des articles cadeaux	Moyenne à élevée
Bijoux, épicerie, high-tech	Forêt aléatoire (Online Retail II)	Faute de données du secteur ; l'horizon de 90 jours ne correspond pas à leur rythme d'achat	À valider

TABLE 8.11 – Modèle d'attrition utilisé selon le secteur

Trois jeux publics réels permettraient de valider les secteurs encore incertains : **Olist Brazilian E-Commerce** (plateforme multi-boutiques couvrant maison, sport, jouets, bijoux et électronique), **eCommerce behavior data from multi category store** (même producteur que le jeu cosmétiques, à dominante électronique) et **Instacart Market Basket Analysis** (épicerie). L'évolution naturelle est un horizon d'attrition par secteur, court pour l'épicerie, long pour le high-tech, puis un ré-entraînement sur les données de Sellio dès que les boutiques auront plusieurs mois d'historique.

Enfin, une boutique qui ouvre n'est jamais sans recommandations : sans donnée, la vitrine affiche les produits populaires de la catégorie puis les nouveautés ; dès deux produits, la similarité de contenu propose des produits aux fiches proches ; le filtrage collaboratif prend le relais à mesure que les visites et les achats s'accumulent, et le réglage de α et de λ commence dès trente cas de test.

8.6 Backlog du Sprint 10

Le tableau 8.12 présente le **backlog du Sprint 10**, consacré au suivi comportemental et à la recommandation.

User Story	Tâches	Estimation
En tant que système, je veux collecter les événements de navigation	Développer le module `tracker.js` : identifiants de visiteur et de session, file d'attente et envoi par lots.	1
	Instrumenter la vitrine (vues, recherches, panier, favoris, commande, achat, clics sur recommandation).	1
	Développer l'API de réception avec validation des types d'événements.	1
	Créer la table partitionnée, les index BRIN et partiels, et la gestion automatique des partitions.	1
	Développer les agrégats quotidiens et la rétention de 13 mois.	1
	Tester la fonctionnalité.	1
En tant que visiteur, je souhaite voir des produits similaires et ń pour vous ż	Nettoyer les jeux Online Retail II, Cosmetics et H&M (scripts et rapports).	1
	Développer le modèle hybride (signal pondéré, SVD, TF-IDF, mélange adaptatif).	1
	Développer le réglage automatique de λ et α par boutique.	1
	Évaluer le modèle sur les trois jeux avec découpage temporel et référence.	1
	Développer `PythonRunner` et `MlTrainingService` (entraînement nocturne et à la demande).	1
	Développer les API ń similaires ż, ń pour vous ż et ń populaires ż.	1
	Développer le composant `RecommendedProducts` sur la fiche produit et l'accueil.	1
En tant que visiteur, je souhaite voir les produits souvent achetés ensemble	Extraire les règles d'association classées par *lift* et les évaluer.	1
	Développer l'API ń achetés ensemble ż et la section du panier.	1
	Total	**15**

Table 8.12 – Backlog du Sprint 10

8.7 Diagramme de cas d'utilisation du Sprint 10

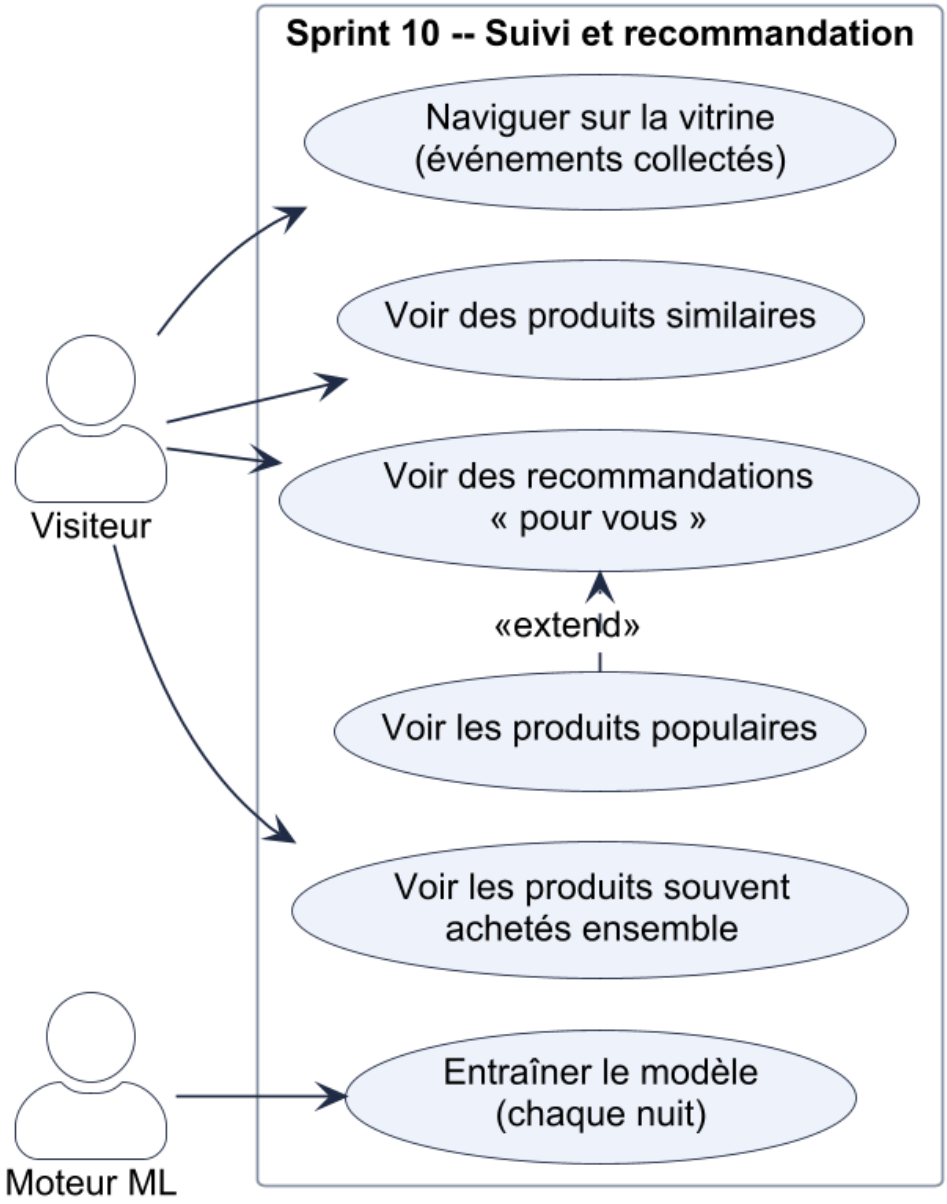


FIGURE 8.2 – Diagramme de cas d'utilisation du Sprint 10

8.8 Diagramme de classes du Sprint 10

La figure 8.3 présente les classes du microservice d'analyse impliquées dans ce sprint (`TrackingController`, `TrackingService`, `EventStore`, `MlTrainingService`, `PythonRunner`, `RecommendationService`).

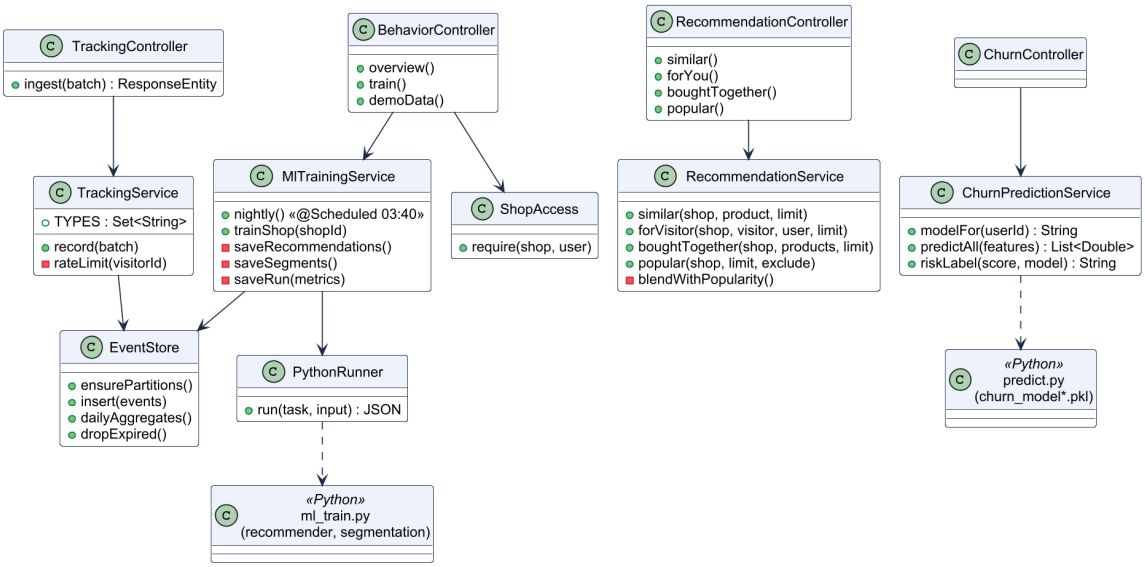


FIGURE 8.3 – Diagramme de classes du microservice d'analyse

8.9 Services web du Sprint 10

Le tableau 8.13 présente les services web du Sprint 10, exposés par `analytics-service`.

Méthode	URL	Description
Collecte — /api/v1/analytics/events		
POST	/api/v1/analytics/events	Recevoir un lot d'événements (boutique, visiteur, session).
Recommandations — /api/v1/analytics/recommendations		
GET	/api/v1/analytics/recommendations/similar	Produits similaires à un produit.
GET	/api/v1/analytics/recommendations/for-you	Recommandations personnalisées pour un visiteur.
GET	/api/v1/analytics/recommendations/bought-together	Produits souvent achetés avec ceux du panier.
GET	/api/v1/analytics/recommendations/popular	Produits populaires (repli et démarrage à froid).

TABLE 8.13 – Services web du Sprint 10

8.10 Les cas d'utilisation du Sprint 10

8.10.1 Cas d'utilisation ń Recevoir des recommandations personnalisées ż

8.10.1.1 Description textuelle

Le tableau 8.14 présente la description textuelle de ce cas.

Acteurs	Visiteur ou Client, Moteur d'apprentissage automatique
Objectif	Afficher à chaque visiteur des produits adaptés à ses centres d'intérêt.
Pré-condition	Le modèle de la boutique a été entraîné au moins une fois.
Scénario principal	1. Le visiteur consulte des produits ; chaque consultation est enregistrée par le module de suivi. 2. À l'affichage de l'accueil ou d'une fiche produit, la vitrine demande les recommandations ń pour vous ż en transmettant l'identifiant du visiteur. 3. Le système récupère l'historique récent du visiteur, pondéré par type d'événement et par ancienneté. 4. Le système lit les voisins pré-calculés de chaque produit de l'historique, en fait la somme pondérée et la mélange à la popularité avec l'α de la boutique. 5. Le système exclut les produits que le visiteur a déjà mis au panier ou achetés, et renvoie les meilleurs. 6. La vitrine affiche les produits ; un clic est enregistré comme ń clic sur recommandation ż, ce qui permet de mesurer l'efficacité du moteur en production.
Scénario alternatif	**Nouveau visiteur sans historique :** à l'étape 3, le système renvoie les produits populaires de la boutique. **Modèle pas encore entraîné :** le système se replie sur les produits populaires de la même catégorie, puis sur les nouveautés. **Nouveau produit sans interactions :** ses voisins sont calculés d'après sa description (TF-IDF), grâce au mélange adaptatif. **Service d'analyse indisponible :** la vitrine masque la section sans bloquer la navigation.

Table 8.14 – Description textuelle du cas ń Recevoir des recommandations personnalisées ż

8.10.1.2 Diagramme de séquence système

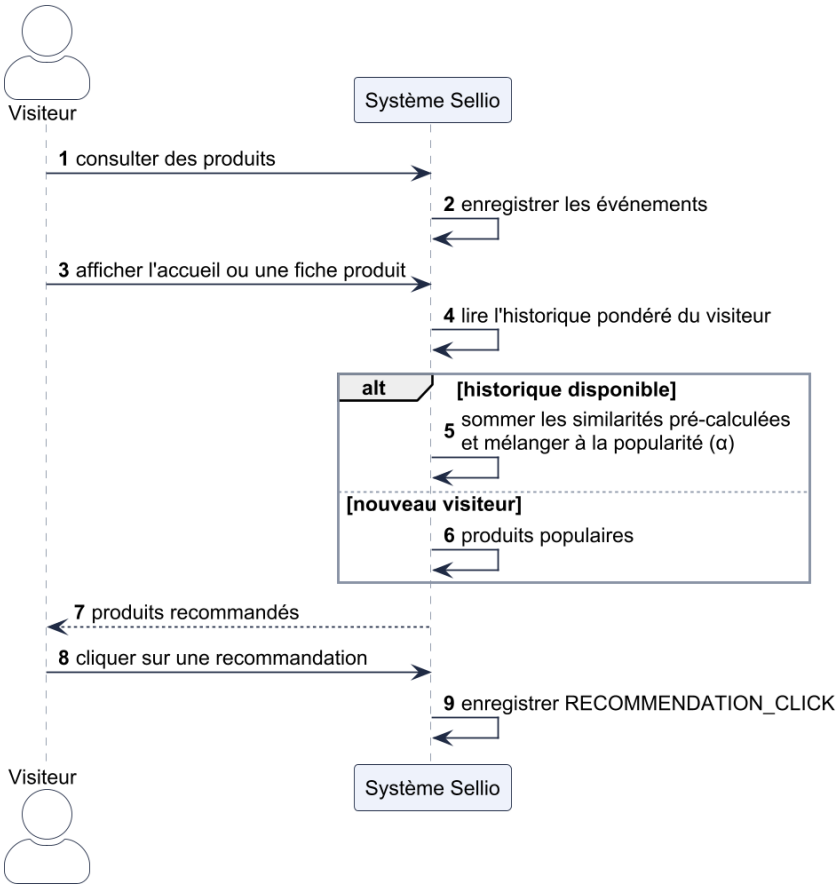


FIGURE 8.4 – Diagramme de séquence système du cas ń Recevoir des recommandations personnalisées ż

8.10.1.3 Diagramme de séquence objet

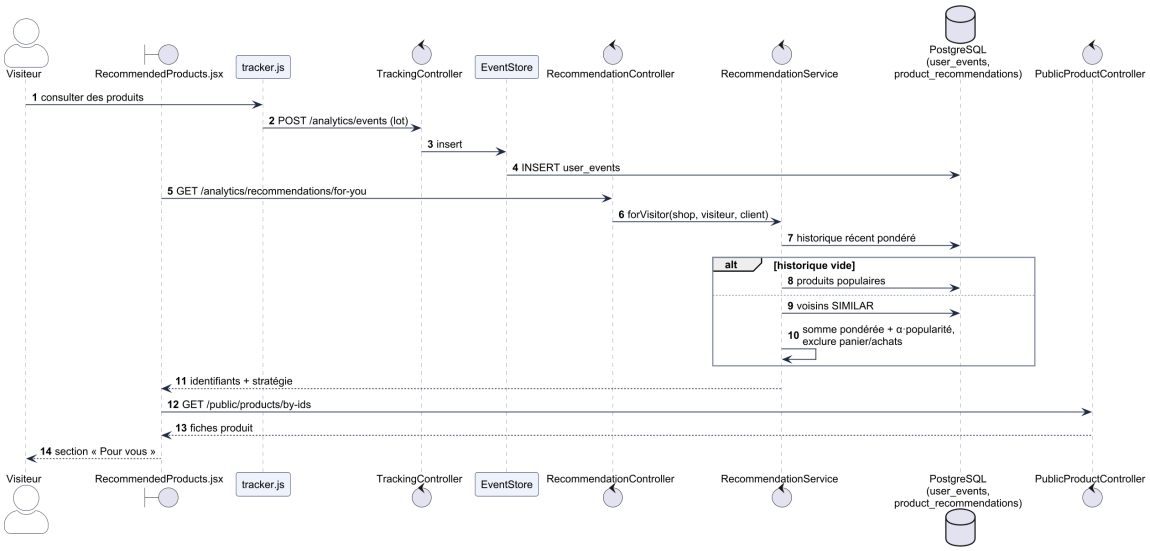


FIGURE 8.5 – Diagramme de séquence objet du cas ń Recevoir des recommandations personnalisées ż

8.11 Réalisation du Sprint 10

Recommandations sur la vitrine : les figures 8.6 et 8.7 présentent les sections ń Vous aimerez aussi ż sur la fiche produit et ń Souvent achetés ensemble ż dans le panier.

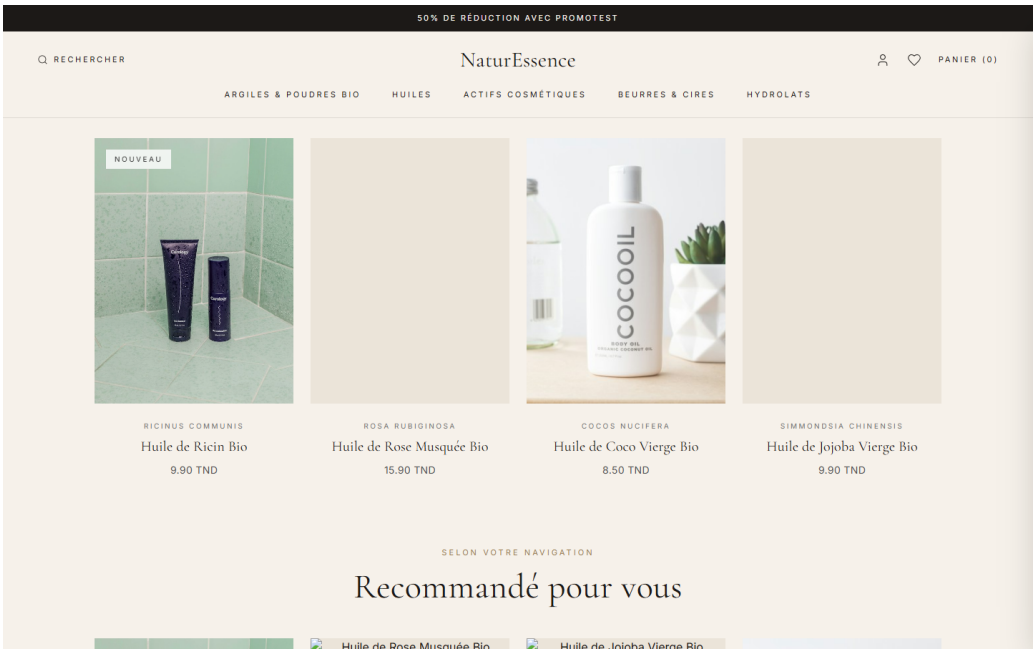


FIGURE 8.6 – Recommandations sur la fiche produit

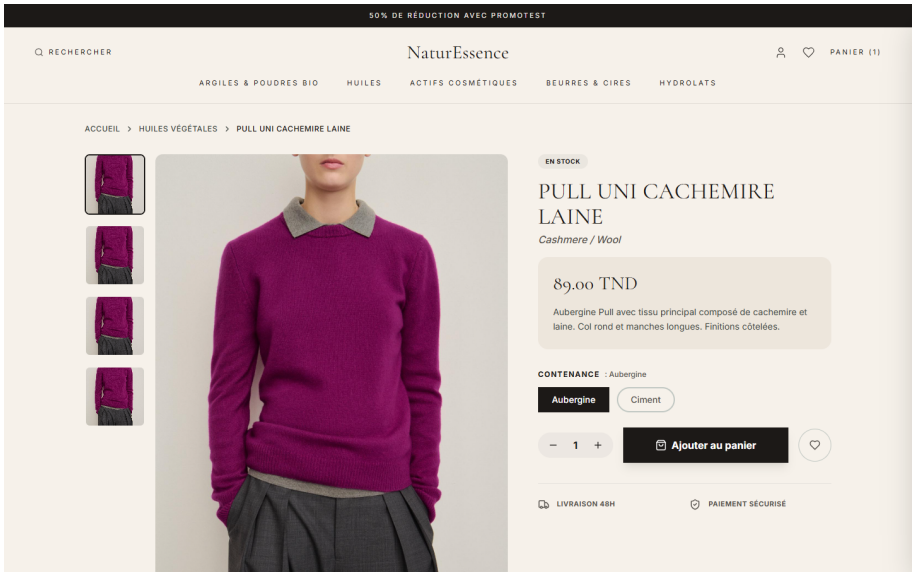


FIGURE 8.7 – Produits souvent achetés ensemble dans le panier

8.12 Backlog du Sprint 11

Le tableau 8.15 présente le **backlog du Sprint 11**.

User Story	Tâches	Estimation
En tant que commerçant, je souhaite voir les segments de mes visiteurs	Calculer les neuf variables de comportement par visiteur.	1
	Développer le K-Means avec transformation, standardisation, écrêtage et choix de K par silhouette.	1
	Nommer les segments et leur associer une action à partir des centres.	1
	Évaluer la qualité et la stabilité (silhouette, Davies-Bouldin, ARI) sur le jeu Cosmetics.	1
En tant que commerçant, je souhaite identifier les clients à risque d'attrition	Construire les instantanés T1, T2, T3 et la cible à 90 jours.	1
	Comparer régression logistique, forêts aléatoires et gradient boosting ; choisir sur T2, évaluer sur T3.	1
	Entraîner le modèle vêtements sur H&M et tester l'apport de l'âge.	1
	Calculer l'importance des variables et les seuils de risque.	1
	Développer `predict.py` et `ChurnPredictionService` (choix du modèle selon le secteur).	1
	Développer les API de score individuel et par lot.	1
En tant que commerçant, je souhaite consulter le tableau de bord ń Comportement ż et ré-entraîner les modèles	Développer l'API de synthèse (entonnoir, événements, produits consultés, segments, dernier entraînement).	1
	Développer la page ń Comportement ż du back-office.	1
	Développer le ré-entraînement à la demande et les données de démonstration (création et suppression).	1
	Restreindre l'accès aux seules données de la boutique du commerçant.	1
	Rédiger la documentation des modèles (README ML).	1
	Tester la fonctionnalité.	1

Suite à la page suivante

User Story	Tâches	Estimation
	Total	**16**

TABLE 8.15 – Backlog du Sprint 11

8.13 Diagramme de cas d'utilisation du Sprint 11

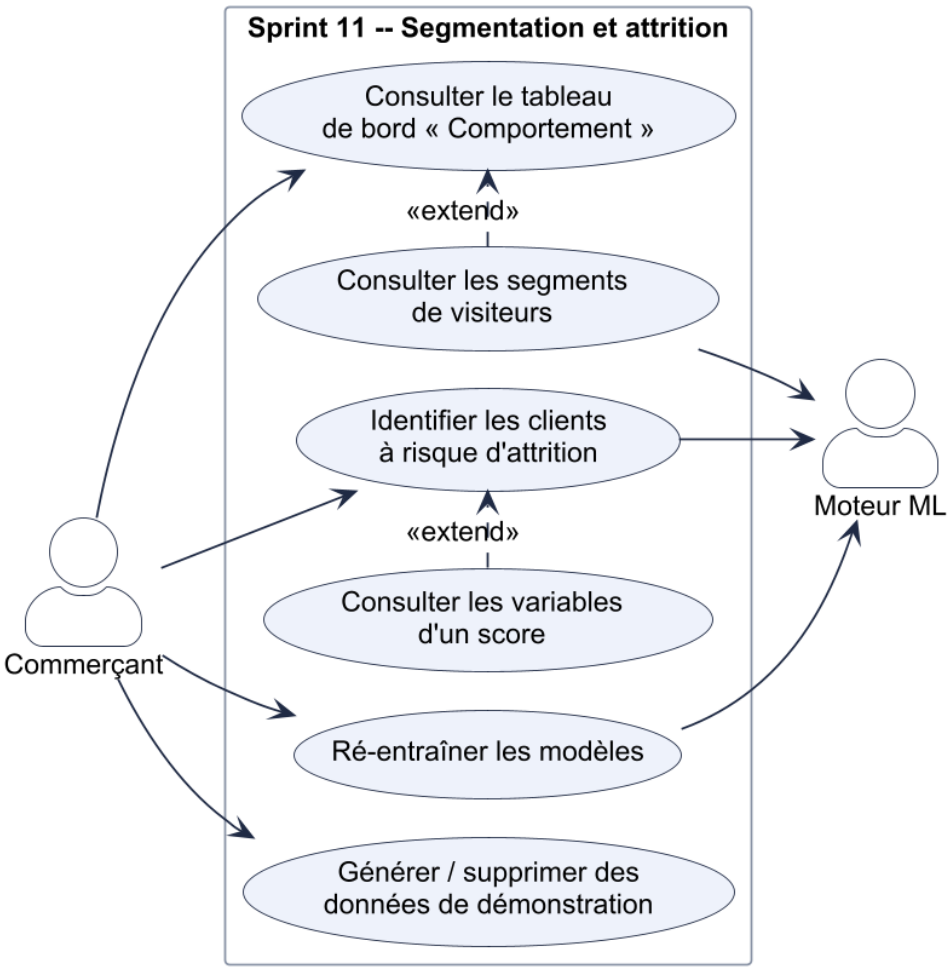


FIGURE 8.8 – Diagramme de cas d'utilisation du Sprint 11

8.14 Services web du Sprint 11

Le tableau 8.16 présente les services web du Sprint 11.

Méthode	URL	Description
Analyse comportementale — /api/v1/analytics/behavior		
GET	/api/v1/analytics/behavior/overview	Synthèse : entonnoir, événements, segments, dernier entraînement.
POST	/api/v1/analytics/behavior/train	Ré-entraîner les modèles de la boutique.

Méthode	URL	Description
POST	/api/v1/analytics/behavior/demo-data	Générer des données de démonstration.
DELETE	/api/v1/analytics/behavior/demo-data	Supprimer les données de démonstration.
Attrition — /api/v1/analytics/churn		
GET	/api/v1/analytics/churn/{userId}	Score et niveau de risque d'un client.
GET	/api/v1/analytics/churn/{userId}/features	Variables utilisées pour le score d'un client.
POST	/api/v1/analytics/churn/batch-ids	Scores d'une liste de clients.
GET	/api/v1/analytics/churn/batch	Scores de tous les clients de la boutique.

Table 8.16 – Services web du Sprint 11

8.15 Les cas d'utilisation du Sprint 11

8.15.1 Cas d'utilisation ń Consulter l'analyse comportementale et les risques d'attrition ż

8.15.1.1 Description textuelle

Le tableau 8.17 présente la description textuelle de ce cas.

Acteurs	Commerçant, Moteur d'apprentissage automatique
Objectif	Comprendre le comportement des visiteurs et agir sur les clients à risque.
Pré-condition	Le commerçant est connecté ; sa boutique est active.
Scénario principal	1. Le commerçant ouvre la page ń Comportement ż. 2. Le système vérifie que la boutique demandée est bien la sienne. 3. Le système renvoie l'entonnoir (visites, produits vus, ajouts au panier, commandes), les produits les plus consultés, la répartition des segments avec leur action suggérée et la date du dernier entraînement. 4. Le commerçant ouvre la liste des clients ; le système calcule les scores d'attrition par lot avec le modèle du secteur de la boutique et affiche pour chacun un risque élevé, moyen ou faible. 5. Le commerçant ouvre la fiche d'un client à risque élevé, consulte les variables qui expliquent son score et le contacte (e-mail, code promo ciblé). 6. S'il le souhaite, il relance l'entraînement ; le système réentraîne recommandation et segmentation et affiche les nouvelles métriques.

Scénario alternatif	**Accès à une autre boutique :** à l'étape 2, le système répond 403. **Boutique sans trafic :** pour une démonstration, le commerçant peut générer environ 60 jours de visites fictives (identifiants préfixés ń demo- ż), puis les supprimer en un clic.

TABLE 8.17 – Description textuelle du cas ń Consulter l'analyse comportementale et les risques d'attrition ż

8.15.1.2 Diagramme de séquence système

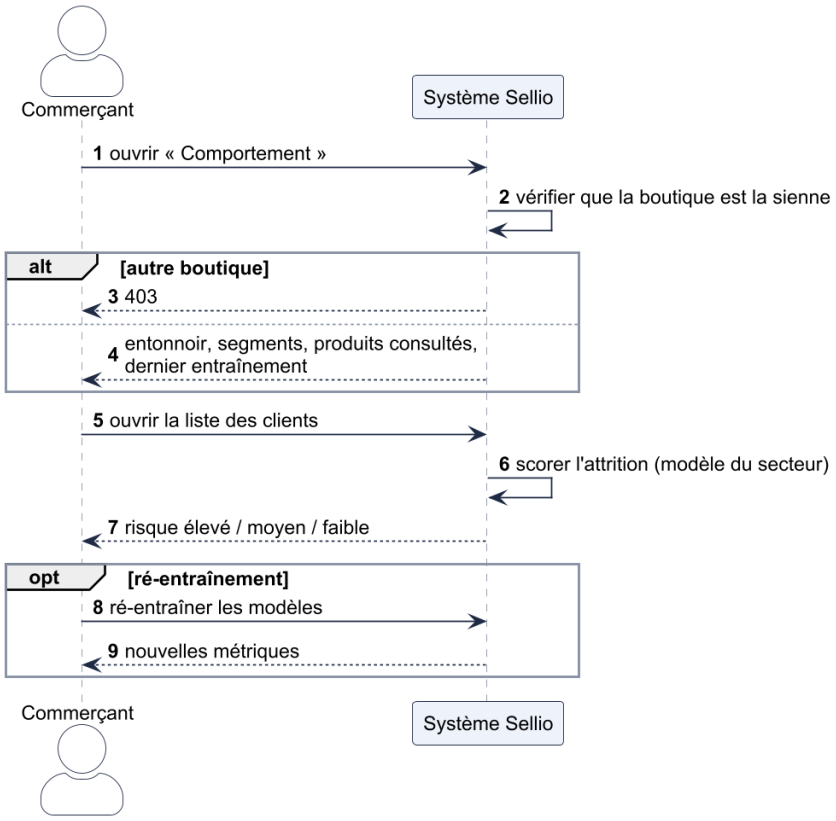


FIGURE 8.9 – Diagramme de séquence système du cas ń Consulter l'analyse comportementale ż

8.15.1.3 Diagramme de séquence objet

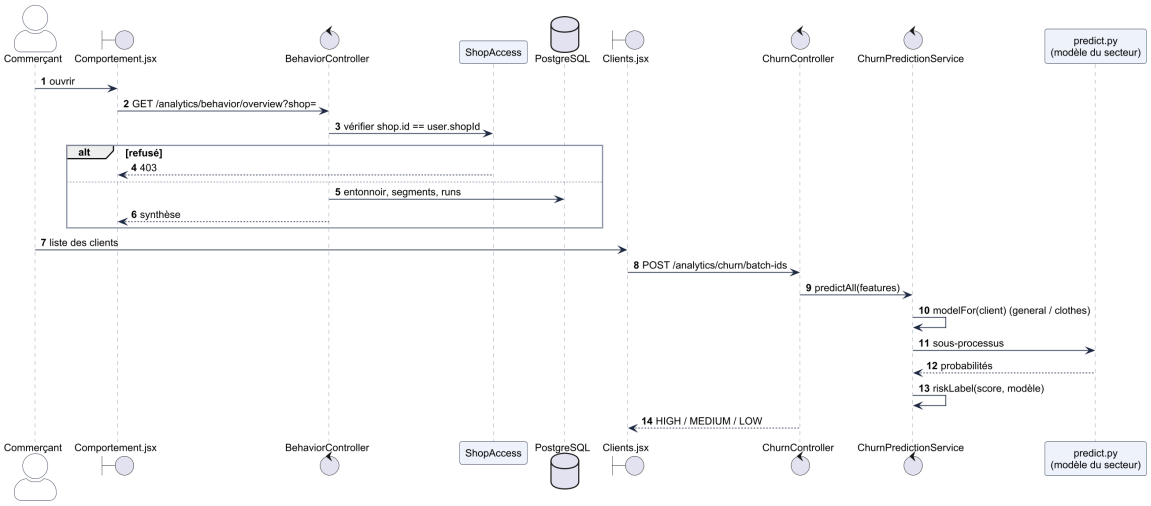


FIGURE 8.10 – Diagramme de séquence objet du cas ń Consulter l'analyse comportementale ż

8.16 Réalisation du Sprint 11

Tableau de bord ń Comportement ż : la figure 8.11 présente la page d'analyse comportementale : entonnoir de conversion, segments de visiteurs et actions suggérées, produits les plus consultés et état des modèles.

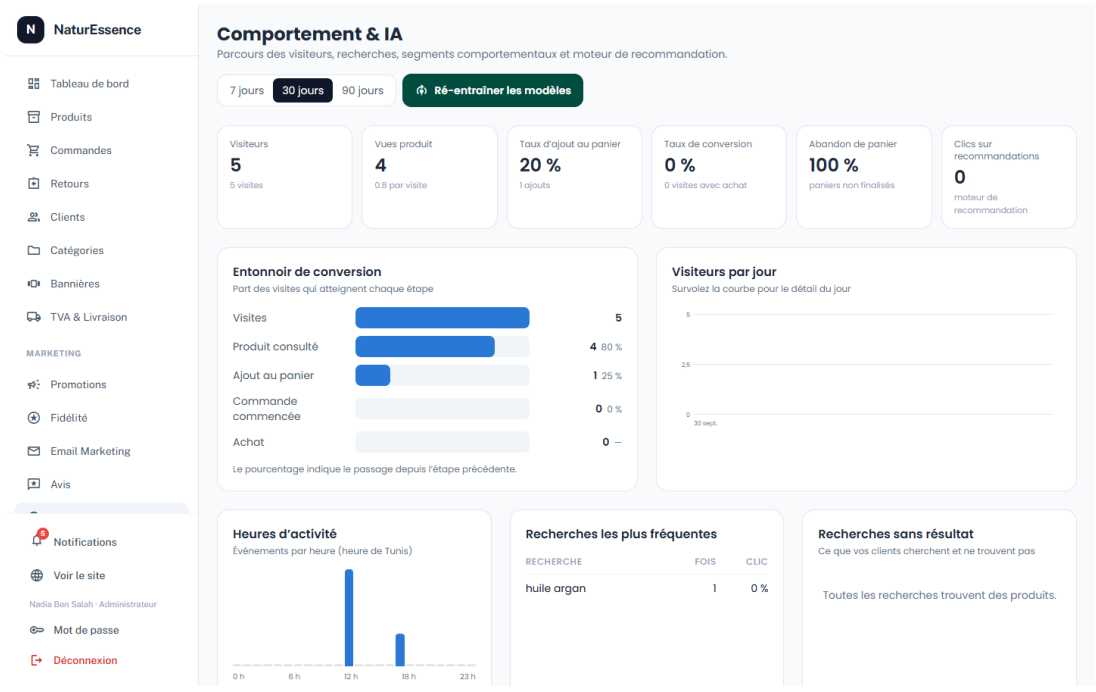


FIGURE 8.11 – Tableau de bord ń Comportement ż

Clients à risque : la figure 8.12 présente le niveau de risque d'attrition affiché dans la liste et la fiche des clients.

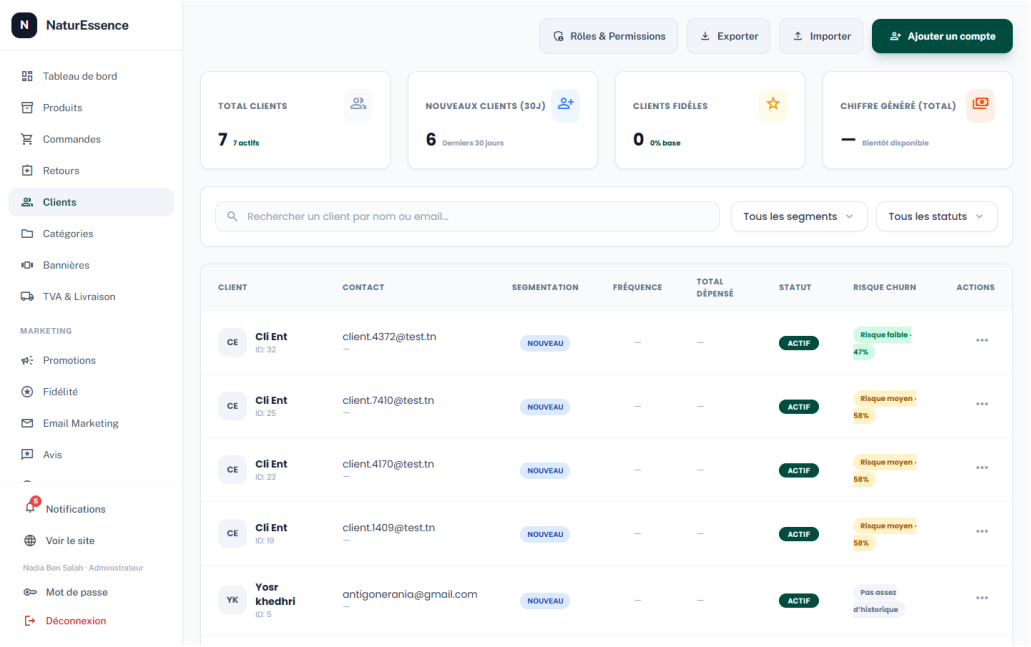


FIGURE 8.12 – Niveau de risque d'attrition des clients

Conclusion

Cette dernière release a doté Sellio d'une véritable intelligence comportementale : collecte d'événements pensée pour le volume, recommandations hybrides, segmentation et prédiction de l'attrition. Chaque modèle a été choisi par comparaison, évalué sur des données réelles avec un protocole temporel, et comparé à une référence simple. Les résultats sont volontairement présentés tels quels : ils montrent un gain réel mais mesuré, cohérent avec la difficulté des tâches.

Chapitre 9

Release 7 : Qualité, conteneurisation et intégration continue

Introduction

Les releases précédentes ont livré les fonctionnalités de Sellio. Cette dernière release s'attache à la manière de les livrer : une plateforme de six microservices, deux applications React et des modèles Python ne peut pas être vérifiée à la main à chaque modification. Le Sprint 12 met en place des tests automatisés sur les trois technologies du projet, conteneurise l'ensemble de la plateforme avec Docker et automatise sa construction, ses tests et la publication de ses images avec GitHub Actions.

9.1 Backlog du Sprint 12

Le tableau 9.1 présente le **backlog du Sprint 12**.

User Story	Tâches	Estimation
En tant que développeuse, je veux que les règles critiques soient vérifiées automatiquement	Tests JUnit de la sécurité multi-boutiques : isolation des boutiques, permissions d'équipe, périmètre de l'équipe Sellio, jetons JWT, `TenantGuard`.	1
	Tests JUnit de la validation de la personnalisation et des secteurs.	1
	Tests Vitest du front-end : ordre des sections, textes personnalisés, types de produits, synchronisation des registres vitrine et back-office.	1
	Tests pytest des modèles : recommandation hybride, segmentation, chargement et cohérence des modèles d'attrition.	1

Suite à la page suivante

User Story	Tâches	Estimation
En tant que développeuse, je veux lancer toute la plateforme en une commande	Externaliser la configuration (base de données, adresses des services dans la passerelle, CORS) en variables d'environnement.	1
	Écrire le `Dockerfile` multi-étapes des services Java, avec une cible dédiée au service d'analyse (Java et Python).	1
	Écrire le `Dockerfile` des deux front-ends (build Vite puis serveur nginx).	1
	Écrire le fichier `docker-compose.yml` (PostgreSQL et ses cinq bases, six services, deux front-ends, volumes).	1
En tant que développeuse, je veux que chaque modification soit construite et testée	Écrire le workflow GitHub Actions : trois tâches parallèles (backend, frontend, ML) avec cache des dépendances.	1
	Construire les huit images Docker une fois les tests passés, avec cache BuildKit.	1
	Publier les images sur GitHub Container Registry à chaque intégration dans `main`.	1
	Conserver les rapports de tests JUnit comme artefacts du pipeline.	1
En tant que développeuse, je veux un dépôt sain et des dépendances à jour	Configurer Dependabot (Maven, npm, pip, actions, images Docker).	1
	Figer les versions Python sur celles des modèles entraînés et commiter le fichier de verrouillage npm.	1
	Retirer les secrets du code source et nettoyer le dépôt (ancien monolithe, sorties de build, code mort).	1
	Total	**15**

Table 9.1 – Backlog du Sprint 12

9.2 Diagramme de cas d'utilisation du Sprint 12

La figure 9.1 présente le diagramme de cas d'utilisation de ce sprint. Les acteurs ne sont plus des utilisateurs de la plateforme mais ceux de sa chaîne de livraison : la développeuse, GitHub Actions et Dependabot.

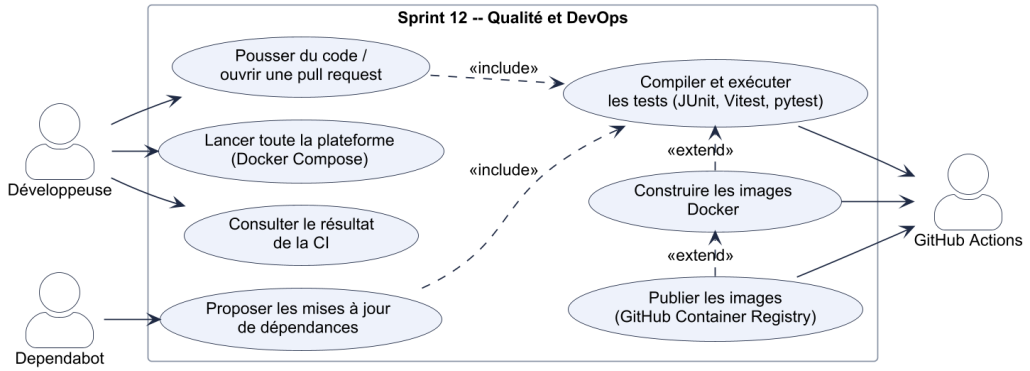


FIGURE 9.1 – Diagramme de cas d'utilisation du Sprint 12

9.3 Choix des outils

Le tableau 9.2 compare trois solutions d'intégration continue.

Critère	**GitHub Actions**	**GitLab CI**	**Jenkins**
Intégration au dépôt	Native (le code est sur GitHub)	Nécessite GitLab	Serveur et extensions à installer
Infrastructure	Machines fournies	Machines fournies	À héberger et maintenir
Registre d'images	GitHub Container Registry inclus	Registre inclus	Externe
Coût pour un dépôt public	Gratuit	Gratuit (quota)	Coût du serveur

TABLE 9.2 – Comparaison des solutions d'intégration continue

Le code de Sellio étant hébergé sur GitHub, **GitHub Actions** [16] s'impose : aucun serveur à maintenir, des machines Linux fournies, et le registre d'images intégré. **Docker** [15] est retenu pour la conteneurisation : chaque service embarque son environnement d'exécution (Java 17, ou Java 17 et Python 3.12 pour l'analyse), ce qui supprime les différences entre le poste de développement, la machine d'intégration continue et le serveur.

9.4 Stratégie de tests

Les tests portent en priorité sur ce qui serait le plus grave en cas d'erreur : la séparation des données entre boutiques, les droits de chaque rôle et les modèles d'apprentissage. Ce sont des tests unitaires rapides, sans base de données : l'ensemble s'exécute en moins de deux minutes. Le tableau 9.3 les résume.

Outil	Suite	Ce qui est vérifié	Tests
JUnit 5	`JwtAuthenticationFilterTest`	Un commerçant ne lit pas une autre boutique ; ses requêtes sont rattachées à sa boutique ; un membre d'équipe n'atteint que ses modules ; l'équipe Sellio n'atteint ni les clients, ni les commandes, ni les analyses ; un jeton falsifié reste anonyme.	5
JUnit 5	`JwtUtilTest`	Contenu du jeton (utilisateur, rôle, boutique, permissions), rejet des jetons falsifiés, signés avec une autre clé ou expirés.	4
JUnit 5	`TenantGuardTest`	Création toujours dans la boutique de l'appelant ; un enregistrement d'une autre boutique est introuvable (404).	3
JUnit 5	`ShopCatalogTest`	Secteurs acceptés et refusés, secteurs ń mode ż pour l'attrition, correspondance des anciens thèmes.	4
JUnit 5	`ShopSettingsValidatorTest`	Personnalisation valide conservée ; liens `javascript:` refusés même imbriqués ; taille maximale ; JSON invalide refusé.	5
Vitest	`sectionRegistry`, `copyTexts`	Ordre des sections, sections masquées, changement de modèle, textes personnalisés et valeurs par défaut.	11
Vitest	`sectors` (vitrine, back-office)	Types de produits, tailles et champs par type, absence de doublons, listes de valeurs complètes, choix de taille dans le panier.	10
Vitest	`homeSections`	Le back-office utilise exactement le registre et les textes de la vitrine.	3
pytest	`test_recommender`	Signal collaboratif, recommandation d'un produit neuf par son contenu, règles d'association, exclusion des produits déjà achetés.	6
pytest	`test_segmentation`, `test_churn_models`	Segments distincts trouvés ; seuil de visiteurs ; les deux modèles d'attrition se chargent et jugent plus risqué un client inactif.	5
		Total	**56**

TABLE 9.3 – Tests automatisés de Sellio

Le test `homeSections` illustre l'intérêt de cette démarche : la liste des sections et les textes par défaut existent à la fois dans la vitrine et dans le back-office. Si l'un des deux est modifié sans l'autre, le test échoue, au lieu qu'un commerçant découvre un éditeur qui ne correspond plus à sa vitrine.

9.5 Conteneurisation

9.5.1 Images

Le tableau 9.4 présente les huit images de la plateforme.

Image	Base d'exécution	Construction
auth, catalog, order, marketing, api-gateway	`eclipse-temurin:17-jre`	Étape de build Maven (`-pl service -am`), puis seul le JAR est copié ; exécution sous un utilisateur non privilégié.
analytics-service	`python:3.12-slim` et JRE 17	Mêmes étapes, plus les bibliothèques Python aux versions des modèles et le dossier `python/` (scripts et modèles).
storefront, backoffice	`nginx:1.27-alpine`	Build Vite avec Node 22, puis fichiers statiques servis par nginx (routes de l'application et cache des fichiers versionnés).

TABLE 9.4 – Images Docker de Sellio

Le `Dockerfile` des services est **multi-étapes** : la première étape contient Maven et le JDK, la seconde uniquement l'environnement d'exécution et le JAR, ce qui réduit la taille des images et leur surface d'attaque. Un seul fichier sert les six services grâce à un argument de construction (`SERVICE`) et à deux cibles (`service`, `analytics`).

9.5.2 Composition de la plateforme

Le fichier `docker-compose.yml` décrit la plateforme complète présentée à la figure 2.6 du chapitre 2 : PostgreSQL avec un volume persistant, un contrôle de santé et un script d'initialisation qui crée les cinq bases des microservices, les six services, chacun relié à sa base et qui attendent qu'elle soit prête, et les deux front-ends. Les secrets (clé JWT, Stripe, Brevo, Decart) ne sont jamais dans les images : ils sont lus au démarrage depuis le fichier `.env`, exclu du dépôt, dont un modèle documenté (`.env.example`) est versionné. Dans les conteneurs, la passerelle joint les services par leur nom (`http://auth-service:8081...`) grâce aux variables `*_SERVICE_URL`, alors qu'elle utilise `localhost` lorsqu'ils sont lancés depuis l'IDE : le même code fonctionne dans les deux cas.

9.6 Pipeline d'intégration et de déploiement continus

La figure 9.2 présente le pipeline exécuté à chaque *push* et à chaque *pull request*.

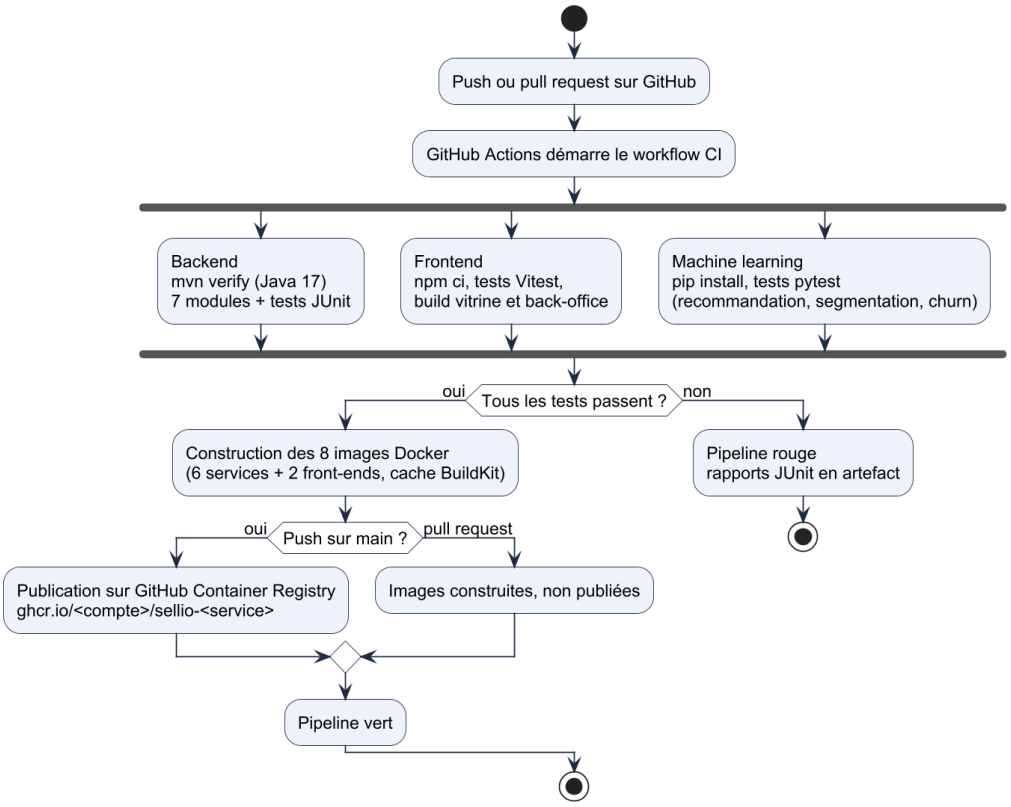


FIGURE 9.2 – Pipeline d'intégration et de déploiement continus

Les trois familles de tests s'exécutent en parallèle sur des machines Ubuntu 24.04 fournies par GitHub, avec mise en cache des dépendances Maven, npm et pip. Les images ne sont construites que si les trois réussissent ; elles sont alors publiées sur le registre lorsque la modification arrive sur la branche `main`. Le tableau 9.5 donne la durée de chaque tâche lors d'une exécution réelle.

Tâche	**Résultat**	**Durée**
Backend : Maven + JUnit	Succès	33 s
Frontend : Vitest + builds de production	Succès	30 s
Machine learning : pytest	Succès	30 s
8 images Docker (en parallèle)	Succès	18 à 32 s chacune
Pipeline complet	**Succès**	**1 min 12 s**

TABLE 9.5 – Durées d'une exécution du pipeline

La figure 9.3 replace ce pipeline dans la chaîne complète, du poste de développement jusqu'au serveur qui exécute les conteneurs.

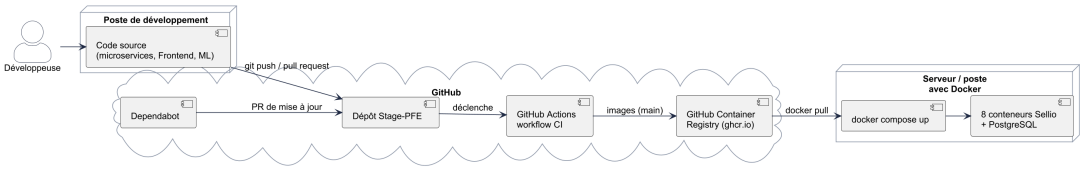


FIGURE 9.3 – Chaîne DevOps de Sellio

9.7 Réalisation du Sprint 12

Exécution du pipeline : la figure 9.4 présente une exécution réussie sur GitHub : les trois tâches de tests puis la matrice de huit constructions d'images.

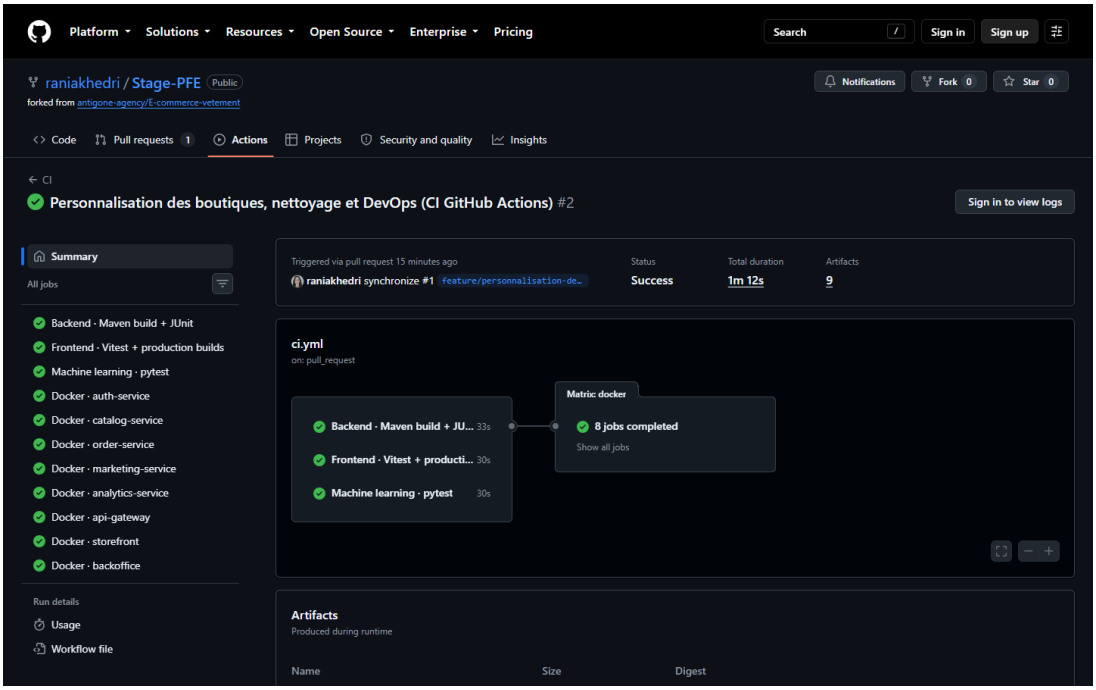


FIGURE 9.4 – Exécution du pipeline dans GitHub Actions

Historique des exécutions : la figure 9.5 présente l'historique des exécutions du workflow, une par modification poussée.

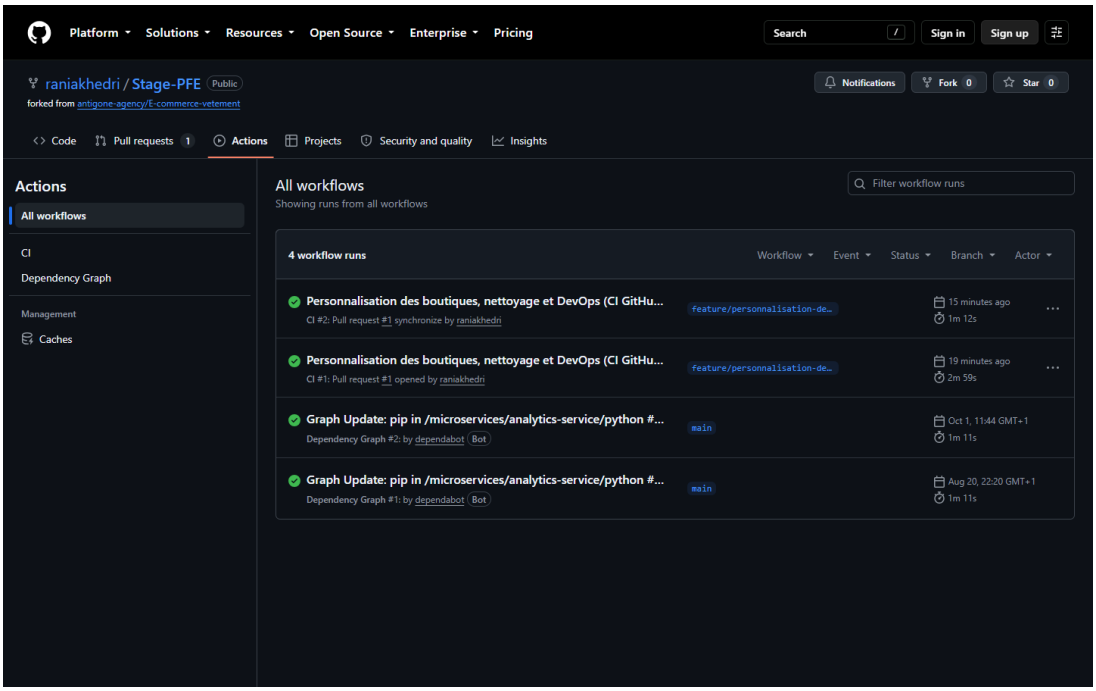


FIGURE 9.5 – Historique des exécutions du workflow CI

Conclusion

Cette release a rendu la livraison de Sellio reproductible : chaque modification est compilée et testée sur les trois technologies du projet, la plateforme se lance en une commande, et les images publiées sont exactement celles qui ont passé les tests. Les tests écrits portent sur les règles dont la violation aurait les conséquences les plus graves, en particulier l'isolation des données entre boutiques.

Conclusion et perspectives

Au terme de ce projet de fin d'études, nous avons conçu et développé **Sellio**, une plateforme transactionnelle intelligente qui permet à des commerçants de huit secteurs d'activité de créer, personnaliser et exploiter leur propre boutique en ligne. Partie d'une boutique unique, NaturEssence, la solution est devenue une plateforme multi-boutiques fondée sur une architecture microservices : cinq services Spring Boot derrière une passerelle, chacun propriétaire de sa base PostgreSQL, et deux applications React.

La plateforme couvre l'ensemble du cycle d'une boutique : intégration et vérification des commerçants (KYC par carte bancaire, pièce d'identité et selfie), catalogue dont la fiche s'adapte au secteur et au type de chaque produit, huit modèles de vitrine dont l'apparence et la page d'accueil se composent sans code avec un aperçu en direct, une isolation stricte des données de chaque boutique, jusqu'à ses clients, tunnel de commande avec paiement Stripe, gestion des commandes, des retours, de la TVA et de la livraison, promotions, e-mailing et programme de fidélité.

Elle se distingue par deux volets intelligents. Le premier est une **cabine d'essayage virtuel en temps réel**, fondée sur le modèle génératif *Lucy VTON* de Decart et intégrée de façon sécurisée par des jetons éphémères. Le second est un ensemble de **modèles d'apprentissage automatique** alimentés par le suivi comportemental des visiteurs : un moteur de recommandation hybride, une segmentation des visiteurs par K-Means et une prédiction de l'attrition adaptée au secteur de chaque boutique. Ces modèles ont été entraînés et évalués sur trois jeux de données publics réels, nettoyés par des scripts reproductibles, avec un découpage temporel et une comparaison systématique à une référence simple. Les gains obtenus sont réels et mesurés : +10 à +16 % de taux de réussite pour la recommandation, une AUC d'environ 0,80 pour l'attrition, des segments stables et interprétables.

La dernière release a rendu cette plateforme livrable de façon fiable : 56 tests automatisés vérifient en particulier l'isolation des boutiques et les droits de chaque rôle, la plateforme entière se lance en une commande grâce à Docker, et chaque modification est construite, testée et publiée par un pipeline GitHub Actions.

Sur le plan professionnel, ce projet nous a permis de consolider nos compétences en architecture distribuée, en sécurité des applications web, en intégration de services tiers (paiement, IA générative, e-mail), en ingénierie des données et en apprentissage automatique, tout en appliquant une démarche Agile fondée sur Scrum. Il nous a surtout appris qu'un modèle ne vaut que par la rigueur de son évaluation : un score présenté sans référence ni protocole ne dit rien.

Perspectives. Plusieurs évolutions peuvent être envisagées. Sur le plan des paiements, l'intégration d'un **prestataire de paiement local** prenant en charge le dinar tunisien, la confirmation des commandes par *webhook* et la revérification serveur des prix et du stock sont les prochaines étapes indispensables avant une mise en production. La vérification des commerçants pourrait être automatisée par une **reconnaissance faciale** comparant le selfie à la photo de la pièce d'identité, l'administrateur n'intervenant plus que sur les cas douteux.

Sur le plan de l'architecture, une communication asynchrone par messages entre les microservices (par exemple pour les événements de commande, aujourd'hui transmis par API internes), l'hébergement de chaque base sur son propre serveur, l'**orchestration des conteneurs** (Kubernetes) et le **déploiement continu dans le Cloud** à partir des images déjà publiées amélioreraient l'isolation, la disponibilité et la capacité de montée en charge. Sur le plan de l'intelligence, le modèle d'attrition pourrait être validé secteur par secteur (jeux Olist, REES46 multi-catégories, Instacart) avec un horizon propre à chacun, puis **ré-entraîné sur les données tunisiennes** dès que les boutiques auront accumulé plusieurs mois d'historique, et les recommandations évaluées en production par des **tests A/B** mesurant leur effet réel sur le chiffre d'affaires. Enfin, une **application mobile** pour les clients et pour les commerçants élargirait l'usage de la plateforme.

Ainsi, Sellio pourrait évoluer d'une plateforme de création de boutiques vers une véritable solution de commerce intelligent, locale, sécurisée et pilotée par les données de chaque commerçant.

Bibliographie

[1] Antigone. Antigone. https://antigoneagency.com/, 2026. Consulté en février 2026.

[2] Shopify Inc. Shopify. https://www.shopify.com/, 2026. Consulté en février 2026.

[3] Automattic. Woocommerce. https://woocommerce.com/, 2026. Consulté en février 2026.

[4] Wix.com Ltd. Wix ecommerce. https://www.wix.com/ecommerce/website, 2026. Consulté en février 2026.

[5] Converty. Converty – plateforme e-commerce pour le marché tunisien. https://converty.shop/fr, 2026. Consulté en juillet 2026.

[6] Ken Schwaber and Jeff Sutherland. The scrum guide. https://scrumguides.org/, 2020. Consulté en février 2026.

[7] Brevo. Brevo api – transactional emails. https://developers.brevo.com/, 2026.

[8] ISO/IEC. Iso/iec 25010 :2011 – systems and software quality requirements and evaluation (square) – system and software quality models. https://www.iso.org/standard/35733.html, 2011.

[9] The PostgreSQL Global Development Group. Postgresql – documentation (partitionnement, index brin, jsonb). https://www.postgresql.org/docs/, 2026. Consulté en mars 2026.

[10] Microsoft. Visual studio code. https://code.visualstudio.com/, 2026.

[11] GitHub Inc. Github. https://github.com/, 2026.

[12] PlantUML. Plantuml – diagrammes uml à partir de texte. https://plantuml.com/, 2026.

[13] The Apache Software Foundation. Apache maven. https://maven.apache.org/, 2026.

[14] Stripe, Inc. Stripe documentation – payment intents and setup intents. https://docs.stripe.com/, 2026. Consulté en mai 2026.

[15] Docker Inc. Docker documentation – multi-stage builds and docker compose. https://docs.docker.com/, 2026.

[16] GitHub. Github actions documentation. https://docs.github.com/actions, 2026.

[17] Oracle. Java se 17 documentation. https://docs.oracle.com/en/java/javase/17/, 2026.

[18] Mozilla. Javascript – mdn web docs. https://developer.mozilla.org/fr/docs/Web/JavaScript, 2026.

[19] Python Software Foundation. Python. https://www.python.org/, 2026.

[20] VMware Tanzu. Spring boot. https://spring.io/projects/spring-boot, 2026.

[21] VMware Tanzu. Spring cloud gateway. https://spring.io/projects/spring-cloud-gateway, 2026.

[22] Meta Open Source. React. https://react.dev/, 2026.

[23] Vite. Vite – next generation frontend tooling. https://vitejs.dev/, 2026.

[24] Tailwind Labs. Tailwind css. https://tailwindcss.com/, 2026.

[25] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, et al. Scikit-learn : Machine learning in Python. *Journal of Machine Learning Research*, 12 :2825–2830, 2011.

[26] Decart AI. Decart platform – realtime api and lucy vton. https://platform.decart.ai/, 2026. Consulté en juin 2026.

[27] The JUnit Team. Junit 5 user guide. https://junit.org/junit5/docs/current/user-guide/, 2026.

[28] Vitest. Vitest – next generation testing framework. https://vitest.dev/, 2026.

[29] pytest-dev. pytest documentation. https://docs.pytest.org/, 2026.

[30] Google. Mediapipe pose landmarker. https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker, 2026.

[31] Yisol Choi, Sangkyung Kwak, Kyungmin Lee, Hyungwon Choi, and Jinwoo Shin. Improving diffusion models for authentic virtual try-on in the wild (idm-vton). arXiv :2403.05139, 2024.

[32] Yuhao Xu, Tao Gu, Weifeng Chen, and Chengcai Chen. Ootdiffusion : Outfitting fusion based latent diffusion for controllable virtual try-on. arXiv :2403.01779, 2024.

[33] FASHN AI. Fashn api – virtual try-on. https://fashn.ai/, 2026.

[34] Daqing Chen. Online retail ii. UCI Machine Learning Repository, https://archive.ics.uci.edu/dataset/502/online+retail+ii, 2019. Licence CC BY 4.0.

[35] Michael Kechinov. ecommerce events history in cosmetics shop. Kaggle, https://www.kaggle.com/datasets/mkechinov/ecommerce-events-history-in-cosmetics-shop, 2020. Données ouvertes du projet REES46.

[36] H&M Group. H&m personalized fashion recommendations. Kaggle, https://www.kaggle.com/competitions/h-and-m-personalized-fashion-recommendations, 2022.

Résumé

Ce rapport présente le travail réalisé au sein d'Antigone dans le cadre de notre projet de fin d'études. Il porte sur la conception et le développement de Sellio, une plateforme transactionnelle intelligente de type SaaS qui permet à des commerçants de huit secteurs d'activité de créer et d'exploiter leur boutique en ligne. La plateforme repose sur une architecture microservices (Spring Boot, Spring Cloud Gateway, PostgreSQL) et des interfaces React. Elle intègre la vérification d'identité des commerçants, huit modèles de vitrine dont l'apparence et la page d'accueil se personnalisent avec un aperçu en direct, des fiches produit adaptées à chaque secteur, le paiement par Stripe, des outils marketing et de fidélisation, un essayage virtuel en temps réel fondé sur l'API Decart, ainsi qu'un suivi comportemental exploité par apprentissage automatique : recommandation hybride, segmentation des visiteurs et prédiction de l'attrition, évaluées sur des jeux de données publics réels. La plateforme est testée, conteneurisée avec Docker et livrée par un pipeline GitHub Actions. Le projet a été conduit selon la méthode Scrum.

Mots clés : e-commerce, SaaS, microservices, Spring Boot, React, Stripe, essayage virtuel, apprentissage automatique, système de recommandation, attrition, Docker, CI/CD, Scrum.

Abstract

This report presents the work carried out at Antigone as our end-of-studies project. It covers the design and development of Sellio, an intelligent transactional SaaS platform that lets merchants from eight business sectors create and run their own online store. The platform is built on a microservices architecture (Spring Boot, Spring Cloud Gateway, PostgreSQL) with React front ends. It includes merchant identity verification, eight storefront templates whose look and home page are customised with a live preview, sector-specific product sheets, Stripe payments, marketing and loyalty tools, real-time virtual try-on based on the Decart API, and behavioural tracking used by machine-learning models : hybrid recommendation, visitor segmentation and churn prediction, evaluated on real public datasets. The platform is tested, containerised with Docker and delivered by a GitHub Actions pipeline. The project followed the Scrum framework.

Keywords : e-commerce, SaaS, microservices, Spring Boot, React, Stripe, virtual try-on, machine learning, recommender system, churn, Docker, CI/CD, Scrum.